

THE ZCASH ARBORETUM

V

Ironwood Guide

The Zcash Orchard protocol and its Ironwood pool

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the Orchard protocol for a shielded payment in its Ironwood pool: how the payment is constructed, delivered to its recipient and verified; why a valid payment cannot create value, consume a note twice or consume a note without the authority of its owner; and what its public data hide. Every public field of the payment is constructed, in dependency order, as a fixed function of private notes, keys and randomness; the relation that each Action of the payment proves is stated over those objects; and each security property is a theorem under named assumptions. The sealed Orchard pool enters only where a rule shared with the Ironwood pool, or a rule that distinguishes the two pools, is needed to state the Ironwood-pool payment correctly. The volume assumes the *Math Guide* and the *Crypto Guide* and cites the *Halo 2 Guide* for the proof system. It is non-normative: the Zcash Protocol Specification and the Zcash Improvement Proposals are authoritative.

Contents

1	Introduction	5
1.1	Scope and status	5
1.2	The structure of a shielded payment	6
1.3	The Orchard protocol and its two pools	7
1.4	Requirements on a shielded payment	9
2	Notation and primitive instances	10
2.1	Fields, groups, and encodings	10
2.2	GroupHash, domain separation, and nothing-up-my-sleeve generators	13
2.3	Pseudorandom functions and field reductions	14
2.4	Sinsemilla: hash, commitment, and short forms	17
3	Keys and addresses	21
3.1	The spending key and the spend-side secrets	21
3.2	Viewing keys	23
3.3	Diversified addresses	26
3.4	Key capabilities and scope	28
4	Notes and note commitments	31
4.1	Notes	31
4.2	Note commitments	32
4.3	The note seed: Ironwood derivations (lead byte 0x03)	34
4.4	Actions, bundles, and dummy notes	36
5	The note commitment tree	38
5.1	The Merkle hash and the tree	38
5.2	Anchors	40
5.3	Authentication paths	43
6	Nullifiers	46
6.1	The nullifier	46
6.2	Nullifier sets	48
6.3	Nullifier chaining	48
6.4	Nullifier uniqueness and unlinkability	49
7	Spend authorisation	53
7.1	The RedPallas signature scheme	53
7.2	Randomised validating keys	56
7.3	RedPallas unforgeability	58

8	Value commitments and the binding signature	60
8.1	Value commitments	60
8.2	The balancing value	62
8.3	The binding signature	63
9	The Action statement and its proof	66
9.1	Bundle flags	66
9.2	The Action statement	67
9.3	The Action circuit and the Halo 2 proof	75
10	Note encryption	77
10.1	The note plaintext	77
10.2	Encryption to the recipient	78
10.3	The outgoing ciphertext	82
10.4	Trial decryption and note acceptance	84
11	Transactions and consensus rules	86
11.1	Action descriptions and bundles	87
11.2	The transaction format	90
11.3	Transaction digests and signatures	92
11.4	Chain state and pool rules	97
11.5	Construction of a transaction	99
11.6	Verification of a transaction	102
12	Security	104
12.1	Value conservation	106
12.2	Double-spend resistance	107
12.3	Authorisation of spends	109
12.4	Privacy	111
12.5	The end-to-end theorem	115

1 Introduction

This section fixes the subject of the volume, its status and its vocabulary before any construction: the scope; in plain language, the structure of a shielded payment and the division of its verification; the Orchard protocol and its two value pools; and the requirements that a shielded payment must meet.

1.1 Scope and status

This volume of *The Zcash Arboretum* constructs the Orchard protocol for a shielded payment in the Ironwood pool, both terms being defined in §1.3: how the payment is constructed, delivered and verified; why a valid payment cannot create value, consume a note twice, or consume a note without the authority of its owner; and what it hides. The sealed Orchard pool enters only where a rule shared with the Ironwood pool, or a rule that distinguishes the two pools, is needed to state the Ironwood-pool payment correctly. The volume assumes the *Math Guide* and the *Crypto Guide*, and cites the *Halo 2 Guide* for the proof system. It is non-normative: the Zcash Protocol Specification, cited as the protocol specification with the title of a section, and the Zcash Improvement Proposals are authoritative.

Definition 1.1 (Block chain, transaction, consensus rules). A *block chain* is a sequence of blocks that starts at the *genesis block*, the header of each block referring to the preceding block; the *height* of a block is its number of predecessors, so that the genesis block has height 0 (ZIP 200, Terminology; protocol specification, §“The Block Chain”). Each block contains an ordered sequence of one or more *transactions*, the units in which the chain records transfers of value (protocol specification, §“Transactions and Treestates”). The *consensus rules* are the validation rules that determine which block chains, and hence which blocks and transactions, are valid (ZIP 200, Terminology, “Consensus rule set”).

Definition 1.2 (ZIP). A *Zcash Improvement Proposal* (ZIP) is a design document that describes a feature of Zcash, or a change to it, with a concise technical specification and a rationale. Each ZIP carries a status. Two statuses are named in this volume: *Draft*, the status that every ZIP receives on submission, and *Reserved*, the status of a ZIP number that the ZIP editors have reserved while no ZIP text has been published (ZIP 0, Abstract and “ZIP Status Field”). A deployed rule is cited to a section of the protocol specification by its title, or to a ZIP and its section.

Definition 1.3 (Network upgrade). A *network upgrade* is an intentional change of the consensus rules that takes effect at a block height fixed for it, its *activation height*: blocks below that height are validated under the preceding rules, and blocks at or above it under the new rules. Network upgrades are named and activate in a fixed order (ZIP 200, Terminology and “Activation mechanism”). This volume names two: NU6.3, whose deployment ZIP is ZIP 258, and NU7, the upgrade that follows NU6.3, whose deployment ZIP is ZIP 259 (ZIP 258, Abstract; ZIP 259, Abstract and Rationale, “Rationale for no new transaction format”).

The volume describes the protocol as specified for NU7, whose deployment ZIP has status

Draft (ZIP 259, preamble). It names a network upgrade only where the name fixes which rule applies, gives no activation parameter, such as an activation height or a date, and describes no upgrade by what it adds or changes.

Definition 1.4 (Epistemic classes). Where the status of a claim about the deployed protocol needs stating, the volume places it in one of exactly two classes. The claim is *specified* when normative text of the protocol specification or of a ZIP fixes it. It is *designed but unspecified* when the deployed construction is designed to satisfy it but neither the protocol specification, nor a ZIP, nor a published proof establishes it for the deployed instance. An example of each class is given where its object is introduced: knowledge soundness of the Action proof in §1.2, and the cross-address restriction in §1.3.

1.2 The structure of a shielded payment

This subsection names, in plain language, the objects of a shielded payment and the checks that verify it. Each term receives one defining clause here and its construction in the section named.

Definition 1.5 (Notes, note commitments, nullifiers). A shielded payment transfers value held in *notes*. A note records, among other fields, an amount of value and the *address* of its recipient, a public value derived from the recipient’s keys (§3.3); no note is published in the clear (§4.1). For each note that a payment creates, the chain records a commitment to the note in the sense of the *Crypto Guide*, §“Hiding and binding”, its *note commitment*, in the extracted form constructed in §4.2. For each note that a payment consumes, the chain records a tag, the note’s *nullifier*, which marks the note as consumed (§6.1).

Definition 1.6 (Value pool). In this volume a *value pool* is a separately accounted collection of notes with its own record of note commitments, the *note commitment tree* (§5.1); its own record of nullifiers, the *nullifier set* (§6.2); and its own *chain value pool balance*, the total value that the chain accounts as held in the pool (§11.4).

Definition 1.7 (Action, dummy note). A shielded payment is carried by *Actions*. An Action consumes one note of a pool and creates one note of the same pool. A side of an Action that the payment does not use carries a *dummy note*, a note of value zero that pads the Action (§4.4; protocol specification, §“Action Transfers and their Descriptions” and §“Dummy Notes (Orchard)”).

Definition 1.8 (Bundle, balancing value). The Actions of one transaction in one pool, together with data common to them, form the transaction’s *bundle* for that pool; a transaction carries at most one bundle per pool (§4.4; ZIP 229, Abstract and “Transaction Format”). The *balancing value* of a bundle is the net value, public, that the bundle moves out of its pool into the rest of the transaction; it is negative when value enters the pool (§8.2).

Definition 1.9 (Note ciphertext). A created note reaches its recipient through the chain: the Action that creates it also carries a *note ciphertext*, an encryption of the note to the recipient’s address, from which the recipient recovers the note (§10).

Remark 1.10 (Division of verification). A *verifier*, a party that checks transactions against the consensus rules, accepts a bundle, without learning its notes, on one proof per bundle, two kinds of signature and two checks against the state of the chain (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”).

- (1) One zero-knowledge proof per bundle, the *Action proof* (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”; *Halo 2 Guide*, §“The Halo 2 proof system”): an argument that the prover knows, for each Action, private data that together with the Action’s public data satisfy one fixed relation, in the sense of the *Crypto Guide*, §“Languages, relations, and witnesses”, namely the Action statement, defined in “The Action statement” (§9.2). The proof system has no notion of value or ownership; the relation supplies both.
- (2) Per Action, a *spend-authorisation signature* (*Crypto Guide*, §“Syntax and security goal”), made with a signing key obtained from the *spend authorising key*, the signing secret associated with the consumed note’s address (§3.1), by which the holder of that key authorises the consumption (§7).
- (3) Per bundle, one *binding signature*, by which balance is checked: the total value of the notes that the bundle consumes, minus that of the notes it creates, equals its balancing value (§8.3).
- (4) Two checks against the state of the chain. The proof refers to one earlier state of the pool’s note commitment tree, through a digest of that state, the bundle’s *anchor* (§5.2), and the verifier checks that the anchor is one that the chain recorded for an earlier block. The verifier also checks that every nullifier that the bundle publishes is new to the pool (§6.2).

Both kinds of signature sign one digest of the transaction, constructed in “Transaction digests and signatures” (§11.3). The two checks of (4) lie outside the proof because they concern the history of the whole chain, which a proof over the data of one Action cannot attest.

Knowledge soundness of the Action proof as deployed (*Crypto Guide*, §“SNARKs: succinct non-interactive arguments of knowledge”), the property under which an accepted proof establishes the knowledge claimed in (1), is designed but unspecified: the volume takes it as Assumption 9.11 (§9.3), not as a result.

1.3 The Orchard protocol and its two pools

Definition 1.11 (Orchard protocol). The *Orchard protocol* is the cryptographic design shared by two value pools: its primitive instances, among them the Pallas and Vesta curves and the Sinsemilla hash (§2); its keys and addresses; its notes, note commitments and nullifiers; its note encryption; and the one relation that every Action proves, the Action statement, defined in “The Action statement” (§9.2), with the circuit designed to encode it (§9.3). ZIP 229 fixes this vocabulary (ZIP 229, Terminology), and the volume adopts it.

Definition 1.12 (Orchard pool, Ironwood pool). The *Orchard pool* and the *Ironwood pool* are the two value pools of the Orchard protocol. Each has its own note commitment tree, and hence its own anchors, its own nullifier set and its own chain value pool balance (ZIP 229, Terminology, and Rationale, “Separate state”).

The Ironwood pool admits net inflow, value moved into it by a negative balancing value (§11.4). The Orchard pool is *sealed*, in two respects (ZIP 258, “Consensus rules from NU6.3 activation”); each rule is stated where its object is constructed. First, no transaction increases the value held in the Orchard pool (§11.4). Second, every Orchard-pool Action creates its note at the address of the note it consumes (ZIP 229, Rationale, “enableCrossAddress polarity”), a rule made exact in “Bundle flags” (§9.1) and with the Action statement, defined in “The Action statement” (§9.2). ZIP 229, Abstract, states the effect of the second rule: outputs to the Orchard pool go to an address for which the creator of the transaction can authorise spends, which discourages transfers between users within the Orchard pool. Consensus does not prevent such transfers: any party able to authorise spends for an address can create an Orchard-pool note to that address, since the consumed side of the Action can be a dummy note at that address (ZIP 326, “Rationale for key-generation restrictions”; the witness of such an Action is exhibited in §9.2).

The restriction of an Action’s created note to the address of its consumed note, in force for every Orchard-pool Action and for every Ironwood-pool Action whose bundle does not enable cross-address transfers (§9.1), is specified: the protocol specification, §“Action Descriptions”, fixes the meaning of the flag that controls it, although ZIP 2006, which that section cites for it, has status Reserved (ZIP 2006, preamble).

Notes of the two pools have the same form and the same note commitment. They differ only in the note plaintext format, the encoding in which note encryption carries a note: an Ironwood-pool note uses the format whose first byte, the *lead byte* that names the format, is $0x03$, under which the note’s *commitment trapdoor*, the randomness of its note commitment, is derived by hashing a random seed, which the plaintext carries, together with every field that the commitment binds (the format and the lead-byte rule in §4.3, the plaintext in §10.1; ZIP 258, “ZIP 2005 activation”). Actions of both pools prove the same relation, the Action statement, defined in “The Action statement” (§9.2), with one circuit and one *verifying key*, the public parameter against which a proof is checked (§9.3; protocol specification, §“Action Descriptions”; ZIP 258, “Consensus rules from NU6.3 activation”). The pools are distinguished by their note commitment trees, nullifier sets and chain value pool balances, and by the component of the transaction that carries their bundles (§11.2), not by separate circuits (ZIP 229, Abstract, and Rationale, “Reuse of the Orchard protocol with minimal changes”).

Orchard-pool notes are spendable, in version-5 and version-6 transactions (§11.2), subject to the rules that seal the Orchard pool (ZIP 258, “Consensus rules from NU6.3 activation”; ZIP 259, Abstract).

The name “Orchard” alone qualifies the shared cryptography of the Orchard protocol; a statement that depends on the pool names the pool, the Orchard pool or the Ironwood pool.

1.4 Requirements on a shielded payment

Definition 1.13 (Transparent and shielded payments, observer). A *transparent payment* is one whose consumed and created amounts and addresses are public. The goal of a shielded payment is to achieve the effects of a transparent payment, namely transfer of value, exclusion of *double-spending*, the consumption of one note twice, and conservation of value, while hiding the contents of its notes, and the links between the notes it consumes and the notes it creates, from an *observer*: any party that reads the chain but holds no key of the notes concerned. Requirements R1 to R5 below make this precise. Each of R1 to R4 has a *soundness part*, a property of every valid payment, and a *hiding part*, a bound on what the observer learns; R5 has a soundness part only.

Definition 1.14 (Requirements R1 to R5).

- R1. *Hidden representation of value*. Soundness: the chain’s record of a created note binds that note, so that its owner can later prove possession of it and no party can present the record as the record of a different note. Hiding: the record reveals neither the recipient’s address nor the value, and does not distinguish a real note from a dummy note.
- R2. *Membership without identification*. Soundness: every consumed note of non-zero value was previously committed in the same pool, at the state of the pool’s record of commitments to which the payment refers. Hiding: the payment does not identify which committed note it consumes. A note of value zero, such as a dummy note, transfers no value, and R2 does not require it to have been committed.
- R3. *No second consumption*. Soundness: no note of a pool is consumed twice. Hiding: this holds without a public list of consumed notes; the data published when a note is consumed do not identify which committed note it is.
- R4. *Conservation*. Soundness: in every valid bundle the total value of the consumed notes equals the total value of the created notes plus the balancing value, so that no payment creates value. Hiding: no note’s value is disclosed; of the amounts, only the balancing value is public.
- R5. *Spend authority*, a soundness part only. Only the holder of a note’s spending authority, the spend authorising key associated with its address (§3.1), can consume the note; and no other party can alter what an authorised payment transfers, namely the notes it consumes and creates and the value it moves into or out of the pool, without invalidating it. Requirement R5 requires nothing of data that do not change what is transferred; which data the signatures fix is stated in “Transaction digests and signatures” (§11.3).

The requirement map, Table 5, given with the Action statement (§9.2), pairs the soundness and hiding parts of each requirement with the results that discharge them; “Security” (§12) states those results as theorems with their assumptions.

2 Notation and primitive instances

This section fixes the encodings and the primitive instances on which the later constructions rest, each with its Orchard parameters and domain separators. The underlying constructions belong to the *Math Guide* and the *Crypto Guide* and are cited, not re-derived; each named assumption is stated at the first construction whose security needs it.

2.1 Fields, groups, and encodings

The Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is the group of points of the curve $y^2 = x^3 + 5$ over the prime field $\mathbb{F}_{p_{\text{Pallas}}}$, with identity $\mathcal{O}$. It has prime order p_{Vesta} and cofactor 1, and $\mathbb{F}_{p_{\text{Vesta}}}$ is both its scalar field and the base field of the Vesta curve (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle” and §“Pallas and Vesta assembled”, Example “The Pasta curves”; *Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Definition “The Pasta curves”). The group, its order and the cycle with Vesta are cited, not re-derived; the moduli are those of the protocol specification, §“Pallas and Vesta”. Table 1 maps this notation, once, to the names used by the specification and by the lower volumes. Those names appear in the table only; later text writes only the column headed “This volume”. Maps defined on the Pallas group carry the specification’s subscript $\mathbb{P}$, as in $\text{Extract}_{\mathbb{P}}$.

Table 1. Notation for the fields, the Pallas group and the encodings, with the names of the protocol specification and of the *Math* and *Crypto Guides*. The spend validating key is constructed in §3.1; the point $\text{ak}^{\mathbb{P}}$ and the field element $\text{ak} = \text{Extract}_{\mathbb{P}}(\text{ak}^{\mathbb{P}})$ are kept apart throughout.

Object	This volume	Specification	Lower volumes
Pallas base-field modulus	p_{Pallas}	$q_{\mathbb{P}}$	p
Pallas scalar-field modulus, Vesta base-field modulus, order of the Pallas group	p_{Vesta}	$r_{\mathbb{P}}$	q
Pallas base field and scalar field	$\mathbb{F}_{p_{\text{Pallas}}}, \mathbb{F}_{p_{\text{Vesta}}}$	$\mathbb{F}_{q_{\mathbb{P}}}, \mathbb{F}_{r_{\mathbb{P}}}$	$\mathbb{F}_p, \mathbb{F}_q$
Pallas group	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$	$\mathbb{P}$	$E_p(\mathbb{F}_p)$
k -bit little-endian encoding of an integer	LE_k	l2LEBSP_k	—
Encoding of a point P	P^*	$\text{repr}_{\mathbb{P}}(P)$	—
Sinsemilla short commitment	ShortCommit	SinsemillaShortCommit	—
Spend validating key, as a point and as its x -coordinate	$\text{ak}^{\mathbb{P}}, \text{ak}$	$\text{ak}^{\mathbb{P}}, \text{ak}$	—

Scalar multiplication is written $[k]P$, for $k \in \mathbb{F}_{p_{\text{Vesta}}}$ or an integer read modulo p_{Vesta} . The symbol $\parallel$ denotes concatenation of bit strings or of byte strings, and ε the empty string. Byte-string literals and tags are set in monospace, as in `z.cash:Orchard`, and single bytes as `0x...`. The symbol $\perp$ is the failure value of a partial map; a map that may fail takes values in $X \cup \{\perp\}$ for its codomain X .

Both moduli have bit length 255, and $p_{\text{Pallas}} < p_{\text{Vesta}}$:

$$2^{254} < p_{\text{Pallas}} < p_{\text{Vesta}} < 2^{255}.$$

Two consequences are used later. First, the integer representative of an element of either field lies in $\{0, \dots, 2^{255} - 1\}$ and has a 255-bit encoding, which leaves the top bit of a 256-bit string free.

Second, the integer representative of an element of $\mathbb{F}_{p_{\text{Pallas}}}$ lies in $\{0, \dots, p_{\text{Vesta}} - 1\}$; reading it as a Pallas scalar is therefore an injective, not a surjective, map $\mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Vesta}}}$. This map is used wherever a base-field element multiplies a point (§3.3, §6.1).

Definition 2.1 (Integer and byte encodings). For $k \geq 0$ and an integer $m \in \{0, \dots, 2^k - 1\}$, the k -bit little-endian encoding $\text{LE}_k(m) \in \{0, 1\}^k$ is the bit sequence $b_0 b_1 \dots b_{k-1}$ with $m = \sum_{i=0}^{k-1} b_i 2^i$. The map $\text{LE}_k: \{0, \dots, 2^k - 1\} \rightarrow \{0, 1\}^k$ is a bijection with inverse LE_k^{-1} . A field element is encoded through its integer representative in $\{0, \dots, p - 1\}$, for p the modulus. In the conversion names below, I stands for integer, LE for little-endian, BS for bit string, and OS for octet string (a sequence of eight-bit bytes). The digit 2 is read “to”, and P stands for primitive, a basic conversion operation. The subscript k counts bits, not bytes. Three maps relate integers, bit strings and byte strings:

- the map I2LEOSP_k sends $m \in \{0, \dots, 2^k - 1\}$ to its $\lceil k/8 \rceil$ -byte little-endian encoding;
- for k a multiple of 8, the map LEOS2IP_k sends a $k/8$ -byte string $(s_0, \dots, s_{k/8-1})$ to the integer $\sum_i s_i 256^i \in \{0, \dots, 2^k - 1\}$;
- the map $\text{LEBS2OSP}_k: \{0, 1\}^k \rightarrow \{0, \dots, 255\}^{\lceil k/8 \rceil}$ pads its input on the right with zero bits to a multiple of 8 bits, packs each group of 8 bits into a byte, least significant bit first, and keeps the order of the groups.

Hence $\text{LEBS2OSP}_k(\text{LE}_k(m)) = \text{I2LEOSP}_k(m)$. The byte form is needed wherever a bit string enters BLAKE2b or GroupHash (protocol specification, §“Integers, Bit Sequences, and Endianness”).

The widths used later are LE_{10} (the Merkle layer tag, §5.1), LE_{64} (the note value, §4.2), LE_{88} (the diversifier index, §3.3), LE_{255} (field elements) and LE_{256} (the star encoding below; the key encoding of r_{ivk} , §3.2).

Definition 2.2 (Coordinate extraction). Let $\text{Extract}_P: \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$ be

$$\text{Extract}_P(\mathcal{O}) := 0, \quad \text{Extract}_P((x, y)) := x.$$

Its lifting $\text{Extract}_P^\perp: \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\} \rightarrow \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$ maps $\perp$ to $\perp$ and agrees with Extract_P on points (protocol specification, §“Coordinate Extractor for Pallas”).

The map Extract_P supplies a field element wherever a later construction needs one in place of a point: the field element ak (§3.1), the extracted note commitment cmx (§4.2), the node values of the note commitment tree (§5.1) and the nullifier nf (§6.1). It is two-to-one on non-identity points, by Lemma 2.4 and because $P \neq -P$ for $P \neq \mathcal{O}$ in a group of odd order; every binding argument over an extracted coordinate treats the opposite point explicitly.

Lemma 2.3 (Extract vanishes only at the identity). For $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, $\text{Extract}_P(P) = 0$ if and only if $P = \mathcal{O}$.

Proof. If $P = \mathcal{O}$, then $\text{Extract}_P(P) = 0$ by definition. A point $(0, y)$ would satisfy $y^2 = 5$ in $\mathbb{F}_{p_{\text{Pallas}}}$,

but 5 is a quadratic non-residue modulo p_{Pallas} (*Math Guide*, §“Quadratic residues and the Euler criterion”, Example “5 is a non-square in the Pallas base field”). Hence no point other than $\mathcal{O}$ has extracted coordinate 0. $\square$

Lemma 2.4 (Coordinate extraction identifies opposite points). *For $P, P' \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, $\text{Extract}_p(P) = \text{Extract}_p(P')$ if and only if $P' = P$ or $P' = -P$.*

Proof. Negation is $-\mathcal{O} = \mathcal{O}$ and $-(x, y) = (x, -y)$ (*Math Guide*, §“Explicit affine formulas”, Definition “Negation”), so $\text{Extract}_p(-P) = \text{Extract}_p(P)$ for every P , which is the reverse implication. For the forward implication let $\text{Extract}_p(P) = \text{Extract}_p(P')$. If $P = \mathcal{O}$, then $\text{Extract}_p(P') = 0$, and Lemma 2.3 gives $P' = \mathcal{O} = -P$. If $P = (x, y)$, then $x \neq 0$ by Lemma 2.3, so $P' \neq \mathcal{O}$ and $P' = (x, y')$ with $y'^2 = x^3 + 5 = y^2$; hence $y' = \pm y$ and $P' = \pm P$. $\square$

Definition 2.5 (Star encoding). For $P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ the *star encoding* $P^* \in \{0, 1\}^{256}$ is

$$\mathcal{O}^* := \text{LE}_{256}(0), \quad (x, y)^* := \text{LE}_{256}(x + 2^{255}(y \bmod 2)),$$

with x and y read as integer representatives: the 255-bit x -coordinate fills the low bits and the parity of y the top bit. The partial inverse $\text{abst}_p: \{0, 1\}^{256} \rightarrow \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$ reads the first 255 bits of s as $\tilde{x} := \text{LE}_{255}^{-1}(s_0 \dots s_{254})$ and the last bit as the sign bit $\tilde{y} := s_{255}$, and returns

1. the failure value $\perp$ if $\tilde{x} \geq p_{\text{Pallas}}$; otherwise let $x := \tilde{x} \in \mathbb{F}_{p_{\text{Pallas}}}$;
2. the identity $\mathcal{O}$ if $x = 0$ and $\tilde{y} = 0$;
3. the failure value $\perp$ if $x^3 + 5$ is not a square in $\mathbb{F}_{p_{\text{Pallas}}}$;
4. otherwise the point (x, y) with $y^2 = x^3 + 5$ and $y \bmod 2 = \tilde{y}$

(protocol specification, §“Pallas and Vesta”).

The encoding is canonical: $\text{abst}_p(P^*) = P$ for every point P , and $\text{abst}_p(s) \neq \perp$ implies $\text{abst}_p(s)^* = s$. For the first claim, $\mathcal{O}^*$ decodes to $\mathcal{O}$ at step 2. A point (x, y) has $x \neq 0$ (Lemma 2.3), and $y \neq 0$ because a point with $y = 0$ has order 2, which a group of odd order does not contain (*Math Guide*, §“Torsion, the cofactor, and prime-order subgroups”, Remark “No two-torsion on the Pasta curves”). The two square roots $\pm y$ are therefore non-zero and, p_{Pallas} being odd, of opposite parities, so step 4 returns (x, y) . For the second claim, a string decoded at step 2 is $\text{LE}_{256}(0) = \mathcal{O}^*$, and a string decoded at step 4 to (x, y) has low bits $\text{LE}_{255}(x)$ and top bit $y \bmod 2$, so it equals $(x, y)^*$. Consequently $P \mapsto P^*$ is injective, each point has exactly one valid encoding, and every other string decodes to $\perp$, among them $\text{LE}_{256}(2^{255})$ (whose x -part is 0 and sign bit 1) and every string whose x -part is at least p_{Pallas} . The star encoding is the form in which a point enters a hash or a commitment as a bit string.

2.2 GroupHash, domain separation, and nothing-up-my-sleeve generators

Construction 2.6 (GroupHash on Pallas). For byte strings D (the domain separator) and M (the message), the point $\text{GroupHash}(D, M) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ is computed as follows.

1. Let

$$\text{DST} := D \parallel - \parallel \text{pallas} \parallel \text{_XMD:BLAKE2b_SSWU_RO_};$$

the map is defined when DST has at most 255 bytes, which holds for every domain separator of Table 2.

2. The map `hash_to_field` expands (M, DST) by the function `expand_message_xmd` over BLAKE2b-512 into two 64-byte strings. BLAKE2b runs with zero personalisation; the string DST, followed by a byte carrying its length, is part of the hashed input. Each string is read as a big-endian integer and reduced modulo p_{Pallas} , giving $u_0, u_1 \in \mathbb{F}_{p_{\text{Pallas}}}$.
3. The simplified Shallue–van de Woestijne–Ulas map sends each u_i to a point Q_i of a curve 3-isogenous to Pallas.
4. The output is the image of $Q_0 + Q_1$ under the 3-isogeny onto Pallas.

No cofactor is cleared, since the Pallas group has prime order. The isogenous curve, the isogeny coefficients and the constant of the map are those of the protocol specification, §“Group Hash into Pallas and Vesta”, and are not reproduced. The map is deterministic, public and efficiently computable (*Crypto Guide*, §“Hashing to a field element”, Construction “Expand-then-reduce”; §“Hashing to a curve point”, Constructions “Simplified Shallue–van de Woestijne–Ulas map” and “The hash-to-curve pipeline”, Remark “The isogeny required by j -invariant 0”).

Definition 2.7 (Domain separator and fixed generator). A *domain separator* is a fixed ASCII byte string naming a use site; it is the first argument of GroupHash, so that distinct uses draw distinct generators. A *fixed generator* is the value of GroupHash on a public, fixed pair (D, M) . It is nothing-up-my-sleeve: the public pair determines it, and no party chose it (*Crypto Guide*, §“Hashing to a curve point”, Construction “Nothing-up-my-sleeve generators”). The strings are fixed by the protocol specification.

Assumption 2.8 (GroupHash as a random oracle). In security arguments GroupHash is modelled as a random oracle on pairs (D, M) with values in $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”, with the group in place of bit strings): distinct pairs receive independent uniform points. In particular the fixed generators of Table 2 are, in the model, independent uniform points. The assumption is a heuristic about the concrete map, supported by its design for indistinguishability from a random oracle (protocol specification, §“Group Hash into Pallas and Vesta”, note); it is not a theorem. Every later binding, collision, uniqueness or unlinkability result that needs it names it.

Table 2 lists every GroupHash input of the Orchard protocol, with the object it defines and the section that uses it. The messages $\mathfrak{G}$, $\mathfrak{K}$, $\mathfrak{v}$ and $\mathfrak{r}$ are one-byte ASCII strings, and the Sinsemilla

messages are the ASCII strings shown. The rows are those of the protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”, §“Computing ρ values and Nullifiers”, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, §“Sinsemilla Hash Function”, §“Sinsemilla commitments”, §“MerkleCRH^{Orchard} Hash Function” and §“Homomorphic Pedersen commitments (Sapling and Orchard)”.

Table 2. Every input of GroupHash in the Orchard protocol. The hash base of MerkleCRH is derived from its domain without the suffix -M, and MerkleCRH has no blinding base. The table entries $S(j)$ are shared by every Sinsemilla domain.

Domain separator D	Message M	Object	Used in
z.cash:Orchard	G	$G^{\text{SpendAuth}}$, spend-authorisation base	§3.1, §7.1
z.cash:Orchard	K	K^{Orchard} , nullifier base	§6.1
z.cash:Orchard-gd	LEBS2OSP ₈₈ (d)	g_d for a diversifier d	§3.3
z.cash:Orchard-gd	ε	g_d if the row above gives $\mathcal{O}$	§3.3
z.cash:SinsemillaQ	z.cash:Orchard-NoteCommit-M	hash base of NoteCommit	§4.2
z.cash:Orchard-NoteCommit-r	ε	blinding base of NoteCommit	§4.2
z.cash:SinsemillaQ	z.cash:Orchard-CommitIvk-M	hash base of Commit ^{ivk}	§2.4
z.cash:Orchard-CommitIvk-r	ε	blinding base of Commit ^{ivk}	§2.4
z.cash:SinsemillaQ	z.cash:Orchard-MerkleCRH	hash base of MerkleCRH	§5.1
z.cash:Orchard-cv	v	V^{Orchard} , value base	§8.1
z.cash:Orchard-cv	r	R^{Orchard} , randomness base	§8.1
z.cash:SinsemillaS	l2LEOSP ₃₂ (j)	table entry $S(j)$, $0 \leq j < 2^{10}$	§2.4

Remark 2.9 (Domain separation). The pairs of Table 2 are pairwise distinct, and the map from (D, M) to the input hashed by hash_to_field is injective, because DST is closed by a byte carrying its length. Under Assumption 2.8 the table’s generators are therefore independent, and a relation found in one context transfers to no other (*Crypto Guide*, §“Domain separation and personalisation”, Proposition “Domain separation yields independent oracles”). The *Crypto Guide*’s inventory of Orchard tags and personalisations in the same section is cited, not repeated.

2.3 Pseudorandom functions and field reductions

Definition 2.10 (Pseudorandom function). A keyed family $(F_k)_k$ of functions is a *pseudorandom function* (PRF) if no efficient adversary with adaptive oracle access distinguishes F_k , for a secret uniform key k , from a uniformly random function with the same domain and range, except with negligible advantage (*Crypto Guide*, §“Pseudorandom functions and permutations”, Definition “Pseudorandom function”).

Construction 2.11 (Expansion function). For a key $k \in \{0, 1\}^{256}$ and a byte string t ,

$$\text{PRFexpand}_k(t) := \text{BLAKE2b-512}(\text{zcash_ExpandSeed}, \text{LEBS2OSP}_{256}(k) \parallel t),$$

a 64-byte string, identified with $\{0, 1\}^{512}$; here $\text{BLAKE2b-512}(P, x)$ is unkeyed BLAKE2b with a 64-byte output, the 16-byte personalisation P and the input x . A key given as a 32-byte string

enters as is. The key is thus carried in the message, not in BLAKE2b’s native keyed mode (protocol specification, §“Pseudo Random Functions”), so the *Crypto Guide*’s PRF argument for native keying (§“PRFs from hash functions: length extension and keyed BLAKE2”) does not cover this construction; Assumption 2.12 states the property directly. The key is written as a subscript; the leading byte of t names the derived quantity (protocol specification, §“Pseudo Random Functions”).

Assumption 2.12 (Pseudorandomness of PRF expansion). For k uniform on $\{0, 1\}^{256}$, the function $t \mapsto \text{PRFexpand}_k(t)$ from byte strings to $\{0, 1\}^{512}$ is a PRF in the sense of the preceding definition. This is the security requirement that the protocol specification, §“Pseudo Random Functions”, places on PRFexpand .

Definition 2.13 (Field reductions). For $x \in \{0, 1\}^{512}$, read as 64 bytes,

$$\begin{aligned} \text{ToBase}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Pallas}} \in \mathbb{F}_{p_{\text{Pallas}}}, \\ \text{ToScalar}(x) &:= \text{LEOS2IP}_{512}(x) \bmod p_{\text{Vesta}} \in \mathbb{F}_{p_{\text{Vesta}}} \end{aligned}$$

(protocol specification, §“Orchard Key Components”).

For x uniform on $\{0, 1\}^{512}$, the statistical distance of $\text{ToBase}(x)$ from the uniform distribution on $\mathbb{F}_{p_{\text{Pallas}}}$ is at most $p_{\text{Pallas}}/2^{512}$, and that of $\text{ToScalar}(x)$ from the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$ at most $p_{\text{Vesta}}/2^{512}$ (*Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with $L = 512$). Both bounds are below 2^{-257} .

Construction 2.14 (Domain bytes of PRF expansion). Every later use of PRFexpand derives one quantity as

$$y := f(\text{PRFexpand}_k(b \parallel t')),$$

where the lead byte b , the key κ , the remainder t' of the input and the reduction $f \in \{\text{ToBase}, \text{ToScalar}\}$, or no reduction, form one row of Table 3. Each row’s objects are defined in the section it names (protocol specification, §“Orchard Key Components” for $0x06$, $0x07$, $0x08$ and $0x82$; §“Sending Notes (Orchard)” for $0x04$, $0x09$ and $0x0B$).

Lemma 2.15 (Independence of domain-separated expansions). *Let k be uniform on $\{0, 1\}^{256}$ and used only as a PRFexpand key. Let an efficient algorithm choose, possibly adaptively, pairwise distinct inputs $t_1, \dots, t_n$, in particular inputs with pairwise distinct leading bytes, and receive the answers. Under Assumption 2.12, the answers $(\text{PRFexpand}_k(t_1), \dots, \text{PRFexpand}_k(t_n))$ are computationally indistinguishable from n independent uniform elements of $\{0, 1\}^{512}$. Consequently, for reductions $f_i \in \{\text{ToBase}, \text{ToScalar}\}$, the answers $(f_i(\text{PRFexpand}_k(t_i)))_i$ are computationally indistinguishable from independent uniform elements of the target fields, up to an additional statistical distance of at most the sum of the reduction distances. For a key within statistical distance δ of uniform, both conclusions hold with δ added.*

Proof. One hybrid replaces PRFexpand_k by a uniformly random function; the distinguishing advantage changes by at most the PRF advantage of the algorithm that runs the given one and for-

Table 3. Every lead byte of PRFexpand used in this volume. The row 0x82 applies no reduction: its 64-byte output splits into dk, the first 32 bytes, and ovk, the last 32 bytes.

b	Key κ	Remainder t'	f	Output	Section
0x06	sk	ε	ToScalar	ask	§3.1
0x07	sk	ε	ToBase	nk	§3.1
0x08	sk	ε	ToScalar	r_{ivk}	§3.1
0x82	$LE_{256}(r_{ivk})$	$I2LEOSP_{256}(ak) \parallel I2LEOSP_{256}(nk)$	none	dk $\parallel$ ovk	§3.2
0x04	rseed	$I2LEOSP_{256}(\rho)$	ToScalar	esk	§4.3
0x09	rseed	$I2LEOSP_{256}(\rho)$	ToBase	ψ	§4.3
0x0B	rseed	$LEBS2OSP_{256}(g_d^*) \parallel LEBS2OSP_{256}(pk_d^*) \parallel I2LEOSP_{64}(v) \parallel I2LEOSP_{256}(\rho) \parallel I2LEOSP_{256}(\psi)$	ToScalar	rcm	§4.3

wards its queries to its own oracle (Assumption 2.12; adaptive queries are within the definition, *Crypto Guide*, §“Pseudorandom functions and permutations”, Remark “Necessity of adaptivity”). A random function, sampled lazily, answers each new input with a fresh uniform value, so its answers on pairwise distinct inputs are independent and uniform, also when each input depends on earlier answers. For the reduced answers, replacing each f_i of a fresh uniform string by a fresh uniform field element changes the distribution of the whole interaction by at most the reduction distance of f_i ; summing over i bounds the total by the triangle inequality, and the post-processing by the algorithm does not increase it (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”). A key within statistical distance δ of uniform changes the interaction, a function of the key and the algorithm’s coins, by at most δ , by the same theorem. $\square$

The hypothesis of Lemma 2.15 holds for the spending key sk, and for rseed towards an adversary that sees rseed only through PRFexpand outputs; the note plaintext, which contains rseed, is treated in §10.2. It does not hold for the row 0x82, whose key encodes the trapdoor r_{ivk} , which is also used in $Commit^{ivk}$; that row is covered by Assumption 3.13 in §3.2. The lemma is used in §3.1, in Lemma “Rejection in key generation” (§3.2), in Propositions 3.18, 3.17, 4.8, 6.10 and 10.6, in Proposition “Confidentiality of the outgoing ciphertext” (§10.3), in Theorems 12.9 and 12.12, and in Remark “Hypothesis (H) for honest keys” (§11.3).

Remark 2.16 (Hash roles). Each hash is defined at its construction site. BLAKE2b-512 with personalisation Zcash_ExpandSeed serves PRFexpand (this subsection); BLAKE2b-512 without personalisation serves hash_to_field inside GroupHash (§2.2); personalised BLAKE2b-512 serves the RedPallas hash H (§7.1); BLAKE2b-256 serves the note-encryption key derivation and the outgoing cipher key (§10.2, §10.3); BLAKE2b serves the transaction and signature digests (§11.3); Sinsemilla serves MerkleCRH, NoteCommit and $Commit^{ivk}$ (§2.4); Poseidon serves the nullifier PRF, constructed with the nullifier (§6.1). The personalisations of PRFexpand, the note-encryption key derivation and the outgoing cipher key are those of the *Crypto Guide*’s inventory (§“PRFs from hash functions: length extension and keyed BLAKE2”, Remark “Zcash’s use of keyed and personalised BLAKE2”); every personalisation is stated at its construction.

2.4 Sinsemilla: hash, commitment, and short forms

Construction 2.17 (Sinsemilla hash on Pallas). The *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Construction “Sinsemilla hash”, fixes the chunk width $k = 10$ (written w there, where k counts the pieces), the table and the domain base

$$\begin{aligned} S(j) &:= \text{GroupHash}(\text{z.cash:SinsemillaS}, \text{l2LEOSP}_{32}(j)) \quad (0 \leq j < 2^k), \\ Q(D) &:= \text{GroupHash}(\text{z.cash:SinsemillaQ}, D) \end{aligned}$$

for a byte string D , and the accumulator; it writes $\text{LE}_{32}(j)$ for the four-byte encoding $\text{l2LEOSP}_{32}(j)$. The Orchard instance adds the following (protocol specification, §“Sinsemilla Hash Function”).

1. The *chunk bound* c is the largest integer with $2^c \leq (p_{\text{Vesta}} - 1)/2$; thus $c = 253$, and a message has at most $kc = 2530$ bits.
2. *Incomplete addition* $+$ on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}$ returns $\perp$ if either operand is $\perp$; for points it returns $\perp$ in an *exceptional case*, when an operand is $\mathcal{O}$ or the operands have equal x -coordinates (equal or opposite points), and their group sum otherwise (*Math Guide*, §“Explicit affine formulas”, Remark “Complete versus incomplete addition”).
3. For a byte string D and a bit string M of length at most kc , let $n := \lceil |M|/k \rceil \leq c$, pad M with zero bits to nk bits, split the result into k -bit pieces, and let m_i be LE_k^{-1} of the i -th piece, $1 \leq i \leq n$. Then

$$\text{Acc}_0 := Q(D), \quad \text{Acc}_i := (\text{Acc}_{i-1} + S(m_i)) + \text{Acc}_{i-1} \quad (1 \leq i \leq n),$$

and $\text{SinsemillaHashToPoint}(D, M) := \text{Acc}_n$. The output is $\perp$ exactly when some incomplete addition meets an exceptional case.

4. The extracted hash is

$$\text{SinsemillaHash}(D, M) := \text{Extract}_{\mathbb{P}}^{\perp}(\text{SinsemillaHashToPoint}(D, M)) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}.$$

Construction 2.18 (Sinsemilla commitment and short commitment). For a byte string D , a bit string M of length at most kc and a trapdoor $r \in \mathbb{F}_{p_{\text{Vesta}}}$, let

$$\hat{M} := \text{SinsemillaHashToPoint}(D \parallel -\mathbb{M}, M), \quad H_D := \text{GroupHash}(D \parallel -r, \varepsilon).$$

Then, with complete group addition,

$$\text{SinsemillaCommit}_r(D, M) := \begin{cases} \hat{M} + [r]H_D & \text{if } \hat{M} \neq \perp, \\ \perp & \text{otherwise,} \end{cases}$$

$$\text{ShortCommit}_r(D, M) := \text{Extract}_{\mathbb{P}}^{\perp}(\text{SinsemillaCommit}_r(D, M)) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$$

(protocol specification, §“Sinsemilla commitments”). The domain D is routed to the hash as $D \parallel -M$ and to the blinding base as $D \parallel -r$ with the empty message. The *Crypto Guide* constructs the commitment with a generic independent blinding generator (§“Sinsemilla: an algebraic hash-based commitment”, Construction “SinsemillaCommit”); the routing and the $\perp$ case are the Orchard additions.

The unblinded SinsemillaHash is deterministic and therefore not hiding; ShortCommit is its blinded counterpart. Proposition 2.24 and Lemma 2.26 state the security of both.

Construction 2.19 (Orchard instances). The Orchard protocol uses three Sinsemilla instances, each with one fixed message length per domain, all within c chunks (Table 4): MerkleCRH, a SinsemillaHash instance whose layout is given in §5.1; NoteCommit, a SinsemillaCommit instance whose layout is given in Definition 4.4; and $\text{Commit}^{\text{ivk}}$, a ShortCommit instance defined next.

Table 4. The Orchard Sinsemilla instances. Each domain hashes messages of the single length shown, of at most $c = 253$ chunks of $k = 10$ bits.

Instance	Form	Domain	Message bits	Chunks
MerkleCRH	SinsemillaHash	z.cash:Orchard-MerkleCRH	$10 + 2 \cdot 255 = 520$	52
NoteCommit	SinsemillaCommit	z.cash:Orchard-NoteCommit	$2 \cdot 256 + 64 + 2 \cdot 255 = 1086$	109
$\text{Commit}^{\text{ivk}}$	ShortCommit	z.cash:Orchard-CommitIvk	$2 \cdot 255 = 510$	51

Definition 2.20 (The key commitment). For $x, y \in \mathbb{F}_{p_{\text{Pallas}}}$ and $r_{\text{ivk}} \in \mathbb{F}_{p_{\text{Vesta}}}$,

$$\text{Commit}_{\text{ivk}}^{\text{ivk}}(x, y) := \text{ShortCommit}_{\text{ivk}}(z.\text{cash:Orchard-CommitIvk}, \text{LE}_{255}(x) \parallel \text{LE}_{255}(y)),$$

a 510-bit message (protocol specification, §“Sinsemilla commitments”). Its output is $\perp$ exactly when

$$\text{SinsemillaHashToPoint}(z.\text{cash:Orchard-CommitIvk-M}, \cdot)$$

returns $\perp$; it is 0 exactly when the commitment point is $\mathcal{O}$ (Lemma 2.3); otherwise it is the x -coordinate of a non-identity point, an element of $\{1, \dots, p_{\text{Pallas}} - 1\}$ and hence, since $p_{\text{Pallas}} < p_{\text{Vesta}}$, a non-zero Pallas scalar. The x -coordinates of non-identity points form a set of $(p_{\text{Vesta}} - 1)/2$ elements, by Lemma 2.4 and because a group of odd prime order has no point with $y = 0$. The incoming viewing key of §3.2 is this instance applied to (ak, nk) with trapdoor r_{ivk} ; the exclusion of the outputs 0 and $\perp$ by key generation is stated there.

Remark 2.21 (Cost inside a proof). Sinsemilla is used for its cost inside an arithmetic circuit with lookups, one table lookup and two incomplete additions over $\mathbb{F}_{p_{\text{Pallas}}}$ per k -bit chunk (*Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Remark “In-circuit cost of Sinsemilla”; *Halo 2 Guide*, §“The lookup argument” and §“From statement to circuit: arithmetisation in practice”), and its collision resistance reduces to discrete logarithms on Pallas together with GroupHash modelled as a random oracle, not to discrete logarithms alone.

Assumption 2.22 (Discrete logarithms on Pallas). For every classical probabilistic polynomial-time algorithm, given a generator G of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ and $[s]G$ for s uniform in $\mathbb{F}_{p_{\text{Vesta}}}$, the probability of outputting s is negligible (*Crypto Guide*, §“The discrete logarithm problem”, Definition “Discrete logarithm problem, DLP”, instantiated on Pallas; *Math Guide*, §“The elliptic-curve discrete logarithm problem”). Concretely, the best known classical attack costs about 2^{126} group operations (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Proposition “The Pasta curves against the criteria”); the level is cited, not recomputed.

Lemma 2.23 (Relations among fixed generators). *Under Assumptions 2.8 and 2.22, no efficient algorithm outputs, except with negligible probability, distinct inputs of GroupHash with values $P_1, \dots, P_t$ and coefficients $a_1, \dots, a_t \in \mathbb{F}_{p_{\text{Vesta}}}$, not all zero, with $\sum_i [a_i]P_i = \mathcal{O}$.*

Proof. In the model of Assumption 2.8 a reduction receiving a challenge $Y = [s]G$ answers each new GroupHash input with $[u]G + [w]Y$ for fresh uniform $u, w \in \mathbb{F}_{p_{\text{Vesta}}}$, a uniform point and hence distributed as the oracle’s answer. For every value of w exactly one u is consistent with the answer, so the w remain uniform and independent of the algorithm’s view. A relation $\sum_i [a_i]P_i = \mathcal{O}$ gives $c_0 + c_1 s = 0$ with $c_1 = \sum_i a_i w_i$, which is non-zero except with probability $1/p_{\text{Vesta}}$, and then $s = -c_0 c_1^{-1}$ (*Crypto Guide*, §“Pedersen vector commitments”, Theorem “Properties of the vector commitment”, proof; §“Hashing to a curve point”, Remark “Why NUMS generators are binding-safe”). $\square$

Every later argument that ends in a non-trivial discrete-logarithm relation among values of GroupHash ends in this lemma.

Proposition 2.24 (Collision resistance of the Sinsemilla instances). *Consider an Orchard instance of Table 4 with message length l , hash domain D' (z.cash:Orchard-MerkleCRH for MerkleCRH; $D \parallel -M$ for a commitment instance with domain D) and, for a commitment instance, blinding base H_D . Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability:*

(i) *messages $M, M' \in \{0, 1\}^l$ whose hash points*

$$P := \text{SinsemillaHashToPoint}(D', M) \quad \text{and} \quad P' := \text{SinsemillaHashToPoint}(D', M')$$

are not $\perp$ and satisfy $P = P'$ with $M \neq M'$, or $P = -P'$; hence, by Lemma 2.4, no $M \neq M'$ with equal values of the extracted form SinsemillaHash other than $\perp$;

(ii) *for a commitment instance, openings (M, r) and (M', r') in $\{0, 1\}^l \times \mathbb{F}_{p_{\text{Vesta}}}$ whose commitments*

$$C := \text{SinsemillaCommit}_r(D, M) \quad \text{and} \quad C' := \text{SinsemillaCommit}_{r'}(D, M')$$

are not $\perp$ and satisfy $C = C'$ with $M \neq M'$, or $C = -C'$; hence no $M \neq M'$ with equal values of ShortCommit other than $\perp$;

(iii) *a message in $\{0, 1\}^l$ on which SinsemillaHashToPoint($D', \cdot$) returns $\perp$, equivalently on which the SinsemillaHash or commitment form returns $\perp$ for every trapdoor.*

Each such output yields efficiently a non-trivial discrete-logarithm relation among $Q(D')$, $S(0)$, ..., $S(2^k - 1)$ and, in (ii), H_D .

Proof. The proof is by citation, with the Orchard steps made in place. The cited arguments are, in the *Crypto Guide*, §“Sinsemilla: an algebraic hash-based commitment”, Proposition “Collision resistance of Sinsemilla”, with the paragraph after it on the extracted coordinate and the fixed length, and Construction “SinsemillaCommit”; and the Security argument of the protocol specification, §“Sinsemilla Hash Function”: Lemma “An injectivity property for Sinsemilla”, the theorem “Collision resistance of SinsemillaHash and SinsemillaHashToPoint” with its note extending it to added terms with independent bases, which covers SinsemillaCommit and ShortCommit, and the theorem that a $\perp$ output of SinsemillaHashToPoint yields a non-trivial discrete-logarithm relation. These arguments write a non- $\perp$ output on the pieces $m = (m_1, \dots, m_n)$ as

$$\text{SinsemillaHashToPoint}(D', M) = [2^n]Q(D') + \sum_{j=0}^{2^k-1} [\chi_j(m)]S(j), \quad (1)$$

where $\chi_j(m) := \sum_{i=1}^n 2^{n-i} \delta_{m_i, j}$ with δ the Kronecker delta, and with $[r]H_D$ added for a commitment. Equal outputs on distinct messages give

$$\sum_j [\chi_j(m) - \chi_j(m')]S(j) + [r - r']H_D = \mathcal{O},$$

non-trivial because $m \mapsto \chi(m)$ is injective. Opposite outputs give

$$[2^{n+1}]Q(D') + \sum_j [\chi_j(m) + \chi_j(m')]S(j) + [r + r']H_D = \mathcal{O},$$

non-trivial because its coefficient on $Q(D')$ is non-zero. An exceptional case at step i has the form $[\alpha]\text{Acc}_{i-1} + S(m_i) = \mathcal{O}$ with $\alpha \in \{-1, 1, 2\}$; substituting (1) for Acc_{i-1} gives a relation with coefficient $\alpha 2^{i-1}$ on $Q(D')$. For the hash forms the terms in H_D are absent.

Three steps are made in place. (1) Each instance hashes one fixed length l per domain, and l is admissible: $n \leq c$, with $n = 52, 109$ and 51 (Table 4). Admissibility is exactly the hypothesis under which no coefficient wraps modulo p_{Vesta} . Because $2^n \leq 2^c \leq (p_{\text{Vesta}} - 1)/2$, each difference $\chi_j(m) - \chi_j(m')$ has absolute value below $2^n < p_{\text{Vesta}}$ and vanishes modulo p_{Vesta} only if it vanishes as an integer, so uniqueness of binary expansions makes χ injective; and $0 < 2^{n+1} \leq p_{\text{Vesta}} - 1$ and $0 < |\alpha 2^{i-1}| \leq 2^n$ keep the coefficient on $Q(D')$ non-zero. Padding to nk bits is injective on messages of the fixed length l , so $M \neq M'$ gives $m \neq m'$. (2) By the definition of incomplete addition, an instance returns $\perp$ exactly when some incomplete addition meets an exceptional case, the case the cited $\perp$ theorem treats. An operand $\mathcal{O}$ arises only if $Q(D')$ or some $S(j)$ is $\mathcal{O}$, itself a non-trivial relation, because an incomplete addition that meets no exceptional case never returns $\mathcal{O}$. The commitment forms return $\perp$ exactly when the point hash does, for every trapdoor, since the blinding addition is complete. (3) The generators $Q(D'), S(0), \dots, S(2^k - 1)$ and H_D are values of GroupHash on pairwise distinct inputs (Table 2), so the lemma on relations among fixed generators excludes each relation except with negligible probability. The consequences for the extracted forms follow from Lemma 2.4: equal extracted values other than $\perp$ come from equal or opposite points, which are the cases of (i) and (ii). $\square$

Remark 2.25 (Fixed length). The fixed-length hypothesis cannot be dropped. The point hash `SinsemillaHashToPoint` zero-pads its input to a multiple of k bits, so for a message M whose length is not a multiple of k the messages M and $M \parallel 0$ collide with no discrete-logarithm relation. The specification requires collision resistance only between inputs of one fixed length for a given domain (protocol specification, §“Sinsemilla Hash Function”), and every Orchard domain hashes a single length.

Lemma 2.26 (Hiding and binding of `SinsemillaCommit`). *Let D be a byte string and $l \leq kc$ a message length.*

- (i) *Perfect hiding: if $H_D \neq \mathcal{O}$, then for every $M \in \{0, 1\}^l$ with hash point $\hat{M} \neq \perp$ and r uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of M , the commitment $\text{SinsemillaCommit}_r(D, M)$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; hence the distribution of neither it nor $\text{ShortCommit}_r(D, M)$, a fixed function of it, depends on M .*
- (ii) *Computational binding: under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, openings (M, r) and (M', r') with $M \neq M'$ in $\{0, 1\}^l$ whose commitments $\text{SinsemillaCommit}_r(D, M)$ and $\text{SinsemillaCommit}_{r'}(D, M')$ are equal and not $\perp$.*

Proof. The lemma is the *Crypto Guide*’s Construction “`SinsemillaCommit`” (§“Sinsemilla: an algebraic hash-based commitment”) for the Orchard blinding base. (i) Since $H_D \neq \mathcal{O}$ and the group has prime order p_{Vesta} , the map $r \mapsto [r]H_D$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group, so $[r]H_D$ is uniform, and so is its translate by the fixed point $\hat{M}$ (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Perfect hiding”). (ii) Two such openings give the first relation in the proof of Proposition 2.24, whose coefficient vector on the $S(j)$ is non-zero for $n = \lceil l/k \rceil \leq c$ by step (1) of that proof. The only Orchard addition is the blinding base $H_D = \text{GroupHash}(D \parallel -r, \epsilon)$, whose input differs from those of $Q(D \parallel -M)$ and of every $S(j)$, since its domain separator ends in $-r$; the lemma on relations among fixed generators excludes the relation. The specification states the same properties, with hiding conditional on no $\perp$ output (protocol specification, §“Sinsemilla commitments”). $\square$

3 Keys and addresses

This section constructs the key components of the protocol specification, §“Orchard Key Components”, from the instances of §2: the spending key and the spend-side secrets, the viewing keys, and the diversified addresses. It states each assumption that their security needs at its first use and proves, in Proposition 3.18, four one-way separations between key tiers.

3.1 The spending key and the spend-side secrets

Remark 3.1 (Scope of the key derivations). Every derivation of this section is that of a key generated with `use_qsk = false`, where `use_qsk` is the Boolean key-generation flag of ZIP 2005. ZIP 2005, §“Changes to the Protocol Specification”, item §4.2.3, specifies, as a change to the specification’s

§“Orchard Key Components” and not in that section itself, an alternative source of ak^P and r_{ivk} , with nk derived as here; it yields components (ak, nk, r_{ivk}) of the same types, and ak^P again of even y -coordinate. The volume does not construct the alternative and names none of its intermediate keys. Assumption 3.13, Propositions 3.18, 3.17, 6.10 and 10.6, Proposition “Confidentiality of the outgoing ciphertext” (§10.3), and the hypotheses on honest key generation of Theorems 12.9 and 12.12 are stated for $use_qsk = \text{false}$ only. For such keys every key component is a deterministic function of the spending key sk .

Definition 3.2 (Spending key and key generation). The *spending key* sk is a uniformly random 256-bit string, identified with a 32-byte string (protocol specification, §“Constants”). Key generation draws sk , derives from it the components of this section, and discards sk and draws a new one when $ask = 0$ (below), when $ivk \in \{0, \perp\}$ (§3.2), or on the one further condition of the protocol specification, §“Orchard Key Components”, which concerns the internal key of ZIP 32, a key this volume does not construct, and is stated in §3.2. A spending key that key generation does not discard is *accepted*; it determines every other component of a key generated with $use_qsk = \text{false}$.

Construction 3.3 (Spend-side secrets). For a spending key sk , the *spend authorising key* ask , the *nullifier deriving key* nk and the trapdoor r_{ivk} of Commit^{ivk} are

$$\begin{aligned} ask &:= \text{ToScalar}(\text{PRFexpand}_{sk}([0x06])) \in \mathbb{F}_{p_{\text{Vesta}}}, \\ nk &:= \text{ToBase}(\text{PRFexpand}_{sk}([0x07])) \in \mathbb{F}_{p_{\text{Pallas}}}, \\ r_{ivk} &:= \text{ToScalar}(\text{PRFexpand}_{sk}([0x08])) \in \mathbb{F}_{p_{\text{Vesta}}} \end{aligned}$$

(protocol specification, §“Orchard Key Components”). The three lead bytes are rows of Table 3. Key generation discards sk if $ask = 0$.

The scalars ask and r_{ivk} lie in $\mathbb{F}_{p_{\text{Vesta}}}$, the scalar field of the Pallas group, because each multiplies a Pallas point: ask the generator $G^{\text{SpendAuth}}$ below, r_{ivk} the blinding base of Commit^{ivk} (§3.2). The key nk is an element of the base field $\mathbb{F}_{p_{\text{Pallas}}}$.

The three inputs of PRFexpand_{sk} have distinct lead bytes. By Lemma 2.15, under Assumption 2.12, the triple (ask, nk, r_{ivk}) computed from a uniform sk , before rejection and before the normalisation below, is therefore computationally indistinguishable from independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, up to an additional statistical distance of three reduction distances, each below 2^{-257} (§2.3).

Construction 3.4 (Spend validating key and sign normalisation). Let $G^{\text{SpendAuth}} := \text{GroupHash}(z.\text{cash}:\text{Orchard}, \mathbb{G})$ (Table 2), the generator of the spend-authorisation signature scheme (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”). For $ask \neq 0$:

1. compute the point $P := [ask] G^{\text{SpendAuth}}$;
2. if its y -coordinate is odd, that is, if bit 255 of the star encoding P^* is 1, replace ask by $-ask$;

3. let $ak^P := [ask] G^{\text{SpendAuth}}$ for the updated ask, and $ak := \text{Extract}_P(ak^P) \in \mathbb{F}_{p_{\text{Pallas}}}$.

The *spend validating key* is the point ak^P together with its x -coordinate ak , a field element (protocol specification, §“Orchard Key Components”).

After step 2 the point ak^P has even y -coordinate. The point P is not $\mathcal{O}$, since $ask \neq 0$ and $G^{\text{SpendAuth}}$ generates a group of prime order, so $P = (x, y)$ with $y \neq 0$ (§2.1, on the star encoding). Replacing ask by $-ask$ replaces P by $-P = (x, -y)$, and the representatives y and $p_{\text{Pallas}} - y$ have opposite parities because p_{Pallas} is odd.

For an accepted key, $ak \in \{1, \dots, p_{\text{Pallas}} - 1\}$, and ak determines ak^P . Indeed $ak^P \neq \mathcal{O}$, so $ak \neq 0$ by Lemma 2.3. By Lemma 2.4 the points with x -coordinate ak are ak^P and $-ak^P$, whose y -coordinates have opposite parities; the point ak^P is the one with even y -coordinate, that is, $ak^P = \text{abst}_P(\text{LE}_{256}(ak))$, the *even- y lift* of ak . Theorem 12.9 cites this normalisation.

3.2 Viewing keys

Definition 3.5 (Full viewing key). The *full viewing key* of an accepted key is the triple

$$\text{fvk} := (ak, nk, r_{\text{ivk}}) \in \{1, \dots, p_{\text{Pallas}} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Vesta}}},$$

with ak the field element of §3.1; the point ak^P is recomputed from ak by the even- y lift.

Construction 3.6 (Incoming viewing key scalar). With the key commitment of §2.4 and $D := \text{z.cash:Orchard-CommitIvk}$,

$$\text{ivk} := \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(ak, nk) = \text{ShortCommit}_{r_{\text{ivk}}}(D, \text{LE}_{255}(ak) \parallel \text{LE}_{255}(nk)) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}.$$

Explicitly, let $\hat{M}$ be the Sinsemilla hash point of the 510-bit message $\text{LE}_{255}(ak) \parallel \text{LE}_{255}(nk)$ under the domain $\text{z.cash:Orchard-CommitIvk-M}$, and let

$$H_D := \text{GroupHash}(\text{z.cash:Orchard-CommitIvk-r}, \varepsilon)$$

be the blinding base; then $\text{ivk} = \text{Extract}_P(\hat{M} + [r_{\text{ivk}}] H_D)$ if $\hat{M} \neq \perp$, and $\text{ivk} = \perp$ otherwise. Key generation discards sk when $\text{ivk} \in \{0, \perp\}$, so an accepted key has $\text{ivk} \in \{1, \dots, p_{\text{Pallas}} - 1\}$. Because $p_{\text{Pallas}} < p_{\text{Vesta}}$, this integer is a non-zero element of $\mathbb{F}_{p_{\text{Vesta}}}$, the scalar that multiplies the diversified base in §3.3.

Let $X_{\text{Pallas}} := \{\text{Extract}_P(P) : P \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}\} \subset \{1, \dots, p_{\text{Pallas}} - 1\}$. The set X_{Pallas} has $(p_{\text{Vesta}} - 1)/2$ elements (§2.4, Definition “The key commitment”), and an accepted ivk lies in it. The blinding base H_D is not $\mathcal{O}$: the protocol specification, §“Sinsemilla commitments”, states that the commitment is distributed as a uniform point for a uniform trapdoor, which holds only for a blinding base other than $\mathcal{O}$.

Lemma 3.7 (Distribution of ivk). Let r_{ivk} be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of (ak, nk) , and let $\hat{M} \neq \perp$. Then $\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(ak, nk)$ is 0 with probability $1/p_{\text{Vesta}}$ and otherwise uniform on X_{Pallas} .

| *independently of* (ak, nk).

Proof. By Lemma 2.26(i) the point $\hat{M} + [r_{\text{ivk}}] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, whatever $\hat{M}$ is. It is $\mathcal{O}$, of extracted coordinate 0, with probability $1/p_{\text{Vesta}}$; each $x \in X_{\text{Pallas}}$ is the extracted coordinate of exactly the two points $\pm P$ with $P \neq -P$ (Lemma 2.4), and so has probability $2/p_{\text{Vesta}}$. $\square$

In particular ivk is not uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ (protocol specification, §“Orchard Key Components”, note).

The further rejection condition of key generation concerns the internal key of ZIP 32, §“Orchard internal key derivation”. That key has the same ak and nk and a trapdoor $t \in \mathbb{F}_{p_{\text{Vesta}}}$ that is an efficiently computable function of (ak, nk, r_{ivk}), and key generation discards sk when its incoming viewing key $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk})$ is 0 or $\perp$ (protocol specification, §“Orchard Key Components”).

Lemma 3.8 (Rejection in key generation). *Under Assumptions 2.22 and 2.8, a single draw of key generation from independent uniform (ask, nk, r_{ivk}) is rejected with negligible probability. Under Assumptions 2.12, 2.22 and 2.8, the output of key generation is within negligible statistical distance of a single draw from a uniform sk without rejection.*

Proof. A single draw is rejected only if $\text{ask} = 0$, of probability $1/p_{\text{Vesta}}$; or $\hat{M} = \perp$; or $\text{ivk} = 0$, of probability $1/p_{\text{Vesta}}$ by Lemma “Distribution of ivk ”; or $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk}) = 0$. The value $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk})$ commits to the message of ivk , so it is $\perp$ exactly when $\hat{M} = \perp$. The event $\hat{M} = \perp$ has negligible probability under Assumptions 2.22 and 2.8: an algorithm that samples (ask, nk) and outputs the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ finds a $\perp$ input with that probability, which Proposition 2.24(iii) bounds. For $\hat{M} \neq \perp$, $\text{Commit}_t^{\text{ivk}}(\text{ak}, \text{nk}) = 0$ exactly when $\hat{M} + [t] H_D = \mathcal{O}$ (Lemma 2.3). By (1), with $n = 51$ (Table 4), this is a relation among $Q(\text{z.cash:Orchard-CommitIvk-M})$, the $S(j)$ and H_D whose coefficient on the first is 2^{51} , non-zero modulo p_{Vesta} . An algorithm that samples (ask, nk, r_{ivk}), computes t and outputs this relation succeeds with the probability of the event, which the lemma on relations among fixed generators (§2.4) bounds under the same assumptions. The rejection event is efficiently decidable from the three outputs of $\text{PRFexpand}_{\text{sk}}$, so by Lemma 2.15 a first draw from a uniform sk is also rejected with negligible probability. Key generation outputs its first draw whenever that draw is accepted; its output is therefore within that negligible probability, in statistical distance, of a single draw without rejection. $\square$

The proofs of this section analyse a single draw and add the statistical distance bounded by this lemma, under Assumptions 2.12, 2.22 and 2.8.

Construction 3.9 (Diversifier key and outgoing viewing key). Let $K := \text{LE}_{256}(r_{\text{ivk}})$ and

$$R := \text{PRFexpand}_K([0 \times 82] \parallel \text{I2LEOSP}_{256}(\text{ak}) \parallel \text{I2LEOSP}_{256}(\text{nk})),$$

a 64-byte string, where I2LEOSP_{256} encodes the field elements ak and nk as 32-byte little-endian integers. The *diversifier key* dk is the first 32 bytes of R and the *outgoing viewing key* ovk the last 32 bytes (protocol specification, §“Orchard Key Components”; row 0×82 of Table 3). One output of PRFexpand , keyed by the encoding of r_{ivk} , yields both.

Definition 3.10 (Incoming and outgoing viewing keys). The *incoming viewing key* is the pair (dk, ivk) ; the *outgoing viewing key* is ovk . The diversifier key dk derives diversifiers (§3.3). What the holders of ivk and of ovk can compute from note ciphertexts is stated in “Trial decryption and note acceptance” (§10.4) and “The outgoing ciphertext” (§10.3).

Proposition 3.11 (Binding of ivk to (ak, nk)). *Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, triples $(ak, nk, r_{ivk}) \neq (ak', nk', r_{ivk}')$ with $\text{Commit}_{r_{ivk}}^{ivk}(ak, nk) = \text{Commit}_{r_{ivk}'}^{ivk}(ak', nk') \neq \perp$.*

Proof. Equal extracted values come from commitment points C, C' with $C = \pm C'$ (Lemma 2.4). If the messages differ, both cases are excluded by Proposition 2.24(ii), since LE_{255} is injective on integer representatives. If the messages agree, then $r_{ivk} \neq r_{ivk}'$, the case $C = C'$ gives $[r_{ivk} - r_{ivk}'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order, and the case $C = -C'$ is excluded by the same proposition. $\square$

Hence ivk is bound to the pair (ak, nk) , and so is every address of the key (§3.3): its transmission key $pk_d = [ivk] g_d$ with $g_d \neq \mathcal{O}$ determines ivk , an integer below p_{Vesta} . The binding is used in Lemma 12.5 (“Double-spend resistance”, §12.2) and Theorem 12.9 (“Authorisation of spends”, §12.3). Binding alone would follow from any collision-resistant hash of (ak, nk) ; the trapdoor r_{ivk} adds hiding.

Remark 3.12 (Hiding of ivk and the double use of r_{ivk}). For r_{ivk} uniform and independent of every other quantity, ivk is independent of (ak, nk) by Lemma “Distribution of ivk ”. The derivation of (dk, ovk) is keyed by the same r_{ivk} , so once dk or ovk is known that argument no longer applies, and modelling PRFexpand only as a PRF does not restore it: the key of row 0x82 is also the trapdoor of Commit^{ivk} , outside the hypothesis of Lemma 2.15 (§2.3; protocol specification, §“Orchard Key Components”, note on the use of r_{ivk} as both trapdoor and key). The volume states the joint assumption on Pallas scalar multiplication and PRFexpand that the specification proposes as Assumption 3.13.

Assumption 3.13 (Joint hiding of the viewing keys). For keys with $\text{use_qsk} = \text{false}$: for every $(ak, nk) \in \{1, \dots, p_{\text{Pallas}} - 1\} \times \mathbb{F}_{p_{\text{Pallas}}}$, given to the distinguisher, and r_{ivk} uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, the triple (ivk, dk, ovk) computed from (ak, nk, r_{ivk}) by the constructions of this subsection is computationally indistinguishable from

$$(\text{Commit}_{r_{ivk}}^{ivk}(ak, nk), u_1, u_2),$$

where r_{ivk}' is uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and u_1, u_2 are uniform 32-byte strings, all independent. It is a precise form of the joint assumption that the protocol specification proposes (§“Orchard Key Components”, note); it is not perfect hiding. Results apply it after Lemma 2.15 (Assumption 2.12) has made r_{ivk} uniform and independent of (ak, nk) , averaging over the key-generation distribution of (ak, nk) . It is used by Propositions 3.18, 3.17, 6.10 and 10.6, by Proposition “Confidentiality of the outgoing ciphertext” (§10.3) and by Theorem 12.12.

3.3 Diversified addresses

Construction 3.14 (Diversified address). For an index $j \in \{0, \dots, 2^{88} - 1\}$ chosen by the key holder:

1. the *diversifier* is $d := \text{FF1-AES256}_{\text{dk}}(\text{LE}_{88}(j)) \in \{0, 1\}^{88}$, the FF1 mode of NIST SP 800-38G over AES-256 with key dk, radix 2, length 88 and the empty tweak (*Crypto Guide*, §“Format-preserving encryption and FF1”, Construction “The FF1 mode of NIST SP 800-38G”; protocol specification, §“Pseudo Random Permutations”);
2. the *diversified base* is $\mathbf{g}_d := \text{DiversifyHash}(d)$ (protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”), where

$$T := \text{GroupHash}(\text{z.cash:Orchard-gd}, \text{LEBS2OSP}_{88}(d))$$

and

$$\text{DiversifyHash}(d) := \begin{cases} T & \text{if } T \neq \mathcal{O}, \\ \text{GroupHash}(\text{z.cash:Orchard-gd}, \epsilon) & \text{otherwise;} \end{cases}$$

3. the *diversified transmission key* is $\text{pk}_d := [\text{ivk}] \mathbf{g}_d$, with ivk read as a scalar (§2.1).

The *address* is $(d, \text{pk}_d) \in \{0, 1\}^{88} \times (\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\})$ (protocol specification, §“Orchard Key Components”).

The map LEBS2OSP_{88} packs d into 11 bytes, eight bits at a time, least significant bit first (§2.1). Every index yields an address. The map GroupHash returns a point for every input, and DiversifyHash replaces the identity by the fixed fallback point, which is not the identity: the protocol specification types $\text{DiversifyHash}^{\text{Orchard}}$ into the non-identity points. The fallback case has negligible probability under Assumption 2.8. Hence $\mathbf{g}_d \neq \mathcal{O}$ for every d , and $\text{pk}_d \neq \mathcal{O}$ because ivk is a non-zero scalar and the group has prime order. For each key dk the map $j \mapsto d$ is a permutation of the 88-bit strings (*Crypto Guide*, §“Format-preserving encryption and FF1”, Proposition “FF1 is a keyed bijection on $\mathcal{X}$ ”), so distinct indices give distinct diversifiers: one incoming viewing key (dk, ivk) yields 2^{88} distinct addresses, all with transmission keys under the same ivk. No index is excluded.

Assumption 3.15 (FF1-AES256 as a pseudorandom permutation). For a uniform 256-bit key, FF1-AES256 with the empty tweak on $\{0, 1\}^{88}$ is a pseudorandom permutation: no efficient adversary with oracle access distinguishes it from a uniformly random permutation of $\{0, 1\}^{88}$ (*Crypto Guide*, §“Pseudorandom functions and permutations”, Definition “Pseudorandom permutation”, and §“Format-preserving encryption and FF1”, Definition “Format-preserving encryption”). It is the security requirement of the protocol specification, §“Pseudo Random Permutations”, whose note records that the distinguishing attack of Dunkelman, Kumar, Lambrooij and Sanadhya at this domain size and round count is no better than exhaustive search for the 256-bit key. It is used so that a diversifier reveals neither its index nor the key that produced it, in Proposition 3.17 (“Unlinkability of diversified addresses”).

Assumption 3.16 (DDH on Pallas). Let P be a generator of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, let b and c be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and a uniform on X_{Pallas} (§3.2), all independent. The triples $([a]P, [b]P, [ab]P)$ and $([a]P, [b]P, [c]P)$ are computationally indistinguishable. With a uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ instead, this is the DDH assumption of the *Crypto Guide* (§“Diffie–Hellman: computational and decisional”, Definition “Decisional Diffie–Hellman, DDH”). The volume assumes the form with a uniform on X_{Pallas} because ivk is so distributed, and claims no reduction between the two forms; the protocol specification expects the non-uniformity of ivk to cause no security issue (§“Orchard Key Components”, note). It is used by Propositions 3.17 and 10.6 (“Confidentiality and key privacy of note encryption”).

Proposition 3.17 (Unlinkability of diversified addresses). *Let two keys be generated independently with $\text{use_qsk} = \text{false}$. An adversary holds ak , nk and ovk of both keys, and none of r_{ivk} , dk and ivk of either. It chooses indices j_0, j_1 and j' with $j' \notin \{j_0, j_1\}$, and receives the address of key 0 at index j_0 , the address of key 1 at index j_1 , and the address of key β at index j' , for a uniform hidden bit β ; it outputs a bit β' . Conditioned on the event E that the three diversifiers are pairwise distinct, its advantage $|\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$ is negligible under Assumptions 2.12 (through Lemma 2.15), 3.13, 3.15, 2.8 and 3.16, together with Assumption 2.22 for Lemma “Rejection in key generation”. This is the unlinkability requirement of the protocol specification, §“DiversifyHash^{Sapling} and DiversifyHash^{Orchard} Hash Functions”, which stipulates distinct diversifiers.*

Proof. For each β the game is modified by a sequence of hops, each changing the adversary’s output distribution by a negligible amount (*Crypto Guide*, §“The hybrid argument”, Lemma “Hybrid lemma”). Each hop is justified by a reduction that runs the game; the event E is efficiently decidable from the diversifiers and has probability at least $1/2$ in every game below, so conditioning on E at most doubles each cost.

1. Each key is replaced by a single draw without rejection (Lemma “Rejection in key generation”, §3.2).
2. For each key in turn, $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ is replaced by independent uniform elements of the three fields (Lemma 2.15); the reduction derives the remaining components and the addresses.
3. For each key in turn, $(\text{ivk}, \text{dk}, \text{ovk})$ is replaced by

$$(\text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}, (\text{ak}, \text{nk}), u_1, u_2)$$

(Assumption 3.13); the reduction is given (ak, nk) , generates the other key itself, and computes the game from the triple. Now dk and ovk are uniform and independent of all else, and, except on the negligible events $\text{ivk} \in \{0, \perp\}$, the scalar ivk is uniform on X_{Pallas} and independent of (ak, nk) (Lemmas “Distribution of ivk ” and “Rejection in key generation”, §3.2).

4. For $b \in \{0, 1\}$ in turn, $\text{FF1-AES256}_{\text{dk}_b}$ is replaced by a uniformly random permutation π_b of $\{0, 1\}^{88}$ (Assumption 3.15); the reduction queries its oracle at the indices it needs. Then $d' = \pi_\beta(\text{LE}_{88}(j')) \neq d_\beta$ because $j' \neq j_\beta$, and on E the triple (d_0, d_1, d') is uniform on

pairwise distinct triples, for either value of β . The event E fails only if $d_{1-\beta}$ equals d_β or d' , of probability $2/2^{88}$.

5. On E the inputs $\text{LEBS2OSP}_{88}(d)$ of GroupHash are distinct, since LEBS2OSP_{88} is injective, so under Assumption 2.8 the bases g_0, g_1, g' are independent uniform points, each $\mathcal{O}$, which invokes the fallback, with probability $1/p_{\text{Vesta}}$ only.

In the last game the adversary's view consists of $\text{ak}, \text{nk}, \text{ovk}$ of both keys and (d_0, d_1, d') , independent of the rest and identically distributed for $\beta = 0, 1$, together with

$$(g_0, [\text{ivk}_0] g_0), \quad (g_1, [\text{ivk}_1] g_1), \quad (g', [\text{ivk}_\beta] g'),$$

where $\text{ivk}_0, \text{ivk}_1$ are independent and uniform on X_{Pallas} . The first two pairs are public keys of the per-key-base ElGamal scheme with private keys ivk_0 and ivk_1 . For $g_\beta \neq \mathcal{O}$ the uniform point g' equals $[r] g_\beta$ for a uniform scalar r , so the third pair is $([r] g_\beta, [r](\text{ivk}_\beta g_\beta))$, an encryption of $\mathcal{O}$ under key β (*Crypto Guide*, §“Key privacy”, Construction “ElGamal encryption”). Distinguishing β is therefore the game of Definition “Key privacy under chosen-plaintext attack, IK-CPA” of the same subsection, for the message $\mathcal{O}$. The proof of its Theorem “ElGamal is key-private under DDH” bounds the advantage by two DDH advantages. That proof plants the first exponent of the DDH triple as a private key; with private keys uniform on X_{Pallas} , the same proof, with the other key's private key drawn uniformly from X_{Pallas} (the x -coordinate of a uniform non-identity point), reduces to Assumption 3.16. Its reduction samples (d_0, d_1, d') itself and programs GroupHash at their three inputs with the planted bases before running the adversary. The *Crypto Guide*'s Remark “Key privacy of note encryption and diversified addresses” records this correspondence for diversified addresses. Summing the costs of the hops, the advantage is negligible. $\square$

3.4 Key capabilities and scope

Figure 1 gives the derivation graph, each edge labelled by its operation, and each one-way edge also by the assumption that makes it one-way. Proposition 3.18 proves four separations: the full viewing key yields no ask, the incoming viewing key neither nk nor ask , the outgoing viewing key no ivk , and an address no ivk .

Proposition 3.18 (Capability separation). *Let a key be generated honestly with $\text{use_qsk} = \text{false}$. No efficient algorithm, given the public parameters and only the listed key material of the key, outputs the listed secret, except with negligible probability:*

- (a) given the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$, the scalar ask ;
- (b) given the incoming viewing key (dk, ivk) , the key nk or the scalar ask ;
- (c) given ovk , the scalar ivk ;
- (d) given an address (d, pk_d) of the key, the scalar ivk .

The hypotheses are: for (a), Assumptions 2.12 (through Lemma 2.15) and 2.22; for (b), those of (a) and, for nk , Assumption 3.13; for (c), Assumptions 2.12 and 3.13; for (d), Assumptions 2.12, 3.13, 2.8 and 2.22 (“Discrete logarithms on Pallas”). Every part also uses Lemma “Rejection in key generation” (§3.2), under Assumptions 2.12, 2.22 and 2.8. These are the only separations claimed.

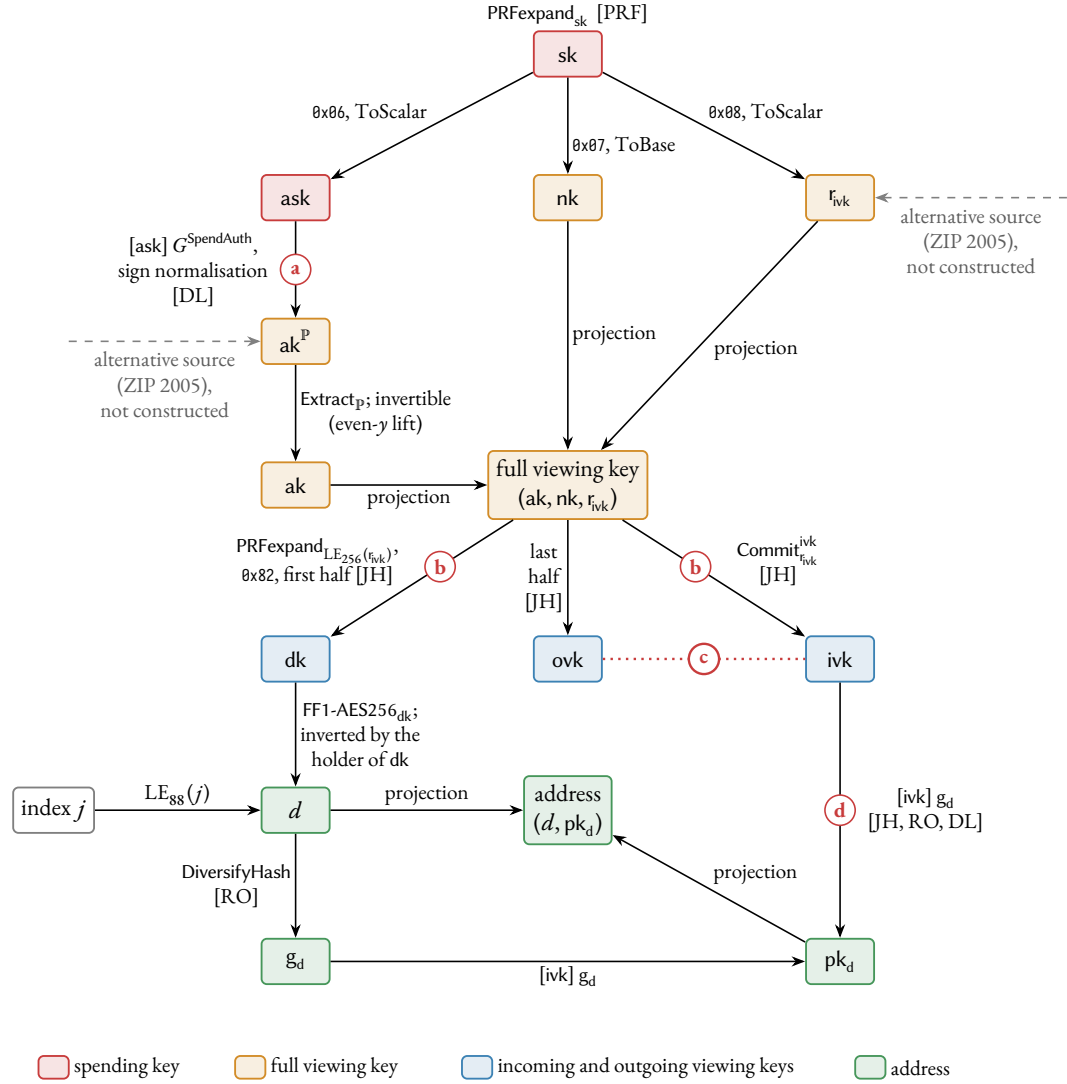


Figure 1. Derivation graph of a key generated with `use_ksk = false`, from the spending key sk to the address (d, pk_d) . Each edge carries its operation, and each one-way edge also carries, in brackets, the assumption under which it is one-way: PRF, Assumption 2.12; DL, Assumption 2.22; JH, Assumption 3.13; RO, Assumption 2.8. Each one-way edge is a PRF call, a scalar multiplication, the joint hiding assumption or GroupHash as a random oracle; bundling edges invert by projection, Extract_p by the even- γ lift, and the FF1 edge is inverted by the holder of dk . The edges out of sk are one-way under Assumption 2.12, since an algorithm recovering sk from (ask, nk, r_{ivk}) distinguishes PRFexpand_{sk} from a random function. Colours give the lowest key tier that computes a node; the index j is uncoloured. The dashed edges into ak^p and r_{ivk} mark the alternative source of ZIP 2005, not constructed. Markers (a) to (d) are the four separations that Proposition 3.18 proves; no other separation is claimed.

Proof. By Lemma “Rejection in key generation” (§3.2), a single draw from a uniform sk replaces the key at a negligible cost. By Lemma 2.15, its triple (ask, nk, r_{ivk}) is then replaced by independent uniform elements of $\mathbb{F}_{p_{Vesta}}$, $\mathbb{F}_{p_{Pallas}}$ and $\mathbb{F}_{p_{Vesta}}$, at a further negligible cost: in each part the success of an algorithm is efficiently decidable from the three values, by deriving the remaining components, running the algorithm and comparing its output with the secret. The result is called the idealised key.

(a) Let an algorithm $\mathcal{A}$ output ask from (ak, nk, r_{ivk}) with probability ϵ for the idealised key. An algorithm $\mathcal{B}$ receives $Y = [s] G^{\text{SpendAuth}}$ for s uniform on $\mathbb{F}_{p_{Vesta}}$, draws nk and r_{ivk} uniformly, and runs $\mathcal{A}$ on $(\text{Extract}_p(Y), nk, r_{ivk})$. The input is distributed as for the idealised key, because the sign normalisation replaces ask by $\pm ask$ and does not change ak (Lemma 2.4). A correct output a satisfies $[a] G^{\text{SpendAuth}} = ak^p$, the even- γ lift of $\text{Extract}_p(Y)$, which is Y or $-Y$; $\mathcal{B}$ outputs whichever of a and $-a$ maps to Y . Hence $\mathcal{B}$ computes discrete logarithms to the generator $G^{\text{SpendAuth}}$ with probability ϵ , and ϵ is negligible by Assumption 2.22.

(b) An algorithm that computes ask from (dk, ivk) computes it from (ak, nk, r_{ivk}) by first deriving (dk, ivk) , so (a) applies. Let $\mathcal{A}$ output nk from (dk, ivk) with probability ϵ . A distinguisher for Assumption 3.13, given (ak, nk) of the idealised key and a triple (x, y, z) , runs $\mathcal{A}$ on (y, x) and outputs 1 exactly when $\mathcal{A}$ outputs nk . On the real triple it outputs 1 with probability ϵ . On the ideal triple $\mathcal{A}$ receives u_1 and $\text{Commit}_{ivk}^{ivk}(ak, nk)$, which is independent of nk unless $\hat{M} = \perp$, a negligible event (Lemma “Rejection in key generation”); since nk is uniform on $\mathbb{F}_{p_{Pallas}}$, $\mathcal{A}$ then succeeds with probability at most $1/p_{Pallas}$ plus a negligible term. So ϵ exceeds the distinguisher’s advantage by a negligible amount at most.

(c) Let $\mathcal{A}$ output ivk from ovk with probability ϵ . The distinguisher given (ak, nk) and (x, y, z) runs $\mathcal{A}$ on z and outputs 1 exactly when $\mathcal{A}$ outputs x . On the ideal triple, $z = u_2$ is independent of $x = \text{Commit}_{ivk}^{ivk}(ak, nk)$, which, for $\hat{M} \neq \perp$, takes each value with probability at most $2/p_{Vesta}$ by Lemma “Distribution of ivk ”. Hence ϵ is at most the advantage under Assumption 3.13 plus $2/p_{Vesta}$ and a negligible term.

(d) Let $\mathcal{A}$ output ivk from the address at an index j , possibly of its choice, with probability ϵ . The distinguisher given (ak, nk) and (x, y, z) computes $d := \text{FF1-AES256}_y(\text{LE}_{88}(j))$, $g_d := \text{DiversifyHash}(d)$ and $pk_d := [x] g_d$, runs $\mathcal{A}$ on (d, pk_d) , and outputs 1 exactly when $\mathcal{A}$ outputs x . On the ideal triple, $\mathcal{A}$ succeeds with probability $\epsilon' \geq \epsilon - \delta$, for δ the advantage under Assumption 3.13; there, outside the negligible event $\hat{M} = \perp$, the value $ivk := x$ is 0 with probability $1/p_{Vesta}$ and otherwise uniform on X_{Pallas} , independently of (ak, nk) and of u_1 , hence of d . An algorithm $\mathcal{B}$ receives a generator G and $Y = [s] G$ for s uniform on $\mathbb{F}_{p_{Vesta}}$. It draws t uniform on $\mathbb{F}_{p_{Vesta}} \setminus \{0\}$ and u_1 uniform, computes d from u_1 and j , programs GroupHash at $(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ with the uniform non-identity point $[t] G$ (Assumption 2.8), answers other queries with fresh uniform points, and runs $\mathcal{A}$ on $(d, [t] Y) = (d, [s]([t] G))$. With probability $(p_{Vesta} - 1)/(2p_{Vesta})$ the exponent s lies in X_{Pallas} ; it is then uniform on X_{Pallas} , and the view of $\mathcal{A}$ is that of the ideal triple conditioned on $ivk \neq 0$, up to statistical distance $1/p_{Vesta}$ for the programmed point. Algorithm $\mathcal{B}$ outputs the output of $\mathcal{A}$ and so computes s with probability at least

$$\frac{p_{Vesta} - 1}{2p_{Vesta}} \left(\epsilon' - \frac{2}{p_{Vesta}} \right),$$

a loss factor of $2p_{Vesta}/(p_{Vesta} - 1)$, about 2, against Assumption 2.22. Hence ϵ' , and with it ϵ , is

negligible. □

Of the objects of this section only an address (d, pk_d) , with the base g_d computed from d , is public; $sk, ask, ak^p, ak, nk, r_{ivk}$, the full viewing key, dk, ivk, ovk and the index j are secret key material. One incoming viewing key yields 2^{88} addresses (§3.3), unlinkable in the sense of Proposition 3.17 (“Unlinkability of diversified addresses”).

What a holder of each key can compute from the objects of later sections is stated where those objects are constructed, each with a back-reference to Proposition 3.18: from nullifiers in “Nullifier uniqueness and unlinkability” (§6.4), and from note ciphertexts in “The outgoing ciphertext” (§10.3) and “Trial decryption and note acceptance” (§10.4). This section states no such power.

Keys and addresses are objects of the Orchard protocol, not of a pool: one key hierarchy and one address format serve both the Orchard pool and the Ironwood pool (ZIP 229, Abstract).

4 Notes and note commitments

This section defines the note, its commitment and its extracted commitment, and proves their hiding and binding; it constructs the derivation of all randomness of a created note from one seed under the lead byte $0x03$, and defines the Action and the bundle as units of a transaction.

4.1 Notes

Definition 4.1 (Unit of value). The *zatoshi* is the smallest unit of value; one ZEC, the unit of the native currency of Zcash, is 10^8 zatoshi (protocol specification, §“Shielded Pools and Notes” and §“Mainnet and Testnet”). Every value in this volume, note values included, is an integer number of zatoshi.

Definition 4.2 (Note). A *note* is a tuple $n = (addr, v, \rho, \psi, rcm)$, where

- the address $addr = (d, pk_d)$ of the recipient, of the type constructed in §3.3: a diversifier $d \in \{0, 1\}^{88}$ and a transmission key $pk_d \in \mathcal{E}(\mathbb{F}_{p_{Pallas}}) \setminus \{\mathcal{O}\}$;
- the value $v \in \{0, \dots, 2^{64} - 1\}$, in zatoshi;
- the elements $\rho \in \mathbb{F}_{p_{Pallas}}$ and $\psi \in \mathbb{F}_{p_{Pallas}}$;
- the commitment trapdoor $rcm \in \mathbb{F}_{p_{Vesta}}$

(protocol specification, §“Shielded Pools and Notes”; the lengths 88 and 64 are those of §“Constants”). The specification types rcm as an integer in $\{0, \dots, 2^{255} - 1\}$ (§“Commitment”). This volume types it in $\mathbb{F}_{p_{Vesta}}$, because every derivation of rcm yields a reduced value and rcm acts only as the scalar of $[rcm]$, which depends on rcm modulo p_{Vesta} alone; this typing makes the binding of Proposition 4.5 a statement about notes. For a created note, ψ and rcm are derived from its note seed, for an Ironwood-pool note in §4.3, and ρ is the nullifier of the note consumed in the same Action, a rule constructed in “Nullifier chaining” (§6.3).

A note is never published. A transaction that creates a note publishes its extracted note commitment cmx (Definition 4.4) and a ciphertext of its note plaintext for the recipient, constructed in “Encryption to the recipient” (§10.2) (protocol specification, §“Note Commitments” and §“Sending Notes (Orchard)”). Of the fields of a created note, only ρ appears on chain in the clear: it equals the nullifier that the same Action publishes for its consumed note (“Nullifier chaining”, §6.3). This representation of value serves requirement R1 of §1.4; Propositions 4.5 and 4.8 state what the commitment hides.

Remark 4.3 (Roles of the fields). The address and the value are the content of the note, its recipient and its amount; the trapdoor rcm randomises its commitment; the elements ρ and ψ , which also enter the commitment, are the note-specific inputs of its nullifier, constructed in “The nullifier” (§6.1).

4.2 Note commitments

Definition 4.4 (Note commitment). Let $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ be a note, $\text{g}_d := \text{DiversifyHash}(d)$ (§3.3) and

$$M(n) := \text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi),$$

with ρ and ψ read as integers in $\{0, \dots, p_{\text{Pallas}} - 1\}$; the string $M(n)$ has the fixed length 1086 of Table 4. With $D := \text{z.cash:Orchard-NoteCommit}$, the *note commitment* of n is

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(M(n)) := \text{SinsemillaCommit}_{\text{rcm}}(D, M(n)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \cup \{\perp\}.$$

By the routing of §2.4, with the hash point and the blinding base

$$\begin{aligned} \hat{M} &:= \text{SinsemillaHashToPoint}(\text{z.cash:Orchard-NoteCommit-M}, M(n)), \\ H_D &:= \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \varepsilon), \end{aligned}$$

the commitment is $\text{cm} = \hat{M} + [\text{rcm}] H_D$ if $\hat{M} \neq \perp$, and $\text{cm} = \perp$ otherwise. The *extracted note commitment* is $\text{cmx} := \text{Extract}_{\mathbb{P}}^{\perp}(\text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\}$ (§2.1). The specification’s NoteCommit takes the five encoded fields as separate arguments, in the order of their encodings in $M(n)$ (protocol specification, §“Sinsemilla commitments”, §“Sending Notes (Orchard)” and §“Coordinate Extractor for Pallas”).

A sender never publishes $\perp$: it redraws the note seed (step (vi) of the derivation of §4.3). The extracted commitment cmx is the value an Action publishes for its created note, and the leaf of that note in the pool’s note commitment tree of “The Merkle hash and the tree” (§5.1), whose nodes are elements of $\mathbb{F}_{p_{\text{Pallas}}}$.

The blinding base H_D is not $\mathcal{O}$: the protocol specification, §“Sinsemilla commitments”, states that SinsemillaCommit is perfectly hiding for a uniform trapdoor, for every domain, which holds only for a blinding base other than $\mathcal{O}$ (as for $\text{Commit}^{\text{ivk}}$, §3.2).

Proposition 4.5 (Hiding and binding of note commitments).

- (a) *Perfect hiding.* Let rcm be uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. For every note content $(d, \text{pk}_d, v, \rho, \psi)$ whose message $M(n)$ has hash point $\hat{M} \neq \perp$, the commitment cm is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$. Hence the distributions of cm and of cmx do not depend on $(g_d, \text{pk}_d, v, \rho, \psi)$, and NoteCommit is perfectly hiding on messages without a $\perp$ output, in the sense of the *Crypto Guide*’s Definition “Hiding game and hiding” (§“Hiding and binding”).
- (b) *Binding of the extracted commitment.* Under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, two notes $n \neq n'$ with $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. In particular two distinct notes share no note commitment other than $\perp$. With Proposition 2.24(iii), which excludes messages with output $\perp$, part (b) gives computational binding in the sense of the *Crypto Guide*’s Definition “Binding game and binding” (same section), with the note content as the message and rcm as the randomness, stated for notes and for the extracted value.

Proposition 4.8 states hiding for the trapdoor that a sender derives.

Proof. (a) This is Lemma 2.26(i) for the domain D and the message length 1086, whose hypothesis $H_D \neq \mathcal{O}$ holds by the paragraph above; the value cmx is a fixed function of cm . Equal distributions for every pair of contents without a $\perp$ output are perfect hiding by the *Crypto Guide*’s Proposition “Distributional characterisation of hiding” (§“Hiding and binding”).

(b) Step 1 reduces distinct notes to distinct openings. The map $(g_d, \text{pk}_d, v, \rho, \psi) \mapsto M(n)$ is injective: the string $M(n)$ concatenates fields of fixed lengths; the star encoding is injective (§2.1); the encoding LE_{64} is injective on $\{0, \dots, 2^{64} - 1\}$; and the encoding LE_{255} is injective on the representatives of $\mathbb{F}_{p_{\text{Pallas}}}$, since $p_{\text{Pallas}} < 2^{255}$ (§2.1). The map $n \mapsto (M(n), \text{rcm})$ is therefore injective except on pairs of notes with $d \neq d'$ and $\text{DiversifyHash}(d) = \text{DiversifyHash}(d')$. Let T and T' be the values of GroupHash at $(z.\text{cash}:\text{Orchard-gd}, \text{LEBS2OSP}_{88}(d))$ and at the corresponding input for d' , distinct inputs because LEBS2OSP_{88} is injective. If $T, T' \neq \mathcal{O}$, equal diversified bases give $T - T' = \mathcal{O}$; otherwise $T = \mathcal{O}$ or $T' = \mathcal{O}$. Each is a non-trivial relation among values of GroupHash on distinct inputs, which the lemma on relations among fixed generators (§2.4) excludes except with negligible probability.

Step 2 treats distinct openings. Let $\text{cmx}(n) = \text{cmx}(n') \neq \perp$. The lifting $\text{Extract}_{\text{p}}^{\perp}$ maps only $\perp$ to $\perp$, so cm and cm' are points, and by Lemma 2.4 $\text{cm}' = \text{cm}$ or $\text{cm}' = -\text{cm}$. The pairs $(M(n), \text{rcm})$ and $(M(n'), \text{rcm}')$ are openings of the instance NoteCommit in $\{0, 1\}^{1086} \times \mathbb{F}_{p_{\text{Vesta}}}$. By Proposition 2.24(ii), outputs with $\text{cm}' = -\text{cm}$, or with $\text{cm}' = \text{cm}$ and $M(n) \neq M(n')$, have negligible probability (in the opposite case the relation in its proof has coefficient 2^{109+1} on the hash base, for a message of 109 chunks). In the remaining case $\text{cm}' = \text{cm}$ and $M(n) = M(n')$, and outside the event of Step 1 the openings differ, so $\text{rcm} \neq \text{rcm}'$ in $\mathbb{F}_{p_{\text{Vesta}}}$; then $[\text{rcm} - \text{rcm}'] H_D = \mathcal{O}$, impossible for $H_D \neq \mathcal{O}$ in a group of prime order p_{Vesta} . A note commitment other than $\perp$ shared by $n \neq n'$ gives equal extracted commitments other than $\perp$, which proves the particular case. $\square$

4.3 The note seed: Ironwood derivations (lead byte 0x03)

Definition 4.6 (Note seed and lead byte). A created note reaches its recipient as a *note plaintext*, the byte string constructed in “The note plaintext” (§10.1) that the sender encrypts. Its first byte, the *lead byte*, names the format of the plaintext and selects the derivation of the note’s randomness. The sender draws one *note seed* $rseed$, a uniform 32-byte string, the key type of PRFexpand (§2.3), which the plaintext carries. The values ψ and r_{cm} of the note and the ephemeral secret key $esk \in \mathbb{F}_{p_{Vesta}}$ of “Encryption to the recipient” (§10.2) are all derived from it (protocol specification, §“Note Plaintexts and Memo Fields” and §“Sending Notes (Orchard)”).

Construction 4.7 (Derivations under lead byte 0x03). The input is an address (d, pk_d) , a value $v \in \{0, \dots, 2^{64} - 1\}$ and $\rho \in \mathbb{F}_{p_{Pallas}}$. Let $g_d := \text{DiversifyHash}(d)$ and $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$. For a point $P = (x, y)$ the 32-byte string $\text{LEBS2OSP}_{256}(P^*)$ equals $\text{I2LEOSP}_{256}(x + 2^{255}(y \bmod 2))$ (§2.1). The sender

- (i) checks that $pk_d \neq \mathcal{O}$;
- (ii) draws $rseed$ uniformly from the 32-byte strings;
- (iii) derives $esk := \text{ToScalar}(\text{PRFexpand}_{rseed}([0x04] \parallel \underline{\rho})) \in \mathbb{F}_{p_{Vesta}}$, and returns to (ii) if $esk = 0$;
- (iv) derives $\psi := \text{ToBase}(\text{PRFexpand}_{rseed}([0x09] \parallel \underline{\rho})) \in \mathbb{F}_{p_{Pallas}}$;
- (v) derives $r_{cm} := \text{ToScalar}(\text{PRFexpand}_{rseed}(\text{pre_rcm})) \in \mathbb{F}_{p_{Vesta}}$, where

$$\begin{aligned} \text{pre_rcm} := & [0x0B] \parallel \text{LEBS2OSP}_{256}(g_d^*) \parallel \text{LEBS2OSP}_{256}(pk_d^*) \\ & \parallel \text{I2LEOSP}_{64}(v) \parallel \underline{\rho} \parallel \text{I2LEOSP}_{256}(\psi), \end{aligned}$$

a string of $1 + 32 + 32 + 8 + 32 + 32 = 137$ bytes;

- (vi) computes cmx of the note $n := ((d, pk_d), v, \rho, \psi, r_{cm})$ by Definition 4.4, and returns to (ii) if $cmx = \perp$.

The output is the note n , its seed $rseed$ and esk (protocol specification, §“Sending Notes (Orchard)”); ZIP 2005, “Changes to the Protocol Specification”, §4.7.3).

The order is forced: the string pre_rcm contains ψ , so r_{cm} is derived after ψ . The strings g_d^* and pk_d^* in pre_rcm are those of the message $M(n)$, packed into bytes, so r_{cm} is a function of $rseed$ and of every committed field $(g_d, pk_d, v, \rho, \psi)$. The leading bytes 0x04, 0x09 and 0x0B are rows of Table 3.

An Ironwood-pool note has the type of Definition “Note” (§4.1) and is committed by the same NoteCommit (Definition 4.4). Every note created by an Ironwood-pool Action, a dummy created note included, has a plaintext with lead byte 0x03 and the derivations above; no note created by an Orchard-pool Action does. This is the only note-level distinction between the two pools (ZIP 229, “Transaction Format”; ZIP 258, “ZIP 2005 activation”).

Proposition 4.8 (Hiding under the derived trapdoor). *An adversary chooses two tuples $(d^{(i)}, \text{pk}_d^{(i)}, v^{(i)}, \rho^{(i)})$, $i \in \{0, 1\}$, of the input type of the construction above, with $\text{pk}_d^{(i)} \neq \mathcal{O}$. The challenger draws a uniform bit b , runs steps (ii) to (vi) of the construction on tuple b , and returns $(\text{cm}_b, \text{esk}_b)$, the note commitment of the note it computes and its esk; the adversary receives no other function of rseed (the ciphertext that carries rseed is treated in Theorem 12.12). It outputs a bit b' . Let the number ϵ_{PRF} bound the advantage against Assumption 2.12 of each algorithm built from the adversary in the proof, each of which queries $\text{PRFexpand}_{\text{rseed}}$ on three inputs with pairwise distinct leading bytes; let $\delta < 2^{-257}$ bound the distance of ToScalar from uniform (§2.3); and let $\epsilon_{\perp}$ be the largest, over i , of the sum of two probabilities that one pass through steps (ii) to (vi) on tuple i returns to step (ii), one with $\text{PRFexpand}_{\text{rseed}}$ as specified and one with its three outputs replaced by independent uniform strings. Then*

$$|\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]| \leq 2(\epsilon_{\text{PRF}} + \delta + \epsilon_{\perp}),$$

and ϵ_{PRF} and $\epsilon_{\perp}$ are negligible under Assumptions 2.12, 2.22 and 2.8. The same bound holds with cmx_b in place of cm_b .

Proof. Fix the adversary's tuples. One pass through steps (ii) to (vi) with a fresh rseed is a *trial*; its output is (cm, esk) , and it is *accepted*, event $\mathcal{A}$, when $\text{esk} \neq 0$ and $\hat{M} \neq \perp$, since $\text{cmx} = \perp$ exactly when $\hat{M} = \perp$. Trials are independent and identically distributed, and the challenger returns the output of the first accepted one; so its answer X_i on tuple i is distributed as the output Y_i of one trial conditioned on $\mathcal{A}$. For a random variable Y and an event $\mathcal{A}$ with $\Pr[\neg \mathcal{A}] = p < 1$, the statistical distance between Y and Y conditioned on $\mathcal{A}$ is at most p : for every set S ,

$$\Pr[Y \in S] - \Pr[Y \in S \mid \mathcal{A}] = \Pr[Y \in S, \neg \mathcal{A}] - \Pr[Y \in S, \mathcal{A}] \frac{p}{1-p},$$

and both terms lie in $[0, p]$.

Hybrid 1 replaces the outputs of $\text{PRFexpand}_{\text{rseed}}$ on $[\text{0x04}] \parallel \underline{\rho}$, $[\text{0x09}] \parallel \underline{\rho}$ and pre_rcm by independent uniform $u_1, u_2, u_3 \in \{0, 1\}^{512}$, so that $\text{esk} = \text{ToScalar}(u_1)$, $\psi = \text{ToBase}(u_2)$ and $\text{rcm} = \text{ToScalar}(u_3)$. The three inputs have distinct leading bytes, the input pre_rcm is chosen after the answer that determines ψ , and rseed is used only as a key of PRFexpand . By Lemma 2.15 (Assumption 2.12), the probability that any efficient test outputs 1 on the trial's output changes by at most ϵ_{PRF} . Hybrid 2 replaces rcm by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$, independent of all else, at statistical cost δ . The event $\mathcal{A}$ does not involve rcm , so its probability is the same in both hybrids.

In Hybrid 2 the values esk and ψ are independent, $\mathcal{A}$ is the intersection of an event on esk and an event on ψ , and rcm is uniform and independent of both. Conditioned on $\mathcal{A}$, therefore, $\hat{M} \neq \perp$ and $\text{cm} = \hat{M} + [\text{rcm}] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Proposition 4.5(a), independently of esk , and esk is distributed as $\text{ToScalar}(u_1)$ conditioned on being non-zero. This distribution $\mathcal{W}$ does not depend on i .

For each i , the adversary's output probability on X_i differs from that on $\mathcal{W}$ by at most the rejection probability of a trial as specified (X_i against Y_i), plus $\epsilon_{\text{PRF}} + \delta$ (Y_i against its form in Hybrid 2), plus the rejection probability in Hybrid 2, which equals that with uniform strings (the unconditioned form in Hybrid 2 against $\mathcal{W}$). The two rejection probabilities sum to at most $\epsilon_{\perp}$.

Adding the bounds for $i = 0$ and $i = 1$ gives the stated bound. For cmx_b , an adversary against cmx_b is one against cm_b that first applies the fixed map $\text{Extract}_p^\perp$.

Negligibility. With uniform strings, $\text{esk} = 0$ has probability at most $1/p_{\text{Vesta}} + \delta$. The event $\hat{M} = \perp$ has the probability with which an efficient algorithm finds a message of hash point $\perp$: the algorithm that runs the adversary's choice of tuples, draws u_2 , and outputs $M(n)$ for tuple i with $\psi = \text{ToBase}(u_2)$. That probability is negligible by Proposition 2.24(iii) under Assumptions 2.22 and 2.8. Rejection is efficiently decidable from the three outputs of $\text{PRFexpand}_{\text{rseed}}$, so the rejection probability as specified exceeds that with uniform strings by at most ϵ_{PRF} , negligible under Assumption 2.12. $\square$

Remark 4.9 (Purpose of the hashed trapdoor). Because rcm is a hash of rseed and of every committed field, Ironwood-pool notes are recoverable in the sense of ZIP 2005. That ZIP proposes, with details subject to change, a Recovery Protocol for recovering Ironwood-pool funds if the Orchard protocol must be disabled because discrete logarithms become computable; a recovery protocol whose statement recomputes rcm by this derivation can bind a note even against an adversary able to compute discrete logarithms (ZIP 2005, Abstract; Rationale, “Proposed Recovery Protocol” and “Repairing note commitments”). Against such an adversary NoteCommit alone is not binding: the discrete-logarithm relations between its bases give a linear equation over $\mathbb{F}_{p_{\text{Vesta}}}$ with several solutions, hence two distinct notes that open one commitment (ZIP 2005, Rationale, “Attacks against binding of note commitments”). What the deployed protocol checks of the derivation is stated with the Action statement, defined in “The Action statement” (§9.2).

The specification defines, for each pool, the set $\text{allowedLeadBytes}^{\text{pool}}$ of lead bytes allowed in a note plaintext of that pool. For every transaction that contains Actions of the pool, the set is $\{\text{0x03}\}$ for the Ironwood pool and $\{\text{0x02}\}$ for the Orchard pool (protocol specification, §“Note Plaintexts and Memo Fields”; ZIP 2005, “Changes to the Protocol Specification”, §3.2.1; ZIP 258, “ZIP 2005 activation”). The lead byte travels encrypted, and each decrypting party enforces the rule: decryption with an incoming viewing key, by the recipient, and decryption with an outgoing viewing key, by its holder, both return $\perp$ when the lead byte is not allowed (§“Decryption using an Incoming Viewing Key (Sapling and Orchard)” and §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”). For an Ironwood-pool output this is check (i) of the acceptance procedure of “Trial decryption and note acceptance” (§10.4). The one consensus check of the lead byte is stated in “Chain state and pool rules” (§11.4).

4.4 Actions, bundles, and dummy notes

Definition 4.10 (Action). An *Action* of a pool consumes one note n^{old} and creates one note n^{new} of that pool; it is the specification's Action transfer, and every Action has both sides (protocol specification, §“Action Transfers and their Descriptions”). Its public data carry the extracted commitment cmx of the created note (Definition 4.4), the nullifier of the consumed note (“The nullifier”, §6.1), a randomised validating key with a spend-authorisation signature (“Randomised validating keys”, §7.2), a net value commitment (“Value commitments”, §8.1), and the ciphertexts of “Encryption to the recipient” (§10.2). “Action descriptions and bundles”

(§11.1) assembles them and redefines neither the Action nor the bundle. The relation that each Action must satisfy is the Action statement, defined in “The Action statement” (§9.2).

Definition 4.11 (Dummy notes). Every Action has one consumed and one created note, so that its public fields are the same whether a side is real or not; a side that a payment does not use carries a *dummy note*, a note of value 0. A *dummy consumed note* is built as follows (protocol specification, §“Dummy Notes (Orchard)” ZIP 2005, “Changes to the Protocol Specification”, §4.8.3):

- (i) draw a fresh spending key as in §3.1, and derive its key components and an address (d, pk_d) (§3.2, §3.3);
- (ii) set $v := 0$, draw rseed uniformly, and set $\rho := \text{Extract}_p(P)$ for a uniform point P of $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$;
- (iii) derive ψ and rcm under the lead byte chosen as for a created note of the pool (ZIP 2005, “Changes to the Protocol Specification”, §4.8.3); in the Ironwood pool the lead byte is $0x03$ and the derivations are steps (iv) and (v) of §4.3;
- (iv) compute cm by Definition 4.4, and repeat from the draw of rseed with a fresh seed if $\text{cm} = \perp$.

Its nullifier is computed as for any note (“The nullifier”, §6.1). For step (iii) the two sources differ: the specification’s section chooses the lead byte with the pool fixed to the Orchard pool, which gives $0x02$ also in the Ironwood pool, whereas ZIP 2005, §4.8.3, makes the choice identical for dummy and non-dummy notes. The volume follows the ZIP. No condition of the Action statement checks the derivation of rcm^{old} (Definition 9.2, Remark “What the statement does not check”).

A *dummy created note* is a created note of value 0, built as any created note; in the Ironwood pool it therefore carries lead byte $0x03$. Its address depends on the flag `enableCrossAddress` of the bundle (“Bundle flags”, §9.1).

- When `enableCrossAddress` = 1, the dummy created note goes to a random address (protocol specification, §“Dummy Notes (Orchard)”).
- When `enableCrossAddress` = 0, as in every Orchard-pool bundle, condition A10 of Definition 9.2 places the note that an Action creates at the expanded receiver, defined in “Bundle flags”, of the note that the same Action consumes. A dummy created note then goes to the expanded receiver of that consumed note, real or dummy. In the Orchard pool, beside a real consumed note, the sender fills the component C_{enc} of the note ciphertext of the dummy created note (“Encryption to the recipient”, §10.2) with random bytes, not with an encryption of its plaintext (ZIP 326, “Fabricated same-address outputs and randomized note ciphertexts”). A dummy consumed note beside a real created note lies at the expanded receiver of that created note: in place of step (i) it takes the created note’s address and the key components of the key from which that address derives, and the Action’s spend-authorisation signature requires that key’s ask (Remark “Meaning of A10”,

§9.2). A fresh key as in step (i) remains admissible when both sides of the Action are dummies at its address.

That the public data do not reveal which sides of an Action are real is part of Theorem 12.12 (§12.4).

Definition 4.12 (Bundle). The *bundle* of a pool in a transaction, which ZIP 229 calls the pool’s component, is the sequence of that pool’s Actions in the transaction together with the bundle-level data that later sections attach: one anchor (“Anchors”, §5.2), one balancing value (“The balancing value”, §8.2), one binding signature (“The binding signature”, §8.3), the flags (“Bundle flags”, §9.1) and one proof for all its Actions (“The Action circuit and the Halo 2 proof”, §9.3). A transaction carries at most one bundle per pool (ZIP 229, “Transaction Format”).

5 The note commitment tree

This section constructs the note commitment tree of a pool and its Merkle hash, defines the anchors, proves in Proposition 5.9 that a valid authentication path to an anchor exhibits a leaf of the anchored tree unless it yields a collision of the hash’s Sinsemilla instance or an input on which that instance returns $\perp$, and proves in Lemma 5.15 that the authentication path of a leaf is a function of public data.

5.1 The Merkle hash and the tree

Remark 5.1 (Requirement R2). Requirement R2 of “Requirements on a shielded payment” (§1.4) asks the consumer of a note of non-zero value to show that the note was previously committed, without identifying it. In the Orchard protocol the claim to be shown is that the extracted commitment cmx of the note (Definition 4.4) is a leaf of the note commitment tree of the note’s pool, and the leaf is not to be disclosed. Membership of a private leaf under a public root is provable in zero knowledge, with the root public and the leaf, its position and its authentication path private (*Crypto Guide*, §“Privacy-preserving membership via zero-knowledge paths”, Theorem “Privacy-preserving membership”). We construct the tree, its roots, called anchors, and its authentication paths; the relation proved for each Action is the Action statement, defined in “The Action statement” (§9.2).

Definition 5.2 (Note commitment tree). Let

$$\text{MerkleDepth}^{\text{Orchard}} := 32, \quad \text{Uncommitted}^{\text{Orchard}} := 2 \in \mathbb{F}_{p_{\text{Pallas}}}$$

(protocol specification, §“Constants”). Each pool of the Orchard protocol has its own *note commitment tree*, an append-only Merkle tree (*Crypto Guide*, §“Binary Merkle trees” and §“Append-only and incrementally updatable trees”) of fixed depth $\text{MerkleDepth}^{\text{Orchard}} = 32$, hence with 2^{32} leaf positions. Its *layers* are numbered from layer 0, the root, to layer 32, the leaves; layer b has 2^b nodes $M_i^b \in \mathbb{F}_{p_{\text{Pallas}}}$, $0 \leq i < 2^b$.

1. *Appending.* Each Action of the pool in a transaction publishes the extracted commitment cmx of its created note (§4.4). When the transaction enters the chain, each such cmx is appended to the pool's tree: it occupies the leaf M_i^{32} for the least unused index i , and i is the *note position* pos of that commitment. Every pool's tree has no appended leaf before the first block of the chain.
2. *Leaves.* After n appends, the leaf M_i^{32} is the value appended at position i for $0 \leq i < n$, and $M_i^{32} := \text{Uncommitted}^{\text{Orchard}} = 2$ for $n \leq i < 2^{32}$.
3. *Internal nodes.* For $0 \leq b < 32$ and $0 \leq i < 2^b$,

$$M_i^b := \text{MerkleCRH}(b, M_{2i}^{b+1}, M_{2i+1}^{b+1}),$$

with MerkleCRH the Merkle hash constructed below; the *root* is M_0^0 .

(Protocol specification, §“Note Commitment Trees”, §“Merkle Path Validity”, §“Note Commitments” and §“Transactions and Treestates”).)

Construction 5.3 (Merkle hash). Let $D := \text{z.cash:Orchard-MerkleCRH}$. The Merkle hash

$$\text{MerkleCRH}: \{0, \dots, 31\} \times \mathbb{F}_{p_{\text{Pallas}}} \times \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$$

is

$$\text{MerkleCRH}(l, \text{left}, \text{right}) := \begin{cases} 0 & \text{if } \eta = \perp, \\ \eta & \text{otherwise,} \end{cases}$$

where

$$\eta := \text{SinsemillaHash}(D, \text{LE}_{10}(31 - l) \parallel \text{LE}_{255}(\text{left}) \parallel \text{LE}_{255}(\text{right})) \in \mathbb{F}_{p_{\text{Pallas}}} \cup \{\perp\},$$

with SinsemillaHash the extracted Sinsemilla hash of §2.4 and field elements encoded through their integer representatives. The protocol specification names the map $\text{MerkleCRH}^{\text{Orchard}}$ and the width 255 $\text{MerkleHashLength}^{\text{Orchard}}$ (§“MerkleCRH^{Orchard} Hash Function” and §“Constants”).

The prefix $\text{LE}_{10}(31 - l) = \text{LE}_{10}(\text{MerkleDepth}^{\text{Orchard}} - 1 - l)$ is defined because $31 < 2^{10}$. It encodes 0 for the combination of two leaves into a node of layer $l = 31$, and 31 for the combination that yields the root, $l = 0$. Every message has the single length $10 + 255 + 255 = 520$ bits, 52 chunks of $k = 10$ bits, within the bound of $c = 253$ chunks of §2.4 (Table 4). The encoding LE_{255} is injective on $\mathbb{F}_{p_{\text{Pallas}}}$ because $p_{\text{Pallas}} < 2^{255}$ (§2.1), so for a fixed layer distinct argument pairs give distinct messages. The protocol specification requires that SinsemillaHash be collision resistant and that no input of 520 bits yielding $\perp$ can feasibly be found (§“MerkleCRH^{Orchard} Hash Function”, security requirements). For the domain D and 520-bit messages these are parts (i) and (iii) of Proposition 2.24, under Assumptions 2.22 and 2.8.

Remark 5.4 (Nodes as field elements). Because MerkleCRH ends in Extract_p , each layer's output has the type of its inputs, and one hash composes from the leaves to the root. Proofs of the Ac-

tion statement, defined in “The Action statement” (§9.2), are Halo 2 proofs with the Vesta curve (protocol specification, §“Zero-Knowledge Proving System”), whose scalar field is $\mathbb{F}_{p_{\text{Pallas}}}$ (*Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”). A node that is one element of $\mathbb{F}_{p_{\text{Pallas}}}$ is one value of that proof’s arithmetic, whereas a point-valued node would carry two coordinates and a curve-equation condition at each of the 32 layers.

Remark 5.5 (Dropping the y -coordinate). A node serves only as a digest, never to recover a point, and $\text{Extract}_p(P) = \text{Extract}_p(P')$ only if $P' = P$ or $P' = -P$ (Lemma 2.4). A collision of $\text{SinsemillaHash}(D, \cdot)$ on 520-bit messages is therefore either a collision of the underlying hash to a point or a pair of messages whose hash points are opposite. Proposition 2.24(i) excludes both, and part (iii) excludes a $\perp$ output, under Assumptions 2.22 and 2.8.

Remark 5.6 (Layer separation). The prefix $\text{LE}_{10}(31 - l)$ separates the inputs of different layers (protocol specification, §“MerkleCRH^{Orchard} Hash Function”, note). The 32 layer functions $\text{MerkleCRH}(l, \cdot, \cdot)$ evaluate one instance $\text{SinsemillaHash}(D, \cdot)$ on message sets with distinct prefixes. Every collision that the proof of Proposition 5.9 extracts therefore lies between two messages with the same prefix, and a node computed at one layer is accepted at another only through such a collision or a $\perp$ input (*Crypto Guide*, §“Layer and domain separation”, Proposition “Separation forces within-domain collisions”).

Lemma 5.7 (Uncommitted leaves). *No point P of the Pallas group satisfies $\text{Extract}_p(P) = 2$. Consequently $\text{Uncommitted}^{\text{Orchard}} = 2$ equals the extracted commitment $\text{cmx} = \text{Extract}_p^\perp(\text{cm})$ of no note, and a leaf position holding 2 holds the commitment of no note.*

Proof. By definition $\text{Extract}_p(\mathcal{O}) = 0 \neq 2$ in $\mathbb{F}_{p_{\text{Pallas}}}$. A point $(x, y) \neq \mathcal{O}$ with $x = 2$ on the curve $y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Pallas}}}$ would satisfy $y^2 = 2^3 + 5 = 13$. By Euler’s criterion $13^{(p_{\text{Pallas}}-1)/2} \equiv -1 \pmod{p_{\text{Pallas}}}$ (*Math Guide*, §“Quadratic residues and the Euler criterion”, Example “13 is a non-square in both Pasta fields”), so 13 is a non-square modulo p_{Pallas} and no such y exists. Only the base-field modulus p_{Pallas} enters. For the consequence, cmx is $\perp$ when $\text{cm} = \perp$ and otherwise Extract_p of a point, so it is not 2. The protocol specification states the consequence as the theorem “Uncommitted^{Orchard} is not in the range of NoteCommit^{Orchard}” (§“Sinsemilla commitments”). $\square$

The protocol specification, §“Note Commitment Trees”, and ZIP 258, “Changes to the Protocol Specification”, state the capacity rule: a block must not add Ironwood-pool note commitments that would make the Ironwood-pool note commitment tree exceed its capacity of $2^{\text{MerkleDepth}^{\text{Orchard}}} = 2^{32}$ leaves; the same rule binds the Orchard-pool tree. The Ironwood-pool tree is a tree of its own, built by the construction above with the same depth, the same value $\text{Uncommitted}^{\text{Orchard}}$ at unused positions and the same hash; only the values cmx published by Ironwood-pool Actions are appended to it.

5.2 Anchors

Definition 5.8 (Treestate and anchor). A pool’s *treestate* comprises its note commitment tree and its nullifier set, constructed in “Nullifier sets” (§6.2). Treestates are chained: the input

treestate of the first block is empty; the input treestate of the first transaction of a block is the final treestate of the preceding block, and that of each later transaction the output treestate of the preceding transaction; the *final treestate* of a block is the output treestate of its last transaction (protocol specification, §“Transactions and Treestates”). The *anchor* of a block in a pool is the root M_0^0 of the pool’s tree in the block’s final treestate, an element of $\mathbb{F}_{p_{\text{Pallas}}}$; every block has exactly one anchor per pool. A bundle carries one anchor field, `anchorIronwood` in the Ironwood pool and `anchorOrchard` in the Orchard pool, shared by all its Actions; an Action carries no anchor of its own (“Action descriptions and bundles”, §11.1; protocol specification, §“Action Transfers and their Descriptions”).

The protocol specification, §“Action Transfers and their Descriptions”, and ZIP 258, “Changes to the Protocol Specification”, state the anchor rule: whenever a transaction has Ironwood-pool Actions, its field `anchorIronwood` must refer to some earlier block’s final Ironwood-pool treestate, that is, equal the Ironwood-pool anchor of a block that precedes the transaction’s block in the chain. The field `anchorOrchard` is bound likewise to the Orchard pool. The rule sets no bound on how far back that block lies.

Proposition 5.9 (Membership soundness). *Let rt be the anchor of a block in a pool, n the number of leaves appended to the pool’s tree through that block, and $L_0, \dots, L_{2^{32}-1}$ the leaves of the tree that rt roots, so that $L_i = \text{Uncommitted}^{\text{Orchard}}$ for $i \geq n$. For $v, s_0, \dots, s_{31} \in \mathbb{F}_{p_{\text{Pallas}}}$ and $\text{pos} = \sum_{t=0}^{31} b_t 2^t$ with $b_t \in \{0, 1\}$, let*

$$a_0 := v, \quad a_{t+1} := \begin{cases} \text{MerkleCRH}(31-t, a_t, s_t) & \text{if } b_t = 0, \\ \text{MerkleCRH}(31-t, s_t, a_t) & \text{if } b_t = 1 \end{cases} \quad (0 \leq t < 32). \quad (2)$$

The triple $(v, \text{pos}, (s_0, \dots, s_{31}))$ is a valid path to rt if $a_{32} = rt$. This is the path verification of the Crypto Guide (§“Authentication paths and membership proofs”, Definition “Path verification”), with level- t compression $\text{MerkleCRH}(31-t, \cdot, \cdot)$ and the bits of pos as direction bits; “Authentication paths” (§5.3) constructs the path of an appended leaf.

- (i) *From a valid path to rt with $v \neq L_{\text{pos}}$ and the leaves $L_0, \dots, L_{n-1}$ one computes efficiently either two distinct 520-bit messages with a common 10-bit prefix and equal values of $\text{SinsemillaHash}(D, \cdot)$, for $D = \text{z.cash:Orchard-MerkleCRH}$, or a 520-bit message on which $\text{SinsemillaHash}(D, \cdot)$ returns $\perp$.*
- (ii) *Under Assumptions 2.22 and 2.8, an efficient adversary, even one that chooses the appended leaves, outputs a valid path to rt with $v \neq L_{\text{pos}}$ with at most negligible probability.*
- (iii) *In particular, if $v = \text{Extract}_p(P)$ for a point P of the Pallas group, as the extracted commitment $\text{cmx} \neq \perp$ of every note is, then, except with negligible probability, $\text{pos} < n$ and v is the value appended at position pos , in the anchoring block or earlier.*

Proof. The argument is the *Crypto Guide*’s Theorem “Soundness of Merkle membership proofs” (§“Soundness: forging a path implies a collision”), with layer-dependent compressions.

(i) Let $j_t := \lfloor \text{pos}/2^t \rfloor$, and write $j \oplus 1$ for the integer j with its lowest bit complemented. The genuine chain is $\bar{a}_t := M_{j_t}^{32-t}$ for $0 \leq t \leq 32$, with genuine siblings $\bar{s}_t := M_{j_t \oplus 1}^{32-t}$ for $0 \leq t < 32$. Since $j_t = 2j_{t+1} + b_t$, the node $\bar{a}_{t+1} = M_{j_{t+1}}^{31-t}$ is MerkleCRH($31-t, \cdot, \cdot$) of its two children, of which $\bar{a}_t$ is the left one if $b_t = 0$ and the right one if $b_t = 1$, the other being $\bar{s}_t$. The genuine chain therefore satisfies (2) with $\bar{s}_t$ in place of s_t , and $\bar{a}_0 = L_{\text{pos}}$, $\bar{a}_{32} = M_0^0 = rt$ (*Crypto Guide*, §“Authentication paths and membership proofs”, Proposition “Completeness”, whose induction this is). The fold on the given path has $a_0 = v \neq \bar{a}_0$ and $a_{32} = rt = \bar{a}_{32}$. Let t^* be the least $t \geq 1$ with $a_t = \bar{a}_t$; it exists, and $a_{t^*-1} \neq \bar{a}_{t^*-1}$ by minimality. At level $t^* - 1$ both chains evaluate MerkleCRH($32 - t^*, \cdot, \cdot$) with the running value on the side fixed by b_{t^*-1} : the first on a_{t^*-1} and s_{t^*-1} , the second on $\bar{a}_{t^*-1}$ and $\bar{s}_{t^*-1}$. The two 520-bit messages hashed there share the prefix $\text{LE}_{10}(31 - (32 - t^*)) = \text{LE}_{10}(t^* - 1)$, the layer tag of the *Crypto Guide*’s Proposition “Separation forces within-domain collisions” (§“Layer and domain separation”). They differ in the half that encodes the running value, LE_{255} being injective on $\mathbb{F}_{p_{\text{pallas}}}$, and their values under MerkleCRH, $a_{t^*} = \bar{a}_{t^*}$, agree. If $\text{SinsemillaHash}(D, \cdot)$ returns $\perp$ on either message, that message is the second output. Otherwise both values of MerkleCRH are values of $\text{SinsemillaHash}(D, \cdot)$, and the pair is the first output. The genuine chain is computed from $L_0, \dots, L_{n-1}$ by the recurrence of the tree. A node whose leaves all lie at positions at least n depends only on its layer, so the computation takes a number of evaluations of MerkleCRH linear in $n + 32$; the fold takes 32 more.

(ii) The extraction of (i) is deterministic and defined for every list of leaves, so it applies to leaves that an adversary chose. An adversary that outputs a valid path with $v \neq L_{\text{pos}}$ with probability ϵ therefore yields an algorithm that outputs, with probability ϵ , a collision of $\text{SinsemillaHash}(D, \cdot)$ on 520-bit messages or a 520-bit message with output $\perp$. Equal values of SinsemillaHash other than $\perp$ come from hash points that are equal on distinct messages or opposite (Lemma 2.4). Proposition 2.24, parts (i) and (iii), excludes both outputs except with negligible probability, under Assumptions 2.22 and 2.8; hence ϵ is negligible.

(iii) If $\text{pos} \geq n$, then $L_{\text{pos}} = 2$, and $v \neq 2$ by Lemma 5.7, so the path is one with $v \neq L_{\text{pos}}$, which (ii) excludes except with negligible probability. Otherwise $\text{pos} < n$, and outside the same event $v = L_{\text{pos}}$, the value appended at position pos ; the n leaves of the tree that rt roots are those appended through the anchoring block. $\square$

Remark 5.10 (Old anchors). Appending leaves changes the final treestates of later blocks and of no earlier one. Once a block B is in the chain, its anchor therefore roots one fixed tree: a valid path to it stays valid after later appends, the anchor stays admissible under the anchor rule while B remains in the chain, and Proposition 5.9 applies to it at any later time. A valid path to the anchor of B for an extracted note commitment shows that the commitment was appended in B or earlier, which is what R2 requires. Excluding a second consumption of a note is not the task of the tree, which is append-only (protocol specification, §“Note Commitment Trees”), but that of the nullifier, constructed in “Nullifiers” (§6).

Definition 5.11 (Reorganisation). A *reorganisation* is the replacement of the chain’s most recent blocks by a competing branch.

With the height of a block as in §1.1, the *tip* is the last block of the chain. A reorganisation that removes the block whose final treestate a bundle’s anchor refers to leaves the transaction in

violation of the anchor rule on the new chain, unless a block of the new chain that precedes the transaction's block has the same anchor; to be included in the new chain, the transaction must then be rebuilt against an anchor of that chain. The block d blocks below a tip of height H , the block at height $H - d$, is removed only by a reorganisation that replaces at least $d + 1$ blocks, since a reorganisation that replaces k blocks removes the blocks at heights $H - k + 1$ to H (ZIP 315, "Rationale for anchor selection"; ZIP 218, "Anchor selection depth").

The *block target spacing*, the intended mean interval between consecutive blocks, is 25 seconds (ZIP 218, "Block target spacing").

Remark 5.12 (Recommended anchor depth). ZIP 315 recommends that a transaction built when the tip is at height H use as its anchor the final treestate of the block at height $H - 3$, three blocks below the tip (ZIP 315, "Anchor selection"); ZIP 218 recommends the same depth of 3 blocks ("Anchor selection depth"). At the 25-second target spacing this depth is $3 \times 25 = 75$ seconds. The depth is a recommendation to the builder of a transaction, not a consensus rule: the anchor rule admits the final treestate of any earlier block.

Remark 5.13 (Anchor choice as public data). The anchor is published with the bundle. Call the set $\{0, \dots, n - 1\}$ of positions of the leaves appended to the tree that an anchor roots the *candidate set* of the anchor; by Proposition 5.9(iii), a valid path to the anchor for an extracted note commitment has its position in this set, except with negligible probability. A later anchor roots a tree that contains every appended leaf of an earlier one, so its candidate set is at least as large. An anchor at a depth other than the one that other builders of transactions use distinguishes its builder. These two observations are not ZIP 315's stated rationale for a fixed depth, which is that too small a depth risks invalidation by a reorganisation, that too large a depth prevents recently received notes from being spent, and that a depth varying with the notes spent leaks information about them (ZIP 315, "Rationale for anchor selection").

5.3 Authentication paths

Construction 5.14 (Authentication path). Let the pool's tree hold n leaves, namely the tree that the anchor rt of a block roots, for a block at or after the one that appended the note's commitment, so that the note position satisfies $\text{pos} < n$. Write $\text{pos} = \sum_{t=0}^{31} b_t 2^t$ with $b_t \in \{0, 1\}$, and $j_t := \lfloor \text{pos} / 2^t \rfloor$. Levels $t = 0, \dots, 31$ count from the leaves: the *level- t route node* is $M_{j_t}^{32-t}$, a left child if $b_t = 0$ and a right child if $b_t = 1$. The *authentication path* of pos is

$$(s_0, \dots, s_{31}) \in \mathbb{F}_{p_{\text{Pallas}}}^{32}, \quad s_t := M_{j_t \oplus 1}^{32-t},$$

the sibling of the level- t route node in the tree of size n , with $j \oplus 1$ the integer j with its lowest bit complemented. Verification of $(v, \text{pos}, (s_0, \dots, s_{31}))$ against rt is the fold (2): the level- t step evaluates $\text{MerkleCRH}(31 - t, \cdot, \cdot)$, whose message carries the prefix $\text{LE}_{10}(t)$, with the running value on the left if $b_t = 0$ and on the right if $b_t = 1$; verification accepts if and only if $a_{32} = rt$, that is, if the triple is a valid path to rt in the sense of Proposition 5.9. In the protocol specification, §"Merkle Path Validity", the entry s_t is the node $M_{\text{MerkleSibling}(b, \text{pos})}^b$ with $b = 32 - t$.

The genuine path of an appended leaf is accepted: the fold on $(L_{\text{pos}}, \text{pos}, (s_0, \dots, s_{31}))$ reproduces the route nodes, $a_t = M_{j_t}^{32-t}$ for every t , and ends at $M_0^0 = rt$ (*Crypto Guide*, §“Authentication paths and membership proofs”, Proposition “Completeness”, whose induction applies with the level- t compression $\text{MerkleCRH}(31 - t, \cdot, \cdot)$, as in the proof of Proposition 5.9). A note’s commitment therefore has a valid path to the anchor of every block from the one that appended it onward.

Lemma 5.15 (Authentication paths from public data). *Let the tree hold n leaves, let $\text{pos} < n$, and let*

$$\begin{aligned} \text{Empty}_0 &:= \text{Uncommitted}^{\text{Orchard}} = 2, \\ \text{Empty}_{t+1} &:= \text{MerkleCRH}(31 - t, \text{Empty}_t, \text{Empty}_t) \quad (0 \leq t < 32). \end{aligned}$$

For each level t the entry s_t of the authentication path of pos is as follows.

- (i) *If $b_t = 1$, then s_t is the root of the complete subtree over the positions $[(j_t - 1) 2^t, j_t 2^t)$, all below pos ; it is a function of the leaves at those positions alone, and the same in every tree of size greater than pos .*
- (ii) *If $b_t = 0$, then s_t is the root of the subtree over the positions $[(j_t + 1) 2^t, (j_t + 2) 2^t)$, a function of the leaves appended in that range, with $\text{Uncommitted}^{\text{Orchard}}$ at its unused positions; it equals Empty_t when $(j_t + 1) 2^t \geq n$.*

Hence the authentication path is a deterministic function of pos and of the first n leaves, which are the values cmx published by the pool’s Actions in the chain through the anchoring block. A holder of the note who has these leaves computes the path without communicating pos , and verification needs, besides the path, only the public anchor.

Proof. By the recurrence of the tree, the node M_i^b is a function of the leaves at positions $[i 2^{32-b}, (i + 1) 2^{32-b})$, by induction on $32 - b$. Apply this with $b = 32 - t$ and $i = j_t \oplus 1$, which equals $j_t - 1$ when $b_t = 1$ and $j_t + 1$ when $b_t = 0$, since b_t is the parity of j_t . When $b_t = 1$ the range ends at $j_t 2^t \leq \text{pos} < n$, so every position in it is appended, and its leaf is fixed once appended because the tree is append-only. When the range starts at or beyond n , every leaf in it is 2, and its root is Empty_t by induction on t : a level- $(t + 1)$ node over an unused range is $\text{MerkleCRH}(31 - t, \cdot, \cdot)$ of two level- t nodes over unused ranges. Positions below n hold the appended values cmx , published in the chain. $\square$

Figure 2 shows the construction and Lemma 5.15 on a tree of depth 3.

Remark 5.16 (Scope of hiding). The anchor is public. The position, the consumed note’s cmx and its authentication path are not fields of the bundle, and the holder of the note computes the path locally (Lemma 5.15): obtaining the path of a particular position from another party would disclose pos to that party. The candidate set for the consumed note is the candidate set of the anchor (§5.2), the positions of the n appended leaves of the anchored tree. Leaves that an observer knows by other means to be commitments of dummy created notes (§4.4) or of notes already

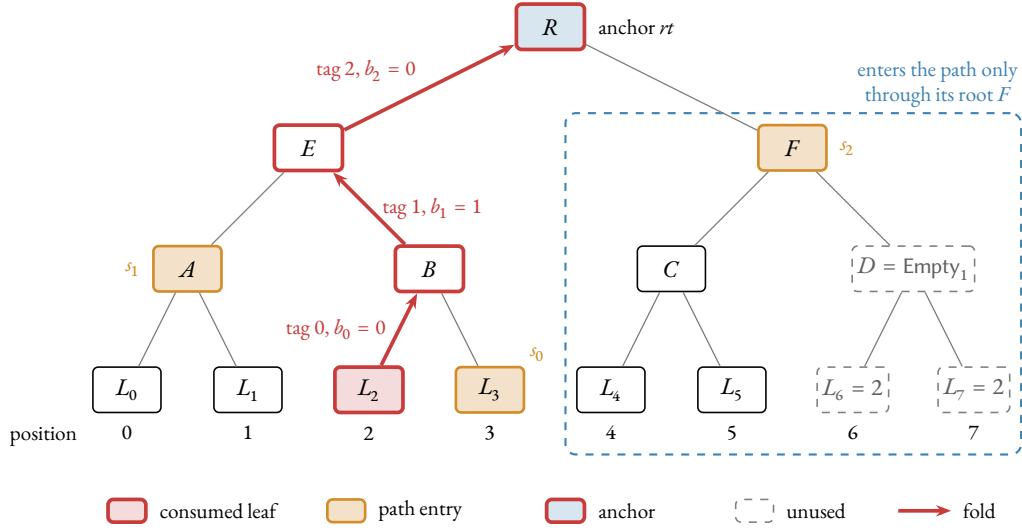


Figure 2. A depth-3 instance of the note commitment tree of §5.1; the protocol’s tree has depth 32. Leaves L_0 to L_5 are appended ($n = 6$); positions 6 and 7 hold $\text{Uncommitted}^{\text{Orchard}} = 2$, so $D = \text{Empty}_1$. The consumed note’s leaf is L_2 , at position 2 with bits $b_0 = 0, b_1 = 1, b_2 = 0$. Its authentication path is $(s_0, s_1, s_2) = (L_3, A, F)$, each entry the sibling of the route node at its level. The bold fold $L_2 \rightarrow B \rightarrow E \rightarrow R$ evaluates MerkleCRH at each level with the prefix $\text{LE}_{10}(t)$, shown as tag t , and the running value on the side given by b_i ; it ends at the root R , the anchor. By Lemma 5.15, the entry A , left of the route, is the root of a complete subtree and the same at every later anchor; the entry F , right of the route and the root over positions 4 to 7, is a function of the leaves appended at the anchor, with 2 at unused positions, and changes when L_6 or L_7 is appended; the entry L_3 , right of the route but already appended, is fixed at every later anchor. The boxed subtree enters the path only through its root F . Every entry is computed from pos and the first n leaves.

consumed, and other data of the transaction, reduce the effective set, so n alone does not measure it. What an Action reveals is stated in Theorem 12.12 (“Privacy”, §12.4).

6 Nullifiers

This section constructs the nullifier of a note, the nullifier sets that consensus maintains per pool, and the rule that makes the nullifier of each consumed note the element ρ of the note created in the same Action. It proves two properties of the construction: uniqueness, from discrete logarithms on Pallas with GroupHash modelled as a random oracle; and, for keys generated with `use_qsk = false`, unlinkability, from these two assumptions, the pseudorandomness of Poseidon and of PRF expansion, and the joint hiding of the viewing keys. It also states what each key tier can compute from nullifiers.

6.1 The nullifier

Requirement R3 of “Requirements on a shielded payment” (§1.4) excludes a second consumption of a note without a public list of consumed notes. Each note therefore determines a *nullifier*, a tag published when the note is consumed, and a verifier (§1.2) rejects a nullifier already in the nullifier set of its pool (§6.2). The construction must make the nullifier

- (a) *deterministic*: a function of the note and of the nullifier deriving key nk of the key that owns it, so that a second consumption publishes the same value; and
- (b) *unlinkable*: to a party without nk , the published value reveals nothing about the note’s commitment or address.

Construction 6.1 (Nullifier PRF). Let $f: \mathbb{F}_{\mathcal{P}_{\text{Pallas}}}^3 \rightarrow \mathbb{F}_{\mathcal{P}_{\text{Pallas}}}^3$ be the Poseidon permutation of width $t = 3$ over the Pallas base field, used as a sponge of rate 2 and capacity 1, with S-box $x \mapsto x^5$, 8 full and 56 partial rounds (*Crypto Guide*, §“Arithmetisation-friendly hashing: Poseidon”, Constructions “The sponge construction” and “The Poseidon permutation”, Example “The Orchard Poseidon instance”). Its round constants and MDS matrix are those of the protocol specification, §“PoseidonHash Function”, and are not reproduced. In constant-input-length mode the hash $\text{PoseidonHash}: \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \times \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \rightarrow \mathbb{F}_{\mathcal{P}_{\text{Pallas}}}$ is

$$\text{PoseidonHash}(x, y) := \pi_1(f(x, y, 2^{65})),$$

where π_1 is the projection onto the first component; the capacity element 2^{65} encodes the input length 2, so no padding is needed (same citations). The *nullifier PRF* is

$$\text{PRF}^{\text{nfOrchard}}: \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \times \mathbb{F}_{\mathcal{P}_{\text{Pallas}}} \rightarrow \mathbb{F}_{\mathcal{P}_{\text{Pallas}}}, \quad \text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) := \text{PoseidonHash}(\text{nk}, \rho),$$

keyed by the nullifier deriving key $\text{nk} \in \mathbb{F}_{\mathcal{P}_{\text{Pallas}}}$ of “The spending key and the spend-side secrets” (§3.1; protocol specification, §“Pseudo Random Functions”).

Assumption 6.2 (Poseidon is a PRF). For nk uniform on $\mathbb{F}_{p_{\text{Pallas}}}$, no efficient adversary with adaptive oracle access distinguishes $\rho \mapsto \text{PoseidonHash}(\text{nk}, \rho)$ from a uniformly random function $\mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$, except with negligible advantage: this is the game of Definition “Pseudorandom function” (§2.3; *Crypto Guide*, §“Pseudorandom functions and permutations”) with key, input and output sets $\mathbb{F}_{p_{\text{Pallas}}}$. It is the security requirement that the protocol specification, §“Pseudo Random Functions”, places on PoseidonHash keyed by its first argument. Its status is cryptanalytic: it is supported by the analysis of the Poseidon round function (protocol specification, §“PoseidonHash Function”, notes) and has no reduction to a standard problem (*Crypto Guide*, Example “The Orchard Poseidon instance”). Of the results of this section only Proposition 6.10 uses it; Proposition 6.8 uses no property of PoseidonHash , and the volume proves no unlinkability result for nullifiers without this assumption.

Definition 6.3 (Nullifier). Let

$$K^{\text{Orchard}} := \text{GroupHash}(z.\text{cash}:\text{Orchard}, \mathbb{K}),$$

the nullifier base, a fixed generator of Table 2. For $\text{nk}, \rho, \psi \in \mathbb{F}_{p_{\text{Pallas}}}$ and a point $\text{cm} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$,

$$\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) := \text{Extract}_{\mathbb{P}}([s] K^{\text{Orchard}} + \text{cm}) \in \mathbb{F}_{p_{\text{Pallas}}},$$

where

$$s := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}} \in \{0, \dots, p_{\text{Pallas}} - 1\} :$$

the sum of the two field elements is taken as its representative in $\{0, \dots, p_{\text{Pallas}} - 1\}$, and that integer multiplies K^{Orchard} . The *nullifier* of a note $n = (\text{addr}, v, \rho, \psi, \text{rcm})$ whose note commitment cm (Definition 4.4) is not $\perp$, under the nullifier deriving key nk of the key that owns addr , is

$$\text{nf} := \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$$

(protocol specification, §“Computing ρ values and Nullifiers”).

Remark 6.4 (Reduction modulo p_{Pallas}). The protocol specification attaches two notes to the definition (§“Computing ρ values and Nullifiers”). First, the sum is reduced modulo p_{Pallas} intentionally, although the scalar field of the Pallas group is $\mathbb{F}_{p_{\text{Vesta}}}$. Since $p_{\text{Pallas}} < p_{\text{Vesta}}$ (§2.1), the representative in $\{0, \dots, p_{\text{Pallas}} - 1\}$ is a scalar without further reduction, by the injective reading $\mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Vesta}}}$ of §2.1, and the multiplier of K^{Orchard} ranges over p_{Pallas} of the p_{Vesta} scalars. Second, the inputs ρ and ψ must be those of the note committed to by cm .

Remark 6.5 (Determinism and masking). For a fixed nk the nullifier is a function of the fields ρ and ψ of the note and of its commitment cm , so a note has exactly one nullifier under the nk of its owner. That every accepted consumption of the note publishes this value is Lemma 12.5, proved in “Double-spend resistance” (§12.2) from the Action statement, defined in “The Action statement” (§9.2). The point $[s] K^{\text{Orchard}}$ is added to cm before extraction. For notes of one key with pairwise distinct ρ , the key generated with $\text{use_qsk} = \text{false}$, the multipliers are jointly pseudorandom to an efficient party that holds the incoming viewing key and $\text{ak}^{\mathbb{P}}$ but not nk , so the extracted values give it no usable information on the commitments or on the address; Proposi-

tion 6.10 states this exactly, under Assumption 6.2 and the further assumptions it names. The element ρ enters as the input of the PRF and ψ as an additive term of the multiplier. Both are fixed when the note is created: ρ by “Nullifier chaining” (§6.3), and ψ by the derivation of §4.3.

6.2 Nullifier sets

Each verifier maintains, for each shielded pool, a *nullifier set*, part of that pool’s treestate (“Anchors”, §5.2). Every Action publishes one nullifier, that of its consumed note, real or dummy (§4.4), and the nullifiers published by the Actions of valid transactions are inserted into the nullifier set of their pool (protocol specification, §“Nullifiers” and §“Nullifier Sets”). The consensus rule of §“Nullifier Sets” is that a nullifier must not repeat, either within a transaction or across transactions in a valid block chain; a nullifier is compared only with nullifiers of its own pool, so equal values in two pools do not repeat a nullifier (the specification treats the pools’ nullifiers as disjoint).

Both pools use the one derivation of Definition 6.3, with the same PRF and the same nullifier base: the nullifier construction belongs to the Orchard protocol, not to a pool (ZIP 229, Terminology). The Orchard pool and the Ironwood pool keep separate nullifier sets, as they keep separate note commitment trees, anchors and value pool balances (ZIP 229, Rationale, “Separate state”; protocol specification, §“Nullifier Sets”). The nullifier of an Ironwood-pool Action is inserted into, and checked against, the nullifier set of the Ironwood pool only.

Remark 6.6 (Pool locality). Equal nullifier values in the two pools do not violate the consensus rule, so every uniqueness property derived from it holds within one pool. In particular the uniqueness of the elements ρ proved in “Double-spend resistance” (Corollary 12.7) is uniqueness within a pool, not across the two pools.

6.3 Nullifier chaining

Construction 6.7 (Chaining rule). In every Action the sender sets the element ρ of the created note to the nullifier that the same Action publishes for its consumed note,

$$\rho^{\text{new}} := \text{nf}^{\text{old}} \in \mathbb{F}_{\mathcal{P}_{\text{Pallas}}},$$

and encodes it as the 32-byte string $\text{I2LEOSP}_{256}(\rho^{\text{new}})$. When the Action consumes no real note, its consumed side is a dummy note (Definition “Dummy notes”, §4.4), and the nullifier of that dummy note is ρ^{new} (protocol specification, §“Sending Notes (Orchard)”, §“Dummy Notes (Orchard)” and §“Faerie Gold attack and fix”).

Every Action has a consumed side (Definition “Action”, §4.4), so every note that an Action creates is constructed with ρ equal to the nullifier of that Action’s consumed note. The leaves that enter a pool’s note commitment tree are the extracted commitments of notes created by that pool’s Actions (“The Merkle hash and the tree”, §5.1), so the rule covers every note whose extracted commitment enters the tree. The consumed dummy note is not created by an Action: its ρ is $\text{Extract}_{\mathcal{P}}$ of a uniformly random point of the Pallas group (§4.4).

The derivations of §4.3 take the chained ρ as input: with $\underline{\rho} := \text{I2LEOSP}_{256}(\rho)$,

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([\text{0x04}] \parallel \underline{\rho})), \quad \psi = \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([\text{0x09}] \parallel \underline{\rho})),$$

and rcm is derived from the string pre_rcm , which contains $\underline{\rho}$ and the encoding of ψ . Hence esk and ψ are functions of rseed and of the nullifier that the note's own Action publishes, and rcm is a function of these and of the remaining note fields.

A *Faerie Gold attack* is one in which a sender creates several notes to one recipient that share a nullifier, so that only one of them can be spent (protocol specification, §“Faerie Gold attack and fix”). What a verifier can rely on, the uniqueness of the elements ρ within a pool and resistance to the Faerie Gold attack, is proved in “Double-spend resistance” (Corollary 12.7) from the nullifier-set rule and the Action statement, defined in “The Action statement” (§9.2). This subsection states the rule that an honest sender follows and draws no conclusion about accepted notes from it.

6.4 Nullifier uniqueness and unlinkability

Proposition 6.8 (Nullifier uniqueness). *Under Assumptions 2.22 and 2.8, no efficient adversary outputs, except with negligible probability, two notes $n = (\text{addr}, v, \rho, \psi, \text{rcm})$ and $n' = (\text{addr}', v', \rho', \psi', \text{rcm}')$ and two nullifier deriving keys $\text{nk}, \text{nk}' \in \mathbb{F}_{p_{\text{Pallas}}}$, all of its choice, whose note commitments cm and cm' are not $\perp$ and satisfy $\text{cm} \neq \text{cm}'$ and*

$$\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm}) = \text{DeriveNullifier}_{\text{nk}'}(\rho', \psi', \text{cm}').$$

The adversary may hold every nullifier deriving key, and $\text{nk} = \text{nk}'$ and $\rho = \rho'$ are admitted. Since two distinct notes have distinct commitments except with negligible probability (Proposition 4.5(b)), the same holds for two distinct notes in place of two distinct commitments.

Proof. The adversary supplies both openings, so no knowledge extraction is used. Let its output satisfy the conditions, and write

$$\begin{aligned} K &:= K^{\text{Orchard}}, \\ Q &:= Q(\text{z.cash:Orchard-NoteCommit-M}), \\ H_D &:= \text{GroupHash}(\text{z.cash:Orchard-NoteCommit-r}, \varepsilon) \end{aligned}$$

for the nullifier base and for the hash base and the blinding base of NoteCommit (Definition 4.4). A reduction that runs the adversary computes $s := (\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho) + \psi) \bmod p_{\text{Pallas}}$ and s' likewise; the pieces $m = (m_1, \dots, m_{109})$ and m' of the messages $M(n)$ and $M(n')$, which have 109 chunks of $k = 10$ bits (Table 4); and rcm, rcm' . Every coefficient below is therefore known to it, the PRF outputs included, and no property of PoseidonHash is used.

Equal nullifiers are equal extracted coordinates, so by Lemma 2.4

$$[s] K + \text{cm} = \theta([s'] K + \text{cm}') \quad \text{for some } \theta \in \{1, -1\}.$$

Since $\text{cm} \neq \perp$, its hash point is not $\perp$, and by the unrolled form (1) of the proof of Proposition 2.24,

$$\text{cm} = [2^{109}] Q + \sum_{j=0}^{2^k-1} [\chi_j(m)] S(j) + [\text{rcm}] H_D, \quad \chi_j(m) = \sum_{i=1}^{109} 2^{109-i} \delta_{m_i, j},$$

and likewise for cm' . Substituting gives, with coefficients in $\mathbb{F}_{p_{\text{Vesta}}}$ and s, s' read as scalars,

$$[s - \theta s'] K + [(1 - \theta) 2^{109}] Q + \sum_j [\chi_j(m) - \theta \chi_j(m')] S(j) + [\text{rcm} - \theta \text{rcm}'] H_D = \mathcal{O}. \quad (3)$$

The relation is non-trivial in each case.

- If $\theta = -1$, the coefficient of Q is 2^{110} , non-zero modulo p_{Vesta} because $0 < 2^{110} < 2^{253} \leq (p_{\text{Vesta}} - 1)/2$.
- If $\theta = 1$ and $M(n) \neq M(n')$, then $m \neq m'$, and $\chi(m) \neq \chi(m')$ modulo p_{Vesta} by step (1) of the proof of Proposition 2.24 ($109 \leq c = 253$); some coefficient of an $S(j)$ is non-zero.
- If $\theta = 1$ and $M(n) = M(n')$, then $\text{rcm} \neq \text{rcm}'$, since equal messages and equal trapdoors give $\text{cm} = \text{cm}'$; the coefficient of H_D is non-zero.

The points $K, Q, S(0), \dots, S(2^k - 1)$ and H_D are values of GroupHash on pairwise distinct inputs (Table 2). The reduction outputs these inputs with the coefficients of (3), a non-trivial relation with known coefficients. By the lemma on relations among fixed generators (§2.4), the reduction behind Proposition 2.24, this happens with negligible probability under Assumptions 2.22 and 2.8. The specification records that its Sinsemilla collision argument extends to additional terms with independent bases for this purpose (protocol specification, §“Sinsemilla Hash Function”, Security argument, note; §“Faerie Gold attack and fix”).

For distinct notes whose commitments are not $\perp$, equal commitments have negligible probability by Proposition 4.5(b), and distinct commitments are the case above. $\square$

Remark 6.9 (Scope of Proposition 6.8). The proposition uses no pseudorandomness of PoseidonHash, and none could be used: the adversary holds the PRF keys, and the reduction knows every coefficient. It does not require distinct elements ρ , which the specification’s security requirement assumes (§“Computing ρ values and Nullifiers”). Two notes with equal commitments are equal except with negligible probability (Proposition 4.5(b)) and so have equal nullifiers under one nk. Excluding equal commitments among the notes of a pool is the role of the chaining rule of §6.3, as Corollary 12.7 proves.

Proposition 6.10 (Nullifier unlinkability). *Let a key be generated with $\text{use_qsk} = \text{false}$ from a uniform spending key, as in §3.1 and §3.2, with nullifier deriving key nk. An adversary receives the incoming viewing key (dk, ivk) and the point ak^P of the key, but not nk. It then submits, adaptively, notes $n_i = (\text{addr}_i, v_i, \rho_i, \psi_i, \text{rcm}_i)$, $1 \leq i \leq q$, of its choice, with pairwise distinct ρ_i and note commitments $\text{cm}_i \neq \perp$, and after each submission receives an answer:*

- in the real game, $\text{nf}_i := \text{DeriveNullifier}_{\text{nk}}(\rho_i, \psi_i, \text{cm}_i)$;

- in the ideal game, $\text{Extract}_p([u_i] K^{\text{Orchard}} + \text{cm}_i)$ for u_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, independent of each other and of all else.

It outputs a bit. Let $\delta := (p_{\text{Vesta}} - p_{\text{Pallas}}) / p_{\text{Vesta}}$, which is below 2^{-167} , the statistical distance between the uniform distribution on $\{0, \dots, p_{\text{Pallas}} - 1\}$, read in $\mathbb{F}_{p_{\text{Vesta}}}$, and the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$. Then the difference of the probabilities that it outputs 1 in the two games is at most

$$2(\epsilon_{\text{key}} + \epsilon_{\text{expand}} + 3 \cdot 2^{-257} + \epsilon_{\text{vk}}) + \epsilon_{\text{pos}} + q \delta,$$

where ϵ_{expand} , ϵ_{vk} and ϵ_{pos} are the largest advantages against Assumptions 2.12 (through Lemma 2.15), 3.13 and 6.2 of the algorithms built from the adversary in the proof, and ϵ_{key} is the probability that a single draw of key generation from a uniform spending key is rejected plus the probability that $\text{ask} = 0$ or $\hat{M} = \perp$ for independent uniform (ask, nk) , with $\hat{M}$ the hash point of $\text{Commit}^{\text{ivk}}$ on (ak, nk) (§3.2). The term ϵ_{key} is negligible under Assumptions 2.12, 2.22 and 2.8 (Lemma “Rejection in key generation”, §3.2); the bound is therefore negligible for every efficient adversary, whose number q of queries is polynomial, under these assumptions and Assumptions 3.13 and 6.2. When $K^{\text{Orchard}} \neq \mathcal{O}$, the point $[u_i] K^{\text{Orchard}} + \text{cm}_i$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ whatever cm_i is, so the ideal answers are independent of the notes and of each other; in the model of Assumption 2.8, $K^{\text{Orchard}} = \mathcal{O}$ has probability $1/p_{\text{Vesta}}$.

Proof. Write $K := K^{\text{Orchard}}$. The proof passes from the real game H_0 through hybrid games $H_1, \dots, H_5$; each hop changes the probability of the output 1 by at most the amount stated (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”), and each reduction runs the adversary and the rest of the game.

Hybrid H_1 : idealised key. The key is generated by a single draw from a uniform sk , without rejection, at the cost of the probability that the draw is rejected (Lemma “Rejection in key generation”, §3.2). Then $(\text{ask}, \text{nk}, r_{\text{ivk}})$ is replaced by independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, from which the other components are derived as in §3.1 and §3.2. By Lemma 2.15, this costs at most ϵ_{expand} plus the three reduction distances, each below 2^{-257} (§2.3): the three inputs of $\text{PRF}_{\text{expand}}^{\text{sk}}$ have distinct lead bytes, and the game is an efficient function of the triple, which derives ak^{P} , ivk and dk , runs the adversary and answers each query with the nullifier under nk .

Hybrid H_2 : joint hiding. When $\text{ask} \neq 0$, the pair (ivk, dk) is replaced by $(\text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}, (\text{ak}, \text{nk}), u)$, for r_{ivk} uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and u a uniform 32-byte string, independent of all else. The cost is at most ϵ_{vk} . The reduction for Assumption 3.13 draws ask and nk itself; for $\text{ask} \neq 0$ the pair (ak, nk) lies in the domain of the assumption (§3.1). It receives a triple for that pair, passes its first two components to the adversary as ivk and dk , together with ak^{P} , and answers every query with the nullifier under its own nk . Giving ak^{P} costs nothing, because after H_1 the scalar ask is independent of $(\text{nk}, r_{\text{ivk}})$ and the assumption holds for every fixed (ak, nk) .

Hybrid H_3 : independent viewing keys. The pair (ivk, dk) is replaced, whatever ask is, by $(\text{Extract}_p(W), u)$ for a uniform point W and a uniform 32-byte string u , independent of all else. When $\text{ask} \neq 0$ and the hash point $\hat{M}$ of the message $\text{LE}_{255}(\text{ak}) \parallel \text{LE}_{255}(\text{nk})$ is not $\perp$, the commitment point $\hat{M} + [r_{\text{ivk}}'] H_D$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ by Lemma 2.26(i), its blinding base H_D not being $\mathcal{O}$ (§3.2), so the two games are identically distributed conditioned on that event, which depends on (ask, nk) only. The cost is at most the probability that $\text{ask} = 0$ or $\hat{M} = \perp$, the

remaining part of ϵ_{key} . In H_3 the key nk enters the game only through the answers: dk , ivk and ak^{p} are independent of it.

Hybrid H_4 : random function. The function $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$ is replaced by a uniformly random function $F: \mathbb{F}_{p_{\text{Pallas}}} \rightarrow \mathbb{F}_{p_{\text{Pallas}}}$. The cost is at most ϵ_{pos} : the reduction for Assumption 6.2 runs H_3 with its oracle in place of $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$, since nk , uniform on $\mathbb{F}_{p_{\text{Pallas}}}$, is used nowhere else. In H_4 the i -th answer is $\text{Extract}_{\text{p}}([s_i] K + \text{cm}_i)$ with $s_i := (F(\rho_i) + \psi_i) \bmod p_{\text{Pallas}}$. The ρ_i are pairwise distinct, so, with F sampled lazily, $F(\rho_i)$ is uniform on $\mathbb{F}_{p_{\text{Pallas}}}$ and independent of everything the adversary has received when it submits n_i . Translation by ψ_i is a bijection modulo p_{Pallas} , so s_i is uniform on $\{0, \dots, p_{\text{Pallas}} - 1\}$ and independent of that view, whatever ψ_i is.

Hybrid H_5 : uniform scalars. For $i = 1, \dots, q$ in turn, the scalar s_i , read in $\mathbb{F}_{p_{\text{Vesta}}}$ (§2.1), is replaced by u_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. The two distributions differ by $1/p_{\text{Pallas}} - 1/p_{\text{Vesta}}$ on each of the p_{Pallas} residues and by $1/p_{\text{Vesta}}$ on each of the other $p_{\text{Vesta}} - p_{\text{Pallas}}$ scalars, so their statistical distance is

$$\frac{1}{2} \left(p_{\text{Pallas}} \left(\frac{1}{p_{\text{Pallas}}} - \frac{1}{p_{\text{Vesta}}} \right) + \frac{p_{\text{Vesta}} - p_{\text{Pallas}}}{p_{\text{Vesta}}} \right) = \frac{p_{\text{Vesta}} - p_{\text{Pallas}}}{p_{\text{Vesta}}} = \delta.$$

The rest of the game is a randomised function of the view before the i -th answer and of the replaced value, which is independent of that view; each step therefore costs at most δ , and the q steps at most $q\delta$ (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, parts (2) and (4)).

In H_5 the answers are those of the ideal game, and the key material is that of H_3 . Neither in H_5 nor in the ideal game do the answers depend on the key, so the hops of H_3 , H_2 and H_1 , taken in reverse order with the answers of the ideal game, lead from H_5 to the ideal game at the same costs. Summing the costs of both passes gives the bound. The last statement of the proposition holds because, for $K \neq \mathcal{O}$, the map $u \mapsto [u]K$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group of prime order p_{Vesta} , and translation by cm_i is a bijection of the group; under Assumption 2.8 the point K is uniform. $\square$

Remark 6.11 (Design statement, not proved). The specification describes the nullifier as combining elliptic-curve cryptography with the Poseidon-based PRF so as to give privacy defence in depth against a weakness in either (protocol specification, §“Faerie Gold attack and fix”). It states that, under the weaker assumption that PoseidonHash preserves sufficient entropy from nk and ρ , the construction would remain unlinkable under a decisional Diffie–Hellman assumption on Pallas even if the Poseidon-based PRF were distinguishable from an ideal PRF (§“PoseidonHash Function”, note; *Crypto Guide*, §“Diffie–Hellman: computational and decisional”, Definition “Decisional Diffie–Hellman, DDH”; the form assumed in this volume is Assumption 3.16). The volume records this as the specification’s design statement and derives no result from it; Proposition 6.10 relies on Assumption 6.2 instead.

For keys with `use_qsk = false`, the key tiers of Proposition 3.18 (“Capability separation”) have the following powers over nullifiers. The full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ contains nk . Its holder computes $\text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ for every note of the key whose fields it knows, and tests whether that value is in the nullifier set of the note’s pool; that every accepted consumption of the note publishes this value is Lemma 12.5. It does so without spending authority, since the full viewing key yields no ask (Proposition 3.18(a)). A holder of only the incoming viewing key (dk, ivk) cannot link a nullifier to its note, for notes with pairwise distinct ρ , by Proposition 6.10.

Remark 6.12 (Limits). Proposition 6.10 gives nothing against a holder of nk , which includes every holder of the full viewing key: such a holder recomputes, and so links, the nullifier of every note of the key whose fields it knows. It does not concern links through other public data, such as timing or the structure of the transaction; the privacy of an Action, with its preconditions, is Theorem 12.12, in “Privacy” (§12.4).

7 Spend authorisation

This section instantiates RedDSA, the re-randomisable Schnorr signature scheme of the *Crypto Guide*, on the Pallas group; constructs the randomised validating key and the spend-authorisation signature of an Action; and proves two properties of them: the published key is, with the signature scheme’s hash modelled as a random oracle, within negligible statistical distance of a key independent of the spend validating key; and a new valid signature under a randomised key yields the key’s discrete logarithm. The signed message is a parameter throughout; “Transaction digests and signatures” (§11.3) fixes it.

7.1 The RedPallas signature scheme

Remark 7.1 (Spending authority and its signature). Requirement R5 of “Requirements on a shielded payment” (§1.4) admits only the holder of a note’s spending authority as a party that consumes the note. For keys generated with $use_qsk = \text{false}$, the spending authority over the notes of every address derived from a spending key is knowledge of its spend authorising key ask of “The spending key and the spend-side secrets” (§3.1), the discrete logarithm of $ak^P = [ask] G^{\text{SpendAuth}}$ to the base $G^{\text{SpendAuth}}$. No lower key tier yields ask , since every lower tier is computed from the full viewing key (Proposition 3.18(a)). Each Action therefore carries a signature that only a holder of ask can produce, under a validating key that does not link Actions consuming notes of the same key: a fresh re-randomisation of ak^P (§7.2), never ak^P itself. Theorem 12.9, in “Authorisation of spends” (§12.3), proves for keys with $use_qsk = \text{false}$ that no efficient party without ask produces an accepted Action that consumes a note of an honestly generated address and whose spend-authorisation signature is on a digest that the key holder did not sign under the Action’s randomised key, except with negligible probability.

Definition 7.2 (RedPallas hash). The *RedPallas hash* is the map H from byte strings to 64-byte strings

$$H(z) := \text{BLAKE2b-512}(\text{Zcash_RedPallasH}, z),$$

with the notation of the expansion function (§2.3) and the 16-byte personalisation Zcash_RedPallasH . Its reduction to scalars is

$$H^{\circledast}(z) := \text{LEOS2IP}_{512}(H(z)) \bmod p_{\text{Vesta}} = \text{ToScalar}(H(z)) \in \mathbb{F}_{p_{\text{Vesta}}}$$

(protocol specification, §“RedDSA, RedJubjub, and RedPallas” and §“BLAKE2 Hash Functions”). The symbol $H^{\circledast}$ is that of the specification and of the *Crypto Guide*’s Remark “RedDSA as deployed” (§“RedDSA: re-randomisable Schnorr for Orchard”); it is distinct from the star encoding P^* of a point.

Assumption 7.3 (The RedPallas hash as a random oracle). In security arguments the RedPallas hash H is modelled as a random oracle with output length 512, that is, with range the 64-byte strings, queried by every party (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”). Consequently $H^\circledast$ is a random function into $\mathbb{F}_{p_{\text{Vesta}}}$: its values at distinct inputs are independent, and each is distributed as $\text{ToScalar}(U)$ for U uniform on the 64-byte strings. That distribution is at statistical distance at most

$$\delta_{512} := p_{\text{Vesta}} / 2^{512} < 2^{-257}$$

from the uniform distribution on $\mathbb{F}_{p_{\text{Vesta}}}$ (*Math Guide*, §“Uniform sampling and the bias of modular reduction”, Proposition “Bias of modular reduction”, with $L = 512$), and it gives each scalar probability at most $1/p_{\text{Vesta}} + 2^{-512}$, since each residue has at most $\lfloor 2^{512}/p_{\text{Vesta}} \rfloor + 1$ preimages among the 2^{512} strings. The assumption is a heuristic about BLAKE2b, not a theorem. Every later result on RedPallas signatures or on the randomiser generator below names it.

Construction 7.4 (RedPallas). RedPallas is the *Crypto Guide*’s Construction “RedDSA” (§“RedDSA: re-randomisable Schnorr for Orchard”) on the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, of prime order p_{Vesta} , with both hashes $H^\circledast$ and with a base point $B \neq \mathcal{O}$ as parameter; the group having prime order, B generates it. For a point P let $\underline{P} := \text{LEBS2OSP}_{256}(P^*)$, its 32-byte encoding; the map $P \mapsto \underline{P}$ is injective, as the star encoding is (§2.1) and LEBS2OSP_{256} is a bijection onto the 32-byte strings. Messages are byte strings.

1. *Keys*. A signing key is a scalar $x \in \mathbb{F}_{p_{\text{Vesta}}}$; its validating key is $X := [x] B$.
2. *Signing* a message M under x : draw T uniformly from the 80-byte strings, where $80 = (512 + 128)/8$ is the specification’s randomness length for a 512-bit hash; let

$$\begin{aligned} r &:= H^\circledast(T \parallel \underline{X} \parallel M), & R &:= [r] B, \\ c &:= H^\circledast(\underline{R} \parallel \underline{X} \parallel M), & S &:= r + c \cdot x \in \mathbb{F}_{p_{\text{Vesta}}}, \end{aligned}$$

and output the 64-byte signature $\sigma := \underline{R} \parallel \text{l2LEOSP}_{256}(S)$.

3. *Validation* of a 64-byte string σ under a point X on M : let $\bar{R}$ be the first 32 bytes of σ and $\bar{S}$ the last 32; let $R := \text{abst}_p(\text{LEBS2OSP}_{256}^{-1}(\bar{R}))$, $S := \text{LEOS2IP}_{256}(\bar{S})$ and $c := H^\circledast(\bar{R} \parallel \underline{X} \parallel M)$. Accept if and only if $R \neq \perp$, $S < p_{\text{Vesta}}$ and

$$[S] B = R + [c] X. \tag{4}$$

(Protocol specification, §“RedDSA, RedJubjub, and RedPallas”; its validation equation carries the cofactor, which is 1 on Pallas.)

The partial inverse abst_p returns $\perp$ on every 256-bit string that is not the star encoding of a point (§2.1); validation therefore rejects a non-canonical $\bar{R}$, and an accepted signature has $\bar{R} = \underline{R}$. An honest signature is accepted: its first half $\underline{R}$ decodes to R , its S is an integer representative below p_{Vesta} , the verifier recomputes the signer’s c , and $[S] B = [r] B + [c \cdot x] B = R + [c] X$ (*Crypto*

Guide, §“From identification to signature via the Fiat–Shamir transform”, Proposition “Perfect correctness”). Both the nonce hash and the challenge hash take the encoded validating key $\underline{X}$: the scheme is *key-prefixed*. The message M is a parameter: the message of every RedPallas signature in this volume is the signature digest constructed in “Transaction digests and signatures” (§11.3), and no argument before that subsection depends on its value.

Definition 7.5 (Spend-authorisation and binding-signature instances). The volume uses two instances of RedPallas, which share the signing and validation algorithms and differ in the base B and in the use of key re-randomisation (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)” and §“Binding Signature (Sapling and Orchard)”):

1. the *spend-authorisation instance*, with key re-randomisation (§7.2) and base

$$B = G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:0rchar}, 6);$$

2. the *binding-signature instance*, without re-randomisation, used in “The binding signature” (§8.3), with base the value-commitment randomness base

$$B = R^{\text{Orchard}} = \text{GroupHash}(\text{z.cash:0rchar-cv}, r).$$

Both bases are rows of Table 2 (§2.2); the specification takes each as the generator parameter of RedDSA.

Definition 7.6 (Randomiser generator and key re-randomisation). The *randomiser generator* GenRandom draws T uniformly from the 80-byte strings and returns $H^{\otimes}(T) \in \mathbb{F}_{p_{\text{Vesta}}}$ (protocol specification, §“RedDSA, RedJubjub, and RedPallas”); its distance from uniform is bounded in §7.2. *Re-randomisation* by a randomiser $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ maps a key pair (x, X) to $(x + \alpha, X + [\alpha] B)$ (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Definition “Key re-randomisation”).

Remark 7.7 (Key prefixing). Let σ be valid under X on M , with decoded components R and S and challenge $c = H^{\otimes}(\underline{R} \parallel \underline{X} \parallel M)$, and let $X' := X + [\beta] B \neq \mathcal{O}$ for a scalar $\beta \neq 0$. The shifted string $\sigma' := \underline{R} \parallel \text{I2LEOSP}_{256}(S + c\beta)$ satisfies $[S + c\beta] B = R + [c] X'$. Validation of σ' under X' , however, recomputes the challenge as $c' := H^{\otimes}(\underline{R} \parallel \underline{X}' \parallel M)$, the value of the oracle at an input other than that of c , since $\underline{X}' \neq \underline{X}$; and (4) holds for σ' under X' only if $[c - c'] X' = \mathcal{O}$, that is, only if $c' = c$. Under Assumption 7.3, for each β the value c' is independent of c and equals it with probability at most $1/p_{\text{Vesta}} + 2^{-512}$. A signature is therefore not transported from one key to a related key by shifting its response. With a challenge that omitted the key, $c' = c$ always, and the shift would turn every signature under X into one under X' (*Crypto Guide*, §“From identification to signature via the Fiat–Shamir transform”, Remark “Two conventions”; §“Key re-randomisation and unlinkability”, Remark “Unforgeability under re-randomised keys, and the key-prefixing subtlety”). The proof of Proposition 7.12, which admits randomisers chosen by the adversary, uses key prefixing in its steps (3) and (5).

7.2 Randomised validating keys

Construction 7.8 (Randomised validating key). For each Action the spender draws a *spend-authorisation randomiser* $\alpha := \text{GenRandom}()$, independently of every other draw, and sets

$$\text{rsk} := \text{ask} + \alpha \in \mathbb{F}_{p_{\text{Vesta}}}, \quad \text{rk} := \text{ak}^P + [\alpha] G^{\text{SpendAuth}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}),$$

with ask the sign-normalised spend authorising key of the spending key of the consumed note and ak^P its spend validating key (§3.1). Since $\text{ak}^P = [\text{ask}] G^{\text{SpendAuth}}$, it follows that $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$ (*Crypto Guide*, §“Key re-randomisation and unlinkability”, Proposition “Re-randomisation consistency”), so (rsk, rk) is a key pair of the spend-authorisation instance. The Action publishes the *randomised validating key* rk and its *spend-authorisation signature*, the RedPallas signature under rsk on the signature digest constructed in “Transaction digests and signatures” (§11.3); it publishes none of ask, α , rsk , ak^P and ak (protocol specification, §“Spend Authorization Signature (Sapling and Orchard)”).

Lemma 7.9 (Distance of GenRandom from uniform). *Under Assumption 7.3, let an algorithm, the observer, make at most q_b queries to H and, at one point, choose a byte string w and receive $\alpha := H^{\otimes}(T \parallel w)$, for T uniform on the 80-byte strings and independent of all else. The pair formed by the observer’s view and α is within statistical distance*

$$\delta_{512} + q_b \cdot 2^{-640} \tag{5}$$

of the pair obtained when α is replaced by a uniform element of $\mathbb{F}_{p_{\text{Vesta}}}$ independent of all else. A randomiser from GenRandom is the case $w = \varepsilon$; the nonce of a RedPallas signature under X on M is the case $w = \underline{X} \parallel M$. The first term of (5) is set by the 512-bit output of H , not by the 640-bit length of T ; the length of T enters only the second term.

Proof. Let E be the event that the observer queries H at $T \parallel w$. In a second experiment, $H(T \parallel w)$ is replaced, in the computation of α only, by a uniform 64-byte string U independent of all else. In the lazy realisation of the random oracle (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”) the answers at inputs other than $T \parallel w$ are independent of $H(T \parallel w)$, so the two experiments are identical until E occurs; E has the same probability in both, and their outputs are within statistical distance $\Pr[E]$ (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”). In the second experiment the observer’s view is independent of T , so each query agrees with $T \parallel w$ in its first 80 bytes with probability at most 2^{-640} , and $\Pr[E] \leq q_b \cdot 2^{-640}$ by the union bound. There U is independent of H , of T and of the observer’s coins, and the pair of view and α is a function of $\alpha = \text{ToScalar}(U)$ and of them. Replacing α by a uniform scalar changes the pair by at most δ_{512} (Assumption 7.3; *Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (4)). The triangle inequality, part (2) of the same theorem, gives (5). $\square$

Two consensus rules of the protocol specification, §“Action Descriptions”, concern spend authorisation:

1. the randomised validating key rk of an Action must not be the identity $\mathcal{O}$;

2. the spend-authorisation signature of the Action must be valid under rk on the signature digest of “Transaction digests and signatures” (§11.3).

Validation rejects a non-canonical encoding of the signature’s point component (§7.1). An honestly computed rk equals $\mathcal{O}$ only if $\alpha = -\text{ask}$; for ask fixed before α is drawn, this event has probability at most $1/p_{\text{Vesta}} + 2^{-512}$ (Assumption 7.3).

Proposition 7.10 (Unlinkability of randomised validating keys). *Let $\text{ak}_1^{\text{P}}, \dots, \text{ak}_n^{\text{P}}$ be points of the Pallas group, equal or distinct, and let $\text{rk}_i := \text{ak}_i^{\text{P}} + [\alpha_i] G^{\text{SpendAuth}}$ for randomisers $\alpha_1, \dots, \alpha_n$ independent of each other and of the points.*

(i) *If each α_i is uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, the tuple $(\text{rk}_1, \dots, \text{rk}_n)$ is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})^n$, whatever the points ak_i^{P} , and hence independent of them. For each i and the scalar ask_i with $\text{ak}_i^{\text{P}} = [\text{ask}_i] G^{\text{SpendAuth}}$, the pair $(\text{ask}_i + \alpha_i, \text{rk}_i)$ is distributed exactly as a fresh key pair $(\gamma, [\gamma] G^{\text{SpendAuth}})$ with γ uniform, independently of ask_i . The distribution of the tuple is thus the same for every assignment of the points, and no observer, whatever its running time, distinguishes two assignments, for instance one in which two Actions share a key from one in which they do not, with non-zero advantage.*

(ii) *If each α_i is drawn by GenRandom, then under Assumption 7.3, for an observer that receives the tuple and makes at most q_b queries to H , the pair of its view and the tuple is within statistical distance*

$$n(\delta_{512} + (q_b + n) \cdot 2^{-640})$$

of the pair obtained from a uniform tuple independent of H ; the pairs arising from two assignments of the points are therefore within statistical distance $2n(\delta_{512} + (q_b + n) \cdot 2^{-640})$.

Proof. (i) For each i this is the *Crypto Guide*’s Theorem “Perfect unlinkability of re-randomised keys” (§“Key re-randomisation and unlinkability”). The base $G^{\text{SpendAuth}}$ generates the Pallas group, of prime order p_{Vesta} (§7.1), so $\alpha \mapsto [\alpha] G^{\text{SpendAuth}}$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto the group, and translation by ak_i^{P} is a bijection of the group; hence rk_i is uniform whatever ak_i^{P} is, and the independence of the α_i gives the product distribution. The scalar $\text{ask}_i + \alpha_i$ is uniform for every fixed ask_i because α_i is, and $\text{rk}_i = [\text{ask}_i + \alpha_i] G^{\text{SpendAuth}}$. Equal distributions give every distinguisher advantage zero (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (3)).

(ii) The randomisers are replaced by independent uniform scalars one at a time, for $i = 1, \dots, n$. At step i the lemma on the distance of GenRandom from uniform applies, with $w = \varepsilon$, to the observer extended by the rest of the experiment: it holds the points and $\alpha_1, \dots, \alpha_{i-1}$, already uniform; it draws $T_{i+1}, \dots, T_n$ itself and queries H at them; it receives α_i , computes the tuple and runs the given observer. It makes at most $q_b + n - i$ queries, and the given observer’s view and the tuple are a function of its view and α_i . By (5) and part (4) of the same theorem, step i changes the pair of view and tuple by at most $\delta_{512} + (q_b + n - i) \cdot 2^{-640}$. By the triangle inequality, the n steps together change it by at most

$$n \delta_{512} + \left(n q_b + \frac{n(n-1)}{2} \right) 2^{-640} \leq n(\delta_{512} + (q_b + n) \cdot 2^{-640}).$$

After the last step the α_i are uniform, independent and independent of H , so by (i) the tuple is uniform and independent of H , for every assignment of the points. The triangle inequality through that common distribution gives the factor 2 for two assignments. $\square$

Remark 7.11 (Scope of Proposition 7.10). The proposition concerns rk alone, for randomisers drawn by the honest spender; a randomiser chosen by an adversary is the subject of Proposition 7.12 (§7.3). The spend-authorisation signature published with rk adds no information about ak^P : under Assumption 7.3 it is simulated from rk and the message M alone by programming H (*Crypto Guide*, §“Security in the random oracle model and the forking lemma”, Lemma “Signature simulation”). The simulator draws u uniform on the 64-byte strings and S uniform on $\mathbb{F}_{p_{Vesta}}$, sets $c := \text{ToScalar}(u)$ and $R := [S] G^{\text{SpendAuth}} - [c] rk$, programs H at $\underline{R} \parallel rk \parallel M$ to u , and outputs $\underline{R} \parallel \text{I2LEOSP}_{256}(S)$. Beyond the abort probability of the lemma, a real signature differs from the simulated one only through its nonce: for a uniform nonce the two are identically distributed, since $S = r + c \cdot rsk$ is then uniform given u and $R = [S] G^{\text{SpendAuth}} - [c] rk$. The difference is therefore at most the bound (5) per signature, with q_b there the number of all other queries to H in the experiment. The remaining public fields of an Action are treated in Theorem 12.12, in “Privacy” (§12.4).

7.3 RedPallas unforgeability

Proposition 7.12 (Unforgeability under re-randomisation). *Assume that discrete logarithms on Pallas are hard (Assumption 2.22) and that H is a random oracle (Assumption 7.3). Let x be uniform on $\mathbb{F}_{p_{Vesta}}$ and $ak^P := [x] G^{\text{SpendAuth}}$. An adversary $\mathcal{A}$ receives ak^P , makes at most q_b queries to H and at most q_s queries to a signing oracle that answers a pair (α, M) of its choice, $\alpha \in \mathbb{F}_{p_{Vesta}}$ and M a byte string, with the spend-authorisation signature under $x + \alpha$ on M , whose validating key is $ak^P + [\alpha] G^{\text{SpendAuth}}$; it outputs a triple (rk^*, M^*, σ^*) with rk^* a point. It forges if σ^* is valid under rk^* on M^* and (rk^*, M^*, σ^*) is not the triple of validating key, message and signature of an oracle answer. Let ϵ be its forging probability, $Q := q_b + q_s + 1$ and*

$$\eta := Q \delta_{512} + q_s(q_b + 2q_s) \cdot 2^{-640}.$$

- (i) Extraction. *There is an algorithm $\mathcal{D}$ that receives ak^P but not x , runs $\mathcal{A}$ twice on shared coins, answers its signing queries without x , and outputs a scalar rsk^* with $rk^* = [rsk^*] G^{\text{SpendAuth}}$, for the rk^* of the first run, with probability at least*

$$\frac{(\epsilon - \eta)^2}{Q} - \frac{Q}{p_{Vesta}} - \frac{q_s(q_b + q_s)}{p_{Vesta}} \quad (\epsilon \geq \eta),$$

the bound of the Crypto Guide’s Theorem “EUFCMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”) with ϵ replaced by $\epsilon - \eta$ and the group order by p_{Vesta} .

- (ii) Unforgeability under re-randomisation. *If $\mathcal{A}$ also outputs $\alpha^* \in \mathbb{F}_{p_{Vesta}}$ with $rk^* = ak^P + [\alpha^*] G^{\text{SpendAuth}}$, of its own choice and possibly dependent on ak^P , then $\mathcal{D}$ outputs $x = rsk^* - \alpha^*$, the discrete logarithm of ak^P , with the same probability. By Assumption 2.22, ϵ is then negligible for every efficient $\mathcal{A}$.*

With every randomiser zero, part (ii) is the existential unforgeability of RedPallas under chosen-message attack (Crypto Guide, §“Syntax and security goal”, Definition “Existential unforgeability under chosen-message attack”) in its strong form (Definition “Strong unforgeability” there), since freshness is required of the triple rather than of the message. Part (ii) implies the SURK-CMA requirement of the protocol specification, §“Signature with Re-Randomizable Keys”, whose forgery is a message–signature pair not returned by the oracle, hence a triple not returned by it.

Proof. The argument is route (ii) of the *Crypto Guide*’s Remark “Unforgeability under re-randomised keys, and the key-prefixing subtlety” (§“Key re-randomisation and unlinkability”), with the bound of its Theorem “EUF-CMA security of Schnorr signatures in the ROM” (§“Security in the random oracle model and the forking lemma”), which applies to RedPallas with the generator replaced by the base $G^{\text{SpendAuth}}$ (Construction “RedDSA”). The Orchard steps are made in place; write $G := G^{\text{SpendAuth}}$.

(1) *Idealisation.* First, the nonce r_i of each oracle answer is replaced by a uniform scalar. By the lemma on the distance of GenRandom from uniform, with $w = \text{rk}_i \parallel M_i$ and the rest of the experiment as observer, each of the at most q_s replacements costs at most $\delta_{512} + (q_b + 2q_s) \cdot 2^{-640}$: apart from this nonce query, the experiment queries H at most q_b times for $\mathcal{A}$, $q_s - 1$ times for other nonces, q_s times for challenges and once for validating the forgery. Second, the answers of H to the queries of $\mathcal{A}$ and to the validation query are replaced by uniform preimages, under ToScalar, of uniform scalars, so that their values under $H^\oplus$ are uniform on $\mathbb{F}_{p_{\text{Vesta}}}$. Each of these at most $q_b + 1$ replacements costs at most δ_{512} , because in both distributions an answer, conditioned on its reduction, is uniform on the preimages of that reduction. The forging probability thus drops by at most η (*Math Guide*, §“Statistical distance”, Theorem “Properties of statistical distance”, part (3)).

(2) *Embedding.* The reduction receives a discrete-logarithm challenge $Y = [x] G$ with x uniform and gives $\mathcal{A}$ the key $\text{ak}^P := Y$.

(3) *Signing.* It answers the i -th query (α_i, M_i) without x , under $\text{rk}_i := \text{ak}^P + [\alpha_i] G$, by the simulator of the *Crypto Guide*’s Lemma “Signature simulation” (§“Security in the random oracle model and the forking lemma”): it draws u_i uniform on the 64-byte strings and S_i uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, sets $c_i := \text{ToScalar}(u_i)$ and $R_i := [S_i] G - [c_i] \text{rk}_i$, programs H at $R_i \parallel \text{rk}_i \parallel M_i$ to u_i , aborting if that entry is already defined, and returns $\sigma_i := R_i \parallel \text{I2LEOSP}_{256}(S_i)$. After step (1) a real answer has a uniform nonce r_i and, at a fresh input, the challenge $c_i = \text{ToScalar}(u_i)$ for a uniform u_i ; then $S_i = r_i + c_i(x + \alpha_i)$ is uniform given u_i , and $R_i = [S_i] G - [c_i] \text{rk}_i$, which is the simulated distribution (*Crypto Guide*, §“The Schnorr identification protocol”, Theorem “Special honest-verifier zero knowledge”). The point R_i is uniform, so the reduction aborts with probability at most $q_s(q_b + q_s)/p_{\text{Vesta}}$, as in the lemma. Key prefixing places rk_i in every programmed input: entries programmed under distinct keys have distinct inputs, and the simulation needs no relation among the rk_i .

(4) *Critical query.* Let $(\text{rk}^*, M^*, \sigma^*)$ be a forgery, with $\sigma^* = \bar{R}^* \parallel \bar{S}^*$ decoding to $R^* \neq \perp$ and $S^* < p_{\text{Vesta}}$. If $\text{rk}^* = \mathcal{O}$, the scalar $\text{rsk}^* := 0$ satisfies (i), and in (ii) $x = -\alpha^*$; the reduction outputs these. Let $\text{rk}^* \neq \mathcal{O}$. The validation input $\bar{R}^* \parallel \text{rk}^* \parallel M^*$ is not a programmed entry. Its first 32 bytes, its next 32 bytes and its remainder would otherwise give $\bar{R}^* = R_i$, $\text{rk}^* = \text{rk}_i$ and $M^* = M_i$ for some i , the second by injectivity of $P \mapsto \underline{P}$, and the challenge would be c_i . Then $R^* = R_i$,

and (4) gives $[S^*] G = R_i + [c_i] \text{rk}_i = [S_i] G$. Since G generates the group, of prime order p_{Vesta} , and $S^*, S_i < p_{\text{Vesta}}$, this forces $S^* = S_i$, so $\sigma^* = \sigma_i$ and the triple is an oracle answer. The input was queried by $\mathcal{A}$, except with probability at most $1/p_{\text{Vesta}}$: otherwise its value under $H^\circledast$ is a uniform scalar drawn at validation, and, since $\text{rk}^* \neq \mathcal{O}$ generates the group, at most one scalar satisfies (4). This guessing term is part of the term Q/p_{Vesta} of the cited theorem.

(5) *Fork*. The reduction wraps $\mathcal{A}$ as the algorithm of the *Crypto Guide*'s Theorem "General forking lemma" (§"Security in the random oracle model and the forking lemma"), with challenge set $\mathbb{F}_{p_{\text{Vesta}}}$, Q prepared challenges $c_1, \dots, c_Q$, and coins that include those of $\mathcal{A}$ and of the simulator. The wrapper answers the j -th distinct unprogrammed query to H , of $\mathcal{A}$ or of validation, with a uniform preimage of c_j under ToScalar , and returns the index of the critical query with $\mathcal{A}$'s output. The two runs of the forking algorithm share the coins and the challenges before that index, so $\mathcal{A}$ makes the same critical query in both, with the same $\bar{R}^*$, the same rk^* , since key prefixing makes rk^* part of the critical input, and the same M^* . In (ii) they also share α^* , which rk^* determines because G generates the group. The runs receive distinct challenges $c \neq c'$ at the critical index and return responses S^* and S'^* .

(6) *Extraction*. The transcripts (R^*, c, S^*) and (R^*, c', S'^*) are accepting for rk^* with $c \neq c'$, so the *Crypto Guide*'s Theorem "2-special soundness" (§"The Schnorr identification protocol") gives

$$\text{rsk}^* = (S^* - S'^*)(c - c')^{-1},$$

the discrete logarithm of rk^* , which is (i). For (ii) the reduction outputs $\text{rsk}^* - \alpha^* = x$. The probability bound follows as in the cited theorem, from the forking bound

$$\text{frk} \geq \text{acc} \left(\frac{\text{acc}}{Q} - \frac{1}{p_{\text{Vesta}}} \right),$$

with acc at least $\epsilon - \eta$ less the abort and guessing terms of steps (3) and (4). $\square$

Remark 7.13 (Uses of Proposition 7.12). Proposition 7.12 holds for every message: it places no condition on M^* beyond the freshness of the triple. Theorem 12.9 applies part (i), and Proposition 11.10 applies part (ii) to a signature on a new digest.

8 Value commitments and the binding signature

This section constructs the net value commitment of an Action, defines the transparent transaction value pool, the balancing value of each pool and the fee, and constructs the binding signature. Proposition 8.10 keeps apart an algebraic identity, under which the binding validating key is a known multiple of the randomness base exactly when a pool balances modulo p_{Vesta} , and a knowledge statement, under which a valid binding signature forces balance modulo p_{Vesta} .

8.1 Value commitments

Remark 8.1 (Requirement R4). Requirement R4 of "Requirements on a shielded payment" (§1.4) is met through the additive homomorphism of the Pedersen commitment (*Crypto Guide*, §"The Pedersen commitment", Theorem "Additive homomorphism"). Each Action publishes a commitment to its net value; the group sum of a bundle's commitments commits to the sum of

the net values under the sum of the trapdoors, and a verifier compares that sum with the bundle's public balancing value (§8.2) without learning any single value.

Definition 8.2 (Net value commitment). Let an Action consume a note of value v^{old} and create a note of value v^{new} , both in $\{0, \dots, 2^{64} - 1\}$, the value type of Definition “Note” (§4.1); a dummy side has value 0 (§4.4). The *net value* of the Action is the integer

$$v^{\text{net}} := v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}.$$

The *value base* and the *randomness base* are

$$\begin{aligned} V^{\text{Orchard}} &:= \text{GroupHash}(\text{z.cash:Orchard-cv}, v), \\ R^{\text{Orchard}} &:= \text{GroupHash}(\text{z.cash:Orchard-cv}, r), \end{aligned}$$

the value-commitment rows of Table 2. For a trapdoor $\text{rcv} \in \mathbb{F}_{p_{\text{Vesta}}}$, the *net value commitment* of the Action is the point

$$\text{cv}^{\text{net}} := \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{net}}) := [v^{\text{net}} \bmod p_{\text{Vesta}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$$

(protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”).

Both bases have order p_{Vesta} : the protocol specification types them as points of order p_{Vesta} (§“Balance and Binding Signature (Orchard)”), a set that excludes $\mathcal{O}$ (§“Pallas and Vesta”). The specification defines $\text{ValueCommit}^{\text{Orchard}}$ on the signed integers

$$\left\{ -\frac{p_{\text{Vesta}} - 1}{2}, \dots, \frac{p_{\text{Vesta}} - 1}{2} \right\}$$

(§“Commitment”), and a signed integer enters the scalar multiplication as its residue modulo p_{Vesta} (§2.1). Reduction modulo p_{Vesta} is injective on this domain, since two of its elements differ by at most $p_{\text{Vesta}} - 1$. The domain contains every net value, and every signed 64-bit integer, because $p_{\text{Vesta}} > 2^{254}$ (§2.1) gives $(p_{\text{Vesta}} - 1)/2 \geq 2^{253}$.

Construction 8.3 (Value-commitment trapdoor). For each Action the author of the transaction draws rcv uniformly from $\mathbb{F}_{p_{\text{Vesta}}}$, by $\text{ValueCommit}^{\text{Orchard}}.\text{GenTrapdoor}$, independently of every other value (protocol specification, §“Sending Notes (Orchard)” and §“Homomorphic Pedersen commitments (Sapling and Orchard)”). Unlike rcm , ψ and esk , which are derived from the note seed rseed (§4.3), the trapdoor rcv is derived from no note, and no note field, note plaintext (§10.1) or transaction field (§11.2) contains it. The author retains every rcv of the bundle; together they determine the binding signing key (§8.3).

The specification types the trapdoor as an integer in $\{0, \dots, 2^{255} - 1\}$ (§“Commitment”). As for rcm (§4.1), this volume types it in $\mathbb{F}_{p_{\text{Vesta}}}$, the set that GenTrapdoor samples, since rcv acts only as the scalar of $[\text{rcv}]$.

Lemma 8.4 (Hiding and binding of net value commitments). *With $G := V^{\text{Orchard}}$ and $H := R^{\text{Orchard}}$, the map $\text{ValueCommit}^{\text{Orchard}}$ is the Pedersen commitment of the Crypto Guide (§“The Pedersen commitment”, Construction “Pedersen commitment”), with messages the residues modulo p_{Vesta} .*

- (i) *Perfect hiding: for rcv uniform on $\mathbb{F}_{p_{\text{Vesta}}}$ and independent of v^{net} , the commitment cv^{net} is uniform on $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, whatever v^{net} is.*
- (ii) *Computational binding: under Assumptions 2.22 and 2.8, no efficient algorithm outputs, except with negligible probability, pairs (v, rcv) and (v', rcv') of an integer and a trapdoor with $v \not\equiv v' \pmod{p_{\text{Vesta}}}$ and $[v] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}} = [v'] V^{\text{Orchard}} + [\text{rcv}'] R^{\text{Orchard}}$. On the domain of $\text{ValueCommit}^{\text{Orchard}}$ distinct integers are distinct residues, so the same bound excludes openings of one point to two distinct net values.*

Proof. (i) This is the *Crypto Guide*’s Theorem “Perfect hiding” (§“The Pedersen commitment”). Since $R^{\text{Orchard}} \neq \mathcal{O}$ and the group has prime order, $\text{rcv} \mapsto [\text{rcv}] R^{\text{Orchard}}$ is a bijection from $\mathbb{F}_{p_{\text{Vesta}}}$ onto $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; the uniform point $[\text{rcv}] R^{\text{Orchard}}$ stays uniform under translation by the fixed point $[v^{\text{net}} \bmod p_{\text{Vesta}}] V^{\text{Orchard}}$. (ii) The double-opening algebra of the *Crypto Guide*’s Theorem “Computational binding under DLog” (same section) gives

$$[v - v'] V^{\text{Orchard}} - [\text{rcv}' - \text{rcv}] R^{\text{Orchard}} = \mathcal{O},$$

a relation with non-zero coefficient $v - v'$ on V^{Orchard} . The two bases are values of GroupHash at distinct inputs; under Assumption 2.8 they replace the independent generators that the cited construction samples, and the lemma on relations among fixed generators (§2.4) excludes the relation except with negligible probability. The specification requires computational binding and states that the scheme is unconditionally hiding (protocol specification, §“Homomorphic Pedersen commitments (Sapling and Orchard)”). $\square$

8.2 The balancing value

Definition 8.5 (Transparent transaction value pool). *Transparent inputs and transparent outputs* are the cleartext value transfers of a transaction outside every shielded pool, with public amounts. Each transaction has a *transparent transaction value pool*: its transparent inputs add their values to it, and its transparent outputs remove theirs. The balancing value of each shielded pool, defined next for the two pools of the Orchard protocol, is also added to it (protocol specification, §“Transactions and Treestates”).

Definition 8.6 (Balancing value). For each pool of the Orchard protocol, the Orchard pool or the Ironwood pool, the *balancing value* $v_{\text{pool}}^{\text{balance}}$ of a transaction is its declared net value, in zatoshi, of the notes of the pool that it consumes minus those that it creates: a signed integer in $\{-2^{63}, \dots, 2^{63} - 1\}$, encoded in cleartext as the two’s-complement 64-bit field `valueBalanceOrchard` or `valueBalanceIronwood`, of type `int64`. It is 0 when the transaction has no

Action of the pool, and the field is then absent. A positive $v_{pool}^{balance}$ takes net value out of the pool and adds it to the transparent transaction value pool; a negative one takes value from the transparent transaction value pool into the pool (protocol specification, §“Balance and Binding Signature (Orchard)”); ZIP 229, “Transaction Format”).

In a correctly constructed transaction, $v_{pool}^{balance}$ equals the sum of the net values v^{net} of the pool’s Actions. A verifier cannot check this equation directly, since each v^{net} is hidden in its cv^{net} ; the binding signature (§8.3) enforces it.

The protocol specification, §“Transactions and Treestates”, states the consensus rule: the value remaining in the transparent transaction value pool must be non-negative. For a transaction other than a coinbase transaction (Definition “Coinbase transaction”, §11.2) the remaining value is the *transaction fee* (ibid.); no transaction field encodes it (ZIP 229, “Transaction Format” and “Non-requirements”). The pool of a coinbase transaction also receives, as implicit inputs, the block subsidy and the fees of the block’s other transactions, and its outputs must consume all of the value in it (protocol specification, §“Transactions and Treestates” and §“Transaction Consensus Rules”). ZIP 317, “Fee calculation”, specifies the *conventional fee*, which wallets SHOULD pay and which is not a consensus requirement:

$$\begin{aligned} marginal_fee &\cdot \max(grace_actions, logical_actions) \text{ zatoshi,} \\ marginal_fee &= 5000, \quad grace_actions = 2, \end{aligned}$$

where *logical_actions* is a sum of per-component contributions, in which each Ironwood-pool Action contributes one (the contribution $nActionsIronwood$). The fee of the worked payment is computed in Example 11.16 (§11.5).

Remark 8.7 (Disclosure by the balancing value). The balancing value is cleartext, so a negative one discloses the amount that enters the pool; the field names no receiving note or address.

8.3 The binding signature

Fix a pool of the Orchard protocol and a transaction whose bundle for that pool has $n \geq 1$ Actions, with net values v_i^{net} , trapdoors rcv_i and net value commitments cv_i^{net} ($i = 1, \dots, n$), and write $v^{balance}$ for the pool’s balancing value.

Definition 8.8 (Binding signing key). The *binding signing key* of the bundle is

$$bsk := rcv_1 + \dots + rcv_n \in \mathbb{F}_{p_{Vesta}}$$

(protocol specification, §“Balance and Binding Signature (Orchard)”). By the additive homomorphism (*Crypto Guide*, §“The Pedersen commitment”, Theorem “Additive homomorphism”),

$$\sum_{i=1}^n cv_i^{net} = \left[\sum_{i=1}^n v_i^{net} \right] V^{Orchard} + [bsk] R^{Orchard}, \quad (6)$$

the integer sum entering as its residue modulo p_{Vesta} .

Definition 8.9 (Binding validating key). The verifier computes, from public fields only, the *binding validating key*

$$\text{bvk} := \left(\sum_{i=1}^n \text{cv}_i^{\text{net}} \right) - [v^{\text{balance}}] V^{\text{Orchard}},$$

where the subtracted point is $\text{ValueCommit}_0^{\text{Orchard}}(v^{\text{balance}})$, the commitment with zero trapdoor, and v^{balance} , an element of the domain of $\text{ValueCommit}^{\text{Orchard}}$, enters as its residue modulo p_{Vesta} . No transaction field encodes bvk (protocol specification, §“Balance and Binding Signature (Orchard)”).

An *opening* of a net value commitment cv_i^{net} is a pair (v'_i, r'_i) of an integer v'_i and a scalar $r'_i \in \mathbb{F}_{p_{\text{Vesta}}}$ with $\text{cv}_i^{\text{net}} = [v'_i \bmod p_{\text{Vesta}}] V^{\text{Orchard}} + [r'_i] R^{\text{Orchard}}$. In the next proposition a *binding signature* under a point X is a signature valid under X in the binding-signature instance of RedPallas, with base R^{Orchard} (§7.1); Construction “Binding signature” below fixes its use.

Proposition 8.10 (Balance and key knowledge). *For the bundle fixed above:*

(i) *Identity. If $\text{cv}_i^{\text{net}} = \text{ValueCommit}_{\text{rcv}_i}^{\text{Orchard}}(v_i^{\text{net}})$ for every i and $\text{bsk} := \sum_i \text{rcv}_i$, then*

$$\text{bvk} - [\text{bsk}] R^{\text{Orchard}} = \left[\sum_{i=1}^n v_i^{\text{net}} - v^{\text{balance}} \right] V^{\text{Orchard}};$$

hence $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$ if and only if $\sum_i v_i^{\text{net}} \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$.

(ii) *Knowledge. Let (v'_i, r'_i) be an opening of cv_i^{net} for every i , and let $b \in \mathbb{F}_{p_{\text{Vesta}}}$ satisfy $\text{bvk} = [b] R^{\text{Orchard}}$. If $D := \sum_i v'_i - v^{\text{balance}}$ is non-zero modulo p_{Vesta} , then*

$$\left(b - \sum_{i=1}^n r'_i \right) D^{-1} \bmod p_{\text{Vesta}}$$

is the discrete logarithm of V^{Orchard} to the base R^{Orchard} , a relation between two values of GroupHash.

Consequently, under Assumptions 2.22, 2.8 and 7.3, an efficient party that outputs the net value commitments and the balancing value of a bundle, openings (v'_i, r'_i) of every cv_i^{net} and a binding signature under the resulting bvk on a message of its choice satisfies $\sum_i v'_i \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, except with negligible probability. The proposition holds for every message.

Proof. (i) Expanding each cv_i^{net} by Definition 8.2 and applying the additive homomorphism gives (6); subtracting $[v^{\text{balance}}] V^{\text{Orchard}}$ gives the identity. Since V^{Orchard} has prime order p_{Vesta} , the point $[c] V^{\text{Orchard}}$ is $\mathcal{O}$ if and only if $c \equiv 0 \pmod{p_{\text{Vesta}}}$.

(ii) By the homomorphism, $\text{bvk} = [D] V^{\text{Orchard}} + [\sum_i r'_i] R^{\text{Orchard}}$. Equating with $[b] R^{\text{Orchard}}$ gives

$$[D] V^{\text{Orchard}} = \left[b - \sum_{i=1}^n r'_i \right] R^{\text{Orchard}},$$

and D is invertible modulo the prime p_{Vesta} . This is Part 2 of the proof of the *Crypto Guide*’s

Theorem “The binding signature is a proof of knowledge of $\log_p B$ ” (§“Binding signatures as a proof of knowledge of a discrete logarithm”), with $P = R^{\text{Orchard}}$ and the balancing value folded into the key as that section describes after the theorem.

Consequence. Let an efficient party $\mathcal{A}$ output, with probability ϵ , a bundle’s net value commitments and balancing value, openings of every commitment, a message and a binding signature on it under the resulting bvk , with D non-zero modulo p_{Vesta} ; this event is efficiently decidable from the output. If $\text{bvk} = \mathcal{O}$, then $b := 0$ satisfies $\text{bvk} = [b] R^{\text{Orchard}}$. Otherwise Part 1 of the proof of the same theorem, special soundness with forking in the random-oracle model for H (Assumption 7.3), rewinds $\mathcal{A}$ to two accepting transcripts with a common first component and distinct challenges and extracts b with $\text{bvk} = [b] R^{\text{Orchard}}$. In this volume the extraction is steps (1) and (4) to (6) of the proof of Proposition 7.12, with no signing queries ($q_i = 0$), the base R^{Orchard} in place of $G^{\text{SpendAuth}}$ and bvk in place of rk^* . Key prefixing places the encoding of bvk in the critical query, so both runs sign under the same bvk , and the openings and the balancing value of the first run satisfy the hypothesis of part (ii) for it. If ϵ is non-negligible, the extraction succeeds with non-negligible probability, and part (ii) then outputs the discrete logarithm of V^{Orchard} to the base R^{Orchard} : a non-trivial relation between values of GroupHash at distinct inputs, which the lemma on relations among fixed generators (§2.4) excludes except with negligible probability under Assumptions 2.8 and 2.22. No argument uses the value of the message, and no concrete bound is claimed. The security argument of the protocol specification, §“Balance and Binding Signature (Orchard)”, is the deployed counterpart. $\square$

The openings are obtained, and the congruence is promoted to an equality of integers, in Lemma 12.3 and Theorem 12.4 (§12.1).

Construction 8.11 (Binding signature). The binding signature scheme $\text{BindingSig}^{\text{Orchard}}$ is the binding-signature instance of RedPallas (§7.1): RedPallas on the base R^{Orchard} , without key re-randomisation; its base differs from the spend-authorisation base $G^{\text{SpendAuth}}$. The author of the transaction computes bsk from the trapdoors it retains and signs with it; the verifier recomputes bvk from the public fields and validates the signature under it (protocol specification, §“Binding Signature (Sapling and Orchard)” and §“Balance and Binding Signature (Orchard)”). The signed message is the signature digest SigHash constructed in “Transaction digests and signatures” (§11.3); which transaction data the signatures fix is stated there, in Proposition 11.10.

For an honestly constructed bundle with $\sum_i v_i^{\text{net}} = v^{\text{balance}}$, Proposition 8.10(i) gives $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, so (bsk, bvk) is a key pair of the instance and the honest signature is accepted (§7.1). Under Assumption 7.3 a valid signature is a proof of knowledge of the discrete logarithm of bvk to the base R^{Orchard} (*Crypto Guide*, §“Binding signatures as a proof of knowledge of a discrete logarithm”, Theorem “The binding signature is a proof of knowledge of $\log_p B$ ”), so, for a party that also outputs openings of every net value commitment, the consequence stated in Proposition 8.10, under Assumptions 2.22, 2.8 and 7.3, makes it certify balance modulo p_{Vesta} without taking any net value as input: it is computed from bsk , the message and fresh randomness. What the public data of a bundle hide is stated in Theorem 12.12 (§12.4).

The construction runs once for each pool of the Orchard protocol. A transaction carries a balancing value and a binding signature, bindingSigOrchard or $\text{bindingSigIronwood}$, for each pool

whose bundle has at least one Action. Each pool's *bvk* sums only that pool's net value commitments and subtracts only that pool's balancing value; both pools use $\text{ValueCommit}^{\text{Orchard}}$ with the bases V^{Orchard} and R^{Orchard} , and $\text{BindingSig}^{\text{Orchard}}$. The consensus rule of the protocol specification, §“Transaction Consensus Rules” (ZIP 229, “Consensus Rules”), is: if a pool's bundle has Actions, its binding signature must be a valid signature of SigHash under that pool's *bvk*. Validation rejects a non-canonical encoding of the signature's point component (§7.1; protocol specification, §“RedDSA, RedJubjub, and RedPallas”).

9 The Action statement and its proof

This section states the relation that is proved for every Action. It fixes the bundle flags that enter the relation as public inputs, states the Action statement over the objects constructed in §2 to §8, pairs each condition with the requirement it serves and with the check outside the proof that completes it, and states the proof system together with the two assumptions on the Action proof that “Security” (§12) uses.

9.1 Bundle flags

Each bundle carries one *flags byte*, present exactly when the bundle has at least one Action: *flagsOrchard* for the Orchard-pool bundle of a version 5 or version 6 transaction, and *flagsIronwood* for the Ironwood-pool bundle of a version 6 transaction (ZIP 229, “Transaction Format”). Bits are numbered from the least significant, bit 0. In both bytes bit 0 is *enableSpends* and bit 1 is *enableOutputs*. The consensus rules of ZIP 229, “Consensus Rules”, fix the remaining bits:

1. in *flagsIronwood*, bit 2 is *enableCrossAddress*, and bits 3 to 7 are reserved and must be 0;
2. in *flagsOrchard*, bits 2 to 7 are reserved and must be 0, in version 5 and version 6 transactions alike.

ZIP 229, “Transaction Format”, lists *enableCrossAddress* at bit 2 of *flagsOrchard* in its format table; this contradicts the consensus rules of the same ZIP, which reserve that bit, and the volume follows the consensus rules. The two texts differ in the role of the bit, not in its value in a valid transaction: ZIP 258, “Consensus rules from NU6.3 activation”, requires every Orchard-pool Action to be created with *enableCrossAddress* = 0.

Definition 9.1 (Flag values and expanded receiver). Let a bundle have flags byte $f = \sum_{i=0}^7 f_i 2^i$ with $f_i \in \{0, 1\}$. Its *flag values*, elements of $\{0, 1\}$, are

$$\begin{aligned} \text{enableSpends} &:= f_0, & \text{enableOutputs} &:= f_1, \\ \text{enableCrossAddress} &:= \begin{cases} f_2 & \text{for an Ironwood-pool bundle,} \\ 0 & \text{for an Orchard-pool bundle,} \end{cases} \end{aligned}$$

and $\text{disableCrossAddress} := 1 - \text{enableCrossAddress}$. For an Orchard-pool bundle bit 2 is reserved and 0, so every Orchard-pool Action has *enableCrossAddress* = 0 by encoding. The *expanded receiver* of a payment address (d, pk_d) is the pair (g_d, pk_d) with $g_d = \text{DiversifyHash}(d)$,

the diversified base of “Diversified addresses” (§3.3); the expanded receiver of a note is that of its address. The flags have the following meaning (protocol specification, §“Action Descriptions”):

- the value `enableSpends` = 1 enables non-zero-valued spends in the bundle’s Actions;
- the value `enableOutputs` = 1 enables non-zero-valued outputs in the bundle’s Actions;
- the value `enableCrossAddress` = 1 enables the expanded receiver of an Action’s created note to differ from that of the note the same Action consumes.

The Action statement takes `disableCrossAddress` as its eighth primary input (same section). No normative text fixes the position of this input in the encoded instance, and none is asserted here.

ZIP 229 calls `enableCrossAddress` = 1 the normal case for Ironwood-pool Actions (Rationale, “`enableCrossAddress` polarity”); the value is not required. No consensus rule requires bit 2 of `flagsIronwood` to be 1. A bundle in which that bit is 0 is admitted, and each of its Actions is then restricted, by condition A10 of Definition 9.2, to create its note at the expanded receiver of the note it consumes.

The flags are properties of the bundle. Encoded once in its flags byte, they are the same for every Action of the bundle (protocol specification, §“Action Descriptions”, note on the encoding of these components once per pool). The verifier reads them from the bundle and supplies `enableSpends`, `enableOutputs` and `disableCrossAddress` as public inputs of each Action; they are not computed inside the proof. They enter the relation only through conditions A8, A9 and A10 of Definition 9.2.

9.2 The Action statement

The Action statement is a relation in the sense of the *Crypto Guide*’s Definition “Language, relation, witness” (§“Languages, relations, and witnesses”), with the primary input as instance and the auxiliary input as witness, each a tuple of the typed objects below, encoded as bit strings by the encodings of §2.1. Every condition is an equation over objects constructed in §2 to §8.

Definition 9.2 (Orchard Action statement). In this definition $\mathcal{E}$ abbreviates the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, with identity $\mathcal{O}$, and $\mathcal{E}^* := \mathcal{E} \setminus \{\mathcal{O}\}$. For $P = (x, y) \in \mathcal{E}^*$ let $x(P) := x$ and $y(P) := y$, and let $x(\mathcal{O}) := y(\mathcal{O}) := 0$, so that $\text{Extract}_P(P) = x(P)$ for every $P \in \mathcal{E}$ (protocol specification, §“Coordinate Extractor for Pallas”). An integer that multiplies a point acts as its residue modulo p_{Vesta} (§2.1).

The relation $\mathcal{R}_{\text{Action}}$ is the set of pairs (pub, aux) of a *primary input*

$$\text{pub} = (rt, cv^{\text{net}}, nf^{\text{old}}, rk, cmx, \text{enableSpends}, \text{enableOutputs}, \text{disableCrossAddress})$$

and an *auxiliary input*

$$\text{aux} = (\text{path}, \text{pos}, g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, rcm^{\text{old}}, cm^{\text{old}}, \alpha, ak^P, nk, r_{ivk}, g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \psi^{\text{new}}, rcm^{\text{new}}, rcv, c)$$

that have the following types and satisfy conditions A1 to A10 below.

- Primary input: $rt, \text{nf}^{\text{old}}, \text{cmx} \in \{0, \dots, p_{\text{Pallas}} - 1\}$, read as elements of $\mathbb{F}_{p_{\text{Pallas}}}$; $\text{cv}^{\text{net}}, \text{rk} \in \mathcal{E}$; and $\text{enableSpend}, \text{enableOutputs}, \text{disableCrossAddress} \in \{0, 1\}$.
- Auxiliary input: $\text{path} = (s_0, \dots, s_{31})$ with every $s_t \in \{0, \dots, p_{\text{Pallas}} - 1\}$; $\text{pos} \in \{0, \dots, 2^{32} - 1\}$; $\text{g}_d^{\text{old}}, \text{pk}_d^{\text{old}}, \text{g}_d^{\text{new}}, \text{pk}_d^{\text{new}}, \text{ak}^{\text{P}} \in \mathcal{E}^*$; $v^{\text{old}}, v^{\text{new}} \in \{0, \dots, 2^{64} - 1\}$; $\rho^{\text{old}}, \psi^{\text{old}}, \psi^{\text{new}}, \text{nk} \in \mathbb{F}_{p_{\text{Pallas}}}$; $\text{cm}^{\text{old}} \in \mathcal{E}$; $\text{rcm}^{\text{old}}, \text{rcm}^{\text{new}}, \text{r}_{\text{ivk}}, \text{rcv}, \alpha \in \{0, \dots, 2^{255} - 1\}$; and $\text{c} = (c_0, \dots, c_{63}) \in \{0, 1\}^{64}$.

Let $\rho^{\text{new}} := \text{nf}^{\text{old}} \in \mathbb{F}_{p_{\text{Pallas}}}$, let $\delta := \text{disableCrossAddress}$, and let

$$\begin{aligned} M^{\text{old}} &:= (\text{g}_d^{\text{old}})^* \parallel (\text{pk}_d^{\text{old}})^* \parallel \text{LE}_{64}(v^{\text{old}}) \parallel \text{LE}_{255}(\rho^{\text{old}}) \parallel \text{LE}_{255}(\psi^{\text{old}}), \\ M^{\text{new}} &:= (\text{g}_d^{\text{new}})^* \parallel (\text{pk}_d^{\text{new}})^* \parallel \text{LE}_{64}(v^{\text{new}}) \parallel \text{LE}_{255}(\rho^{\text{new}}) \parallel \text{LE}_{255}(\psi^{\text{new}}), \end{aligned}$$

the 1086-bit messages of Definition 4.4, with the diversified bases witnessed. The conditions, all equations in $\mathbb{F}_{p_{\text{Pallas}}}$ or in $\mathcal{E}$, are:

A1. Old note commitment integrity: $\text{NoteCommit}_{\text{rcm}^{\text{old}}}(M^{\text{old}}) \in \{\text{cm}^{\text{old}}, \perp\}$.

A2. New note commitment integrity:

$$\text{Extract}_{\text{P}}^{\perp}(\text{NoteCommit}_{\text{rcm}^{\text{new}}}(M^{\text{new}})) \in \{\text{cmx}, \perp\}.$$

A3. Merkle path validity: $v^{\text{old}} \cdot (\text{root} - rt) = 0$ or some evaluation of SinsemillaHash in the fold, the root included, returns $\perp$. Here $\text{root} := a_{32}$ is the result of the fold (2) of Proposition 5.9 with $a_0 := \text{Extract}_{\text{P}}(\text{cm}^{\text{old}})$, siblings $s_0, \dots, s_{31}$ and the bits of pos as direction bits, in which the evaluation of MerkleCRH at level t encodes its left argument x as $\text{LE}_{255}(\tilde{x})$, where $\tilde{x} := x + c_{2t} p_{\text{Pallas}}$ if this integer is below 2^{255} and $\tilde{x} := x$ otherwise, and its right argument likewise with c_{2t+1} .

A4. Value commitment integrity:

$$\text{cv}^{\text{net}} = \text{ValueCommit}_{\text{rcv}}^{\text{Orchard}}(v^{\text{old}} - v^{\text{new}}) = [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}.$$

A5. Nullifier integrity: $\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$, that is,

$$\text{nf}^{\text{old}} = \text{Extract}_{\text{P}}\left(\left[(\text{PRF}_{\text{nk}}^{\text{nfOrchard}}(\rho^{\text{old}}) + \psi^{\text{old}}) \bmod p_{\text{Pallas}}\right] K^{\text{Orchard}} + \text{cm}^{\text{old}}\right).$$

A6. Spend authority: $\text{rk} = \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$.

A7. Diversified address integrity: with

$$\text{ivk} := \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}_{\text{P}}(\text{ak}^{\text{P}}), \text{nk}),$$

$$\text{either } \text{ivk} = \perp \text{ or } \text{pk}_d^{\text{old}} = [\text{ivk}] \text{g}_d^{\text{old}}.$$

A8. Enable spend flag: $v^{\text{old}} \cdot (1 - \text{enableSpend}) = 0$.

A9. Enable output flag: $v^{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$.

A10. Cross-address restriction: the four equations

$$\begin{aligned} \delta \cdot (x(g_d^{\text{old}}) - x(g_d^{\text{new}})) &= 0, & \delta \cdot (y(g_d^{\text{old}}) - y(g_d^{\text{new}})) &= 0, \\ \delta \cdot (x(pk_d^{\text{old}}) - x(pk_d^{\text{new}})) &= 0, & \delta \cdot (y(pk_d^{\text{old}}) - y(pk_d^{\text{new}})) &= 0. \end{aligned}$$

(Protocol specification, §“Action Statement (Orchard)”, for the typed inputs and conditions A1 to A9; §“Action Descriptions” for the eighth primary input and the meaning of `enableCrossAddress` that A10 formalises.)

The anchor rt is the bundle’s anchor (“Anchors”, §5.2). The rule $rk \neq \mathcal{O}$ of “Randomised validating keys” (§7.2) is checked outside the statement. The typing is part of the relation: the spend validating key ak^p , both diversified bases and both transmission keys are non-identity points (protocol specification, §“Action Statement (Orchard)”, note on types). The trapdoors rcm^{old} , rcm^{new} , r_{ivk} and rcv and the randomiser α act only as scalars on points of order p_{Vesta} , and the statement does not require them to be below p_{Vesta} (same note). The created note’s element ρ is not an auxiliary input; condition A2 fixes it. The relation is an NP-relation in the sense of the cited definition: its instances and witnesses have fixed length, and membership is decided by evaluating the typing and the ten conditions.

Points of the primary input enter the proof by their coordinates. The specification’s encoding note fixes the first seven primary inputs as the nine elements

$$rt, x(cv^{\text{net}}), y(cv^{\text{net}}), nf^{\text{old}}, x(rk), y(rk), cmx, \text{enableSpend}, \text{enableOutputs}$$

of $\mathbb{F}_{p_{\text{Pallas}}}$, in this order (protocol specification, §“Action Statement (Orchard)”, note on the encoding of primary inputs); the convention $x(\mathcal{O}) = y(\mathcal{O}) = 0$ encodes $cv^{\text{net}} = \mathcal{O}$. The eighth input, `disableCrossAddress`, is one further element of $\mathbb{F}_{p_{\text{Pallas}}}$ (§“Action Descriptions”); no normative text fixes its position, and the volume asserts none.

Conditions and the checks that complete them. Each condition is paired below with the requirement of “Requirements on a shielded payment” (§1.4) that it serves and, where one exists, with the check outside the proof that completes it; Table 5 collects the pairing and Figure 3 draws it.

Conditions A1 and A2 bind the consumed and the created note to the commitments cm^{old} and cmx under `NoteCommit`, the Sinsemilla commitment with domain `z.cash:Orchard-NoteCommit` (Definition 4.4), whose binding is Proposition 4.5; they serve R1. Condition A2 fixes the created note’s ρ to the nullifier that the same Action publishes for the note it consumes, the chaining rule of “Nullifier chaining” (§6.3). Its consequences for the uniqueness of ρ and for the Faerie Gold attack are proved in “Double-spend resistance” (Corollary 12.7).

Condition A3 serves R2. Since $0 \leq v^{\text{old}} < 2^{64} < p_{\text{Pallas}}$ (§2.1), the value v^{old} is zero in $\mathbb{F}_{p_{\text{Pallas}}}$ only if it is the integer 0, and a field has no zero divisors. Hence, outside its $\perp$ case and with canonical encodings ($c = 0$), A3 holds if and only if $v^{\text{old}} = 0$ or $root = rt$, the specification’s form: either $v^{\text{old}} = 0$, or $(\text{Extract}_p(cm^{\text{old}}), \text{pos}, \text{path})$ is a valid path to rt in the sense of Proposition 5.9, with the layer-tagged hash of “The Merkle hash and the tree” (§5.1) and the paths of depth 32 of “Authentication paths” (§5.3). A consumed note of value zero is therefore exempt from membership.

The exemption lets an Action whose consumed side is a dummy note (“Actions, bundles, and dummy notes”, §4.4) create a real note without consuming a committed one. For $v^{\text{old}} \neq 0$ the condition is membership of $\text{Extract}_p(\text{cm}^{\text{old}})$ under $\mathcal{R}$, up to the two relaxations below; Proposition 5.9 gives its soundness together with the anchor rule of “Anchors”, the check outside the proof that completes A3.

The two relaxations are the circuit’s: it does not check that the encoding of each layer’s input is canonical, and its path check may pass when a hash on the path, the root included, is 0, which occurs only when SinsemillaHash returns $\perp$ (protocol specification, §“Merkle Path Validity”, notes on Orchard; §“Action Statement (Orchard)”, notes on the Merkle path validity check). Remark “ $\perp$ -weakened conditions” below treats the $\perp$ case. The bits c , absent from the specification’s auxiliary input, record the encodings that the circuit witnesses. Since p_{Pallas} is odd and has bit length 255 (§2.1), $2p_{\text{Pallas}} > 2^{255}$, so the integers below 2^{255} congruent to an argument $x \in \{0, \dots, p_{\text{Pallas}} - 1\}$ are x and, when it is below 2^{255} , $x + p_{\text{Pallas}}$: the bits range over every encoding the circuit admits, and $c = 0$ gives the canonical ones. Non-canonical encodings leave the extraction in the proof of Proposition 5.9(i) valid. At the least $t^* \geq 1$ at which the fold agrees with the genuine chain of that proof, the running values at level $t^* - 1$ differ as elements of $\mathbb{F}_{p_{\text{Pallas}}}$, so no representative of the one equals a representative of the other, and the two 520-bit messages hashed there remain distinct with a common prefix. Parts (ii) and (iii) of the proposition therefore hold for the fold of A3.

Condition A4 serves R4. It is Definition 8.2: the integer $v^{\text{old}} - v^{\text{new}} \in \{-2^{64} + 1, \dots, 2^{64} - 1\}$, written as $\text{sign} \cdot \text{magnitude}$ with $\text{magnitude} \in \{0, \dots, 2^{64} - 1\}$ and $\text{sign} \in \{-1, +1\}$, acts as a scalar modulo p_{Vesta} . The specification requires the scalar multiplication to be correct on this whole signed range, which differs from the range of the balancing value (protocol specification, §“Action Statement (Orchard)”, note on the signed range). The binding signature of “The binding signature” (§8.3) is the check outside the proof that completes value conservation.

Condition A5 serves R3. It is Definition 6.3; the multiplier of K^{Orchard} is the integer in $\{0, \dots, p_{\text{Pallas}} - 1\}$ obtained by reducing the sum modulo p_{Pallas} . The nullifier-set rule of “Nullifier sets” (§6.2) is the check outside the proof that completes it.

Conditions A6 and A7 serve R5. Condition A6 is the re-randomisation of “Randomised validating keys” (§7.2), with $G^{\text{SpendAuth}} = \text{GroupHash}(\text{z.cash:Orchard}, \mathbb{G})$. Condition A7 recomputes the incoming viewing key as the $\text{Commit}^{\text{ivk}}$ instance of “Viewing keys” (§3.2), under the domain $\text{z.cash:Orchard-CommitIvk}$, from the field element $\text{Extract}_p(\text{ak}^{\text{P}})$, not from the point; it ties the witnessed key components ak^{P} , nk and r_{ivk} to the expanded receiver $(g_d^{\text{old}}, pk_d^{\text{old}})$ of the consumed note. Outside its $\perp$ case it forces $\text{ivk} \neq 0$, since $pk_d^{\text{old}} \neq \mathcal{O}$, and the receiver determines ivk , an integer below $p_{\text{Pallas}} < p_{\text{Vesta}}$, since g_d^{old} is a non-identity point of a group of prime order p_{Vesta} . The tie is the binding of ivk to (ak, nk) (Proposition “Binding of ivk to (ak, nk) ”, §3.2): under Assumptions 2.22 and 2.8, an efficient algorithm outputs two witnesses that satisfy A7 outside its $\perp$ case for one receiver, with distinct triples $(\text{Extract}_p(\text{ak}^{\text{P}}), \text{nk}, r_{\text{ivk}} \bmod p_{\text{Vesta}})$, only with negligible probability. The tie fixes the point ak^{P} only up to sign (Lemma 2.4); Lemma 12.5 and Theorem 12.9 use it. The spend-authorisation signature under rk , verified outside the proof (§7.2), completes both conditions.

Conditions A8 and A9 have the product form of A3. The values are below $2^{64} < p_{\text{Pallas}}$ and the flags are bits, so A8 holds if and only if $v^{\text{old}} = 0$ or $\text{enableSpend} = 1$, and A9 if and only if $v^{\text{new}} = 0$ or $\text{enableOutput} = 1$, the specification’s forms. They tie the hidden values to the public flags of

“Bundle flags” (§9.1).

Condition A10 also has the product form of A3: each of its equations is a public field element times a difference of witnessed values. The four points lie in $\mathcal{E}^*$, where the affine coordinates determine the point. For $\delta = 1$, A10 therefore holds if and only if $(g_d^{\text{new}}, pk_d^{\text{new}}) = (g_d^{\text{old}}, pk_d^{\text{old}})$, equality of the expanded receivers as points and not only of their x -coordinates; for $\delta = 0$ it is vacuous. Condition A10 formalises the meaning of `enableCrossAddress` stated in the protocol specification, §“Action Descriptions”. Every Orchard-pool Action has $\delta = 1$; an Ironwood-pool Action has $\delta = 1 - \text{enableCrossAddress}$, normally 0 (ZIP 229, Rationale, “`enableCrossAddress` polarity”). Conditions A8 to A10 serve no requirement of their own.

Remark 9.3 (Meaning of A10). Condition A10 constrains each Action separately: when `disableCrossAddress` = 1, the note an Action creates is at the expanded receiver of the note that same Action consumes. The consumed note may be a dummy of value 0, which A3 exempts from membership. Hence a bundle with `disableCrossAddress` = 1 can still create a note at any expanded receiver (g_d, pk_d) for which the transaction creator produces a dummy-spend Action in the same bundle. Such an Action has a witness: let its consumed note be a dummy of value 0 at (g_d, pk_d) and its created note, of any value v , be at the same receiver, and let (ak^p, nk, r_{ivk}) be the key components of that receiver. Condition A3 holds because $v^{\text{old}} = 0$, A8 for either value of `enableSpends`, A10 with $\delta = 1$ because the two receivers agree, and A9 with `enableOutputs` = 1 when $v \neq 0$. Conditions A1, A2, A4 and A5 hold when cm^{old} , cmx , cv^{net} and nf^{old} are computed from the witness by their definitions, and A6 and A7 hold for the key components of the receiver, whose address satisfies $pk_d = [ivk] g_d$ (§3.3). The Action also carries a valid spend-authorisation signature under its rk . By Theorem 12.9, in “Authorisation of spends”, for an honestly generated address with `use_qsk` = false this requires that address’s spend authorising key ask , except with negligible probability. Consensus enforces no more than this per-Action restriction (ZIP 326, “Rationale for key-generation restrictions”).

Remark 9.4 (Effect on the Orchard pool). For the Orchard pool, whose flag encoding forces `disableCrossAddress` = 1, the pool-level effect is the one that ZIP 229, Abstract, states: outputs to the Orchard pool go to an address for which the transaction creator can authorise spends, which discourages economic activity between users within that pool. By the dummy-spend construction of the preceding remark, consensus does not prevent transfers between users within the Orchard pool (ZIP 326, “Rationale for key-generation restrictions”; ZIP 258, “Consensus rules from NU6.3 activation”, for the rule on Orchard-pool Actions).

Remark 9.5 (Status). The expanded receiver (g_d, pk_d) , the meaning of `enableCrossAddress` that A10 formalises, and the eight-input primary input with `disableCrossAddress` = $1 - \text{enableCrossAddress}$ are specified in the protocol specification, §“Action Descriptions”; the flag encoding in ZIP 229; the Orchard-pool rule and the selection of the verifying key by upgrade in ZIP 258. The specification’s §“Action Statement (Orchard)” and the proof-validity rule of §“Action Descriptions” list the first seven primary inputs and conditions A1 to A9 only; Definition 9.2 follows §“Action Descriptions” for the rest. The specification names the conditions; the numbering A1 to A10 is this volume’s. ZIP 2006, to which the specification refers for the restriction, is a reserved number without content and is not cited for content.

Remark 9.6 (Sign of ak^P). The statement does not constrain the sign of the witnessed point ak^P . Condition A7 reads only $\text{Extract}_P(ak^P)$, which is the same for ak^P and $-ak^P$ (Lemma 2.4), so the even- γ normalisation of key generation (“The spending key and the spend-side secrets”, §3.1) is not enforced by the proof (protocol specification, §“Action Statement (Orchard)”, note on the sign of the spend validating key). Theorem 12.9, in “Authorisation of spends”, treats both signs.

Remark 9.7 ($\perp$ -weakened conditions). Conditions A1, A2, A3 and A7 are the specification’s $\perp$ -weakened forms: each holds when the Sinsemilla instance it evaluates, NoteCommit , the hash of MerkleCRH or $\text{Commit}^{\text{ivk}}$, returns $\perp$ on the witnessed input, for A3 on some input of the fold, because the circuit evaluates Sinsemilla with incomplete addition and cannot exclude its exceptional cases (protocol specification, §“Action Statement (Orchard)”, note on $\perp$ outputs and note on the Merkle path validity check; §“Merkle Path Validity”, notes on Orchard). The relation is therefore the disjunction, not the plain conjunction. The weakening costs a negligible term. A witness that satisfies A1 or A2 only through $\perp$ contains a 1086-bit message, M^{old} or M^{new} , on which $\text{SinsemillaHashToPoint}(z.\text{cash}:\text{Orchard-NoteCommit-M}, \cdot)$ returns $\perp$; one that satisfies A3 only through $\perp$ contains a 520-bit message on which $\text{SinsemillaHash}(z.\text{cash}:\text{Orchard-MerkleCRH}, \cdot)$ returns $\perp$; one that satisfies A7 only through $\perp$ contains the 510-bit message $\text{LE}_{255}(\text{Extract}_P(ak^P)) \parallel \text{LE}_{255}(nk)$ with the same property for the domain $z.\text{cash}:\text{Orchard-CommitIvk-M}$ (§2.4). An efficient algorithm that runs an adversary together with the extractor of Assumption 9.11 and outputs such a message succeeds with negligible probability by Proposition 2.24(iii), which turns the message into a non-trivial discrete-logarithm relation among the GroupHash generators, under Assumptions 2.22 and 2.8. For the witnesses extracted from accepted proofs, A1, A2, A3 and A7 therefore hold in their unweakened forms, without the $\perp$ case, except with negligible probability.

Remark 9.8 (What the statement does not check).

- (i) No condition involves ask . The point $ak^P = [\text{ask}] G^{\text{SpendAuth}}$ is computed at key generation, outside the statement, and ask is not a witness. Conditions A6 and A7 relate rk and the address to the witnessed ak^P , and the spend-authorisation signature under rk supplies knowledge of the signing key; Theorem 12.9 combines the two.
- (ii) The trapdoors rcm^{old} and rcm^{new} and the elements ψ^{old} and ψ^{new} are free witnesses: no condition checks that rcm and ψ were derived from rseed as in “The note seed” (§4.3). The sender computes that derivation; the recipient checks it, in “Trial decryption and note acceptance” (§10.4), and consensus checks it only for coinbase outputs (“Chain state and pool rules”, §11.4). Hence, in the deployed protocol, the hashed rcm adds no binding: against an adversary able to compute discrete logarithms among the bases of NoteCommit , NoteCommit is not binding, and such an adversary can open one commitment to two distinct notes (ZIP 2005, Rationale, “Attacks against binding of note commitments”). The volume claims nothing further in that setting.
- (iii) The statement reads none of the Action’s note-encryption fields, which “Note encryption” (§10) constructs; the specification records that the absence of a check on the ephemeral key is intentional (§“Action Statement (Orchard)”, note).

Table 5. The requirement map: for each requirement of §1.4, the conditions of Definition 9.2, the checks outside the proof and the results that discharge its soundness part, and the results that discharge its hiding part. It is the only requirement map of the volume.

Requirement	Soundness	Hiding
R1, hidden representation of value	A1 and A2 bind the consumed and the created note to their commitments (Proposition 4.5)	commitment hiding (Propositions 4.5 and 4.8); value-commitment hiding (§8.1); indistinguishability of real and dummy Actions (Theorem 12.12)
R2, membership without identification	A3 with the anchor rule of “Anchors” (Proposition 5.9)	zero knowledge of the proof (Assumption 9.14) with a locally computed authentication path (Lemma 5.15), composed in Theorem 12.12
R3, no second consumption	A5 with the nullifier-set rule (Theorem 12.6)	nullifier unlinkability (Proposition 6.10)
R4, conservation	A4 with the binding signature (Theorem 12.4)	value-commitment hiding (Theorem 12.12)
R5, spend authority	A6 and A7 with the spend-authorisation signature (Theorem 12.9) and bundle binding (Proposition 11.10)	none: R5 has a soundness part only
Flags	A8 to A10 serve no requirement of their own: they tie the hidden values and receivers to the public flags of §9.1	—

Remark 9.9 (Key source). The statement takes the key components (ak^P, nk, r_{ivk}) as witnesses and checks them only through A5, A6 and A7. It therefore applies equally to key components whose ak^P and r_{ivk} come from the alternative source of ZIP 2005 (“Changes to the Protocol Specification”, §4.2.3 “Orchard Key Components”) cited in “The spending key and the spend-side secrets” (§3.1). The key-derived results of the volume, namely Proposition 3.18, Assumption 3.13, Propositions 3.17, 6.10 and 10.6, and the hypotheses on honest key generation of Theorems 12.9 and 12.12, remain restricted to keys with `use_qsk = false`.

Table 5 is the requirement map of the volume. For each requirement of “Requirements on a shielded payment” (§1.4) it lists the soundness content, with the conditions, the checks outside the proof and the results that discharge it, and the hiding content, with the results that discharge it.

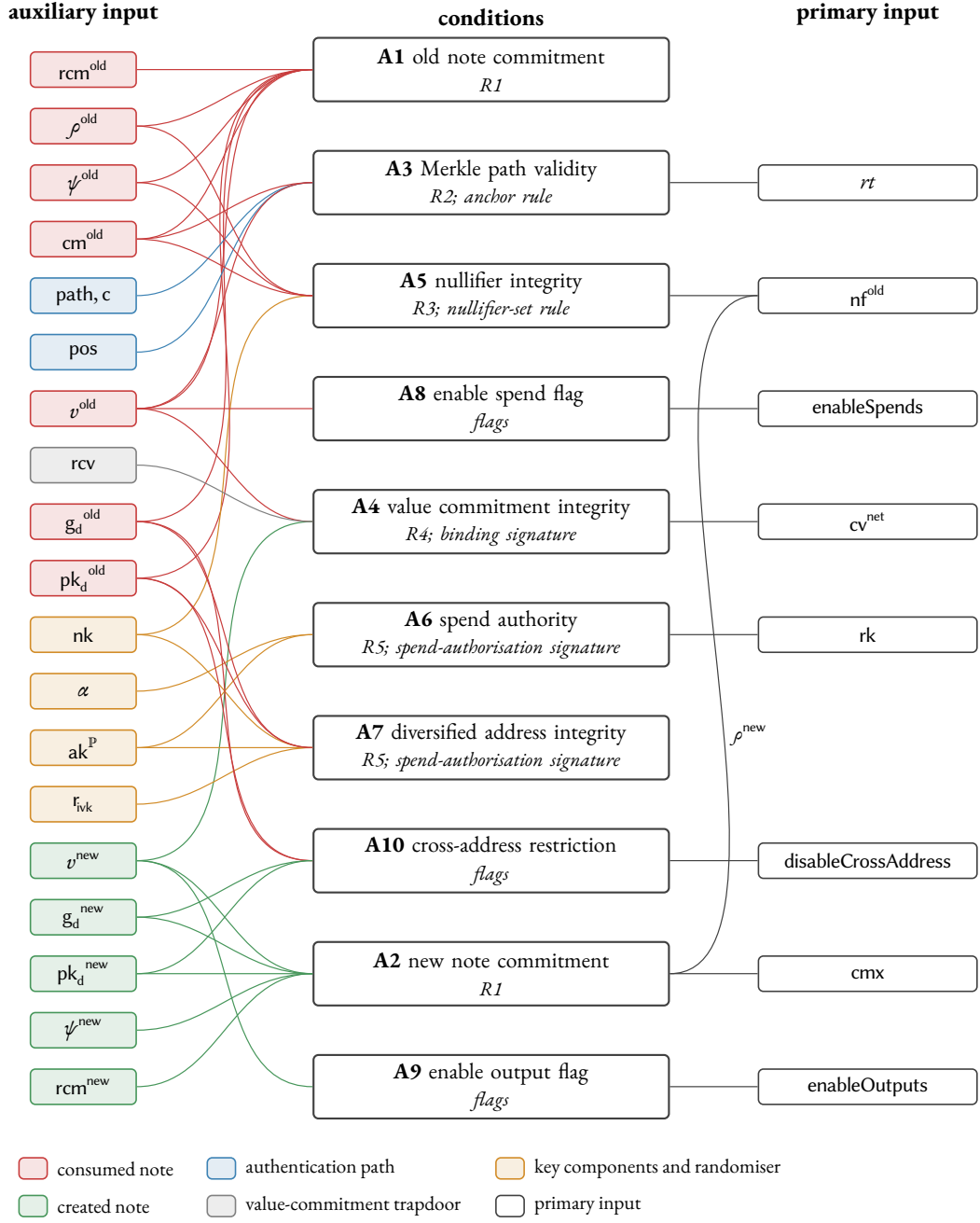


Figure 3. The Action statement of Definition 9.2. The auxiliary input is on the left, coloured by group as in the legend; the primary input is on the right; conditions A1 to A10 are in the centre, ordered by the side of the Action they constrain rather than by number. Each condition has an edge to every variable it reads; the edge from nf^{old} to A2 carries $\rho^{new} := nf^{old}$. Each condition is tagged with the requirement of §1.4 that it serves, or with “flags” for A8 to A10, and, where one exists, with the check outside the proof that completes it: the anchor rule for A3, the nullifier-set rule for A5, the binding signature for A4, and the spend-authorisation signature for A6 and A7.

9.3 The Action circuit and the Halo 2 proof

The Action statement of Definition 9.2 is a relation and prescribes no computation. The *post-NU6.3 Action circuit* is one fixed PLONKish circuit (*Halo 2 Guide*, §“PLONKish arithmetisation”) whose constraints are designed to encode the typing and conditions A1 to A10; that every satisfying assignment yields a witness of $\mathcal{R}_{\text{Action}}$ is part of Assumption 9.11 below. The same circuit serves every Action. Its instance is the encoding of the eight primary inputs of Definition 9.2, bound to the proof as in the *Halo 2 Guide*, §“Instance columns and binding the public statement”; this instance is the only Orchard-specific part of the proof system constructed in this volume. Proofs are Halo 2 proofs (*Halo 2 Guide*, §“The Halo 2 proof system” and §“The complete Halo 2 protocol”; protocol specification, §“Zero-Knowledge Proving System”; ZIP 229, Abstract): a PLONKish polynomial interactive oracle proof compiled with the inner-product polynomial commitment and made non-interactive by the Fiat–Shamir transform, with no trusted setup.

The circuit’s native field is $\mathbb{F}_{p_{\text{Pallas}}}$, the base field of the Pallas group, so the Pallas point operations of A1 to A10 are arithmetic in the native field. The proof’s polynomial commitments lie in the Vesta group G_{Vesta} , the group of points of the curve $y^2 = x^3 + 5$ over $\mathbb{F}_{p_{\text{Vesta}}}$, which has prime order p_{Pallas} (§2.1; *Math Guide*, §“Base fields, scalar fields, and the Pasta cycle”); the committed polynomials therefore have coefficients in $\mathbb{F}_{p_{\text{Pallas}}}$. The Pallas group carries the protocol’s keys, commitments, signatures and note encryption; the Vesta group carries only the proof’s commitments.

The circuit has 2^{11} rows and a constraint-degree bound of 9 across its gates and its permutation and lookup identities (*Halo 2 Guide*, §“From statement to circuit: arithmetisation in practice”). Both are cited constants; in this volume they enter only the Schwartz–Zippel term of the first supporting analysis below.

Every Action of either pool, whatever it consumes or creates, dummy sides included, is an instance of the same relation and circuit. The proofs of a bundle’s Actions are aggregated into one proof, encoded once per bundle, which the verifier checks against the primary inputs of all the bundle’s Actions and the verifying key; each Action carries its own spend-authorisation signature (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”, note on proof aggregation; ZIP 229, “Transaction Format”). The verification procedure is stated in “Verification of a transaction” (§11.6).

Both pools use the post-NU6.3 circuit and its proving and verifying keys; the pools are distinguished by their note commitment trees, nullifier sets, value pool balances and component position in the transaction, not by circuits (ZIP 229, Abstract, and Rationale, “Reuse of the Orchard protocol with minimal changes”). Consensus selects the Action verifying key by network upgrade (protocol specification, §“Action Descriptions”, consensus rules). The key in force for every Action, in either pool and in version 5 and version 6 transactions alike, is the post-NU6.3 verifying key, named by upgrade and not by pool (ZIP 258, “Consensus rules from NU6.3 activation”).

Assumption 9.10 (Discrete logarithms on Vesta). For every classical probabilistic polynomial-time algorithm, given a generator G of G_{Vesta} and $[s]G$ for s uniform in $\mathbb{F}_{p_{\text{Pallas}}}$, the probability of outputting s is negligible (*Crypto Guide*, §“Standing assumptions”, Remark “Standing assumptions for the sequel”, item 1, for the Vesta curve). Concretely, the best known classical attack costs about 2^{126} group operations (*Crypto Guide*, §“Instantiation on elliptic curves; the Pasta curves”, Proposition “The Pasta curves against the criteria”); the level is cited, not recom-

puted. The volume uses this assumption only in the extraction analysis of the inner-product commitment that supports Assumption 9.11 (the second supporting analysis below).

Assumption 9.11 (Knowledge soundness of the Action proof). Let the hash of the Fiat–Shamir transform of the proof system be modelled as a random oracle. For every efficient adversary $\mathcal{A}$ that outputs primary inputs $\text{pub}_1, \dots, \text{pub}_n$ and an aggregate proof π , there is an efficient extractor $\mathcal{E}$, with access to $\mathcal{A}$ and to its random-oracle queries, that outputs auxiliary inputs $\text{aux}_1, \dots, \text{aux}_n$ such that the probability of the event

$$\begin{aligned} &\pi \text{ is accepted for } (\text{pub}_1, \dots, \text{pub}_n) \text{ under the post-NU6.3 verifying key} \\ &\text{and } (\text{pub}_i, \text{aux}_i) \notin \mathcal{R}_{\text{Action}} \text{ for some } i \end{aligned}$$

is negligible. This is the knowledge-soundness clause of the *Crypto Guide*’s Definition “SNARK” (§“SNARKs: succinct non-interactive arguments of knowledge”) for the relation $\mathcal{R}_{\text{Action}}$ of Definition 9.2. It includes that every satisfying assignment of the post-NU6.3 circuit yields a witness of $\mathcal{R}_{\text{Action}}$, for which the specification requires canonical decompositions of the scalar of A5 and of the scalar ivk in $[\text{ivk}] \text{g}_d^{\text{old}}$ (protocol specification, §“Action Statement (Orchard)”, note on canonical scalar decompositions).

Assumption 9.11 is an assumption, not a proved result. Its status is designed but unspecified: bounds exist for the analysed protocol, and no theorem carries them to the deployed Fiat–Shamir-compiled protocol (*Halo 2 Guide*, §“The algebraic group model”). The volume assumes the property, proves no bound and asserts no loss factor. The two supporting analyses that follow are cited, not re-proved, and not combined into a bound.

Remark 9.12 (Supporting analysis (i): a Schwartz–Zippel term). A non-zero gap polynomial D over $\mathbb{F}_{p_{\text{Pallas}}}$, fixed by the prover’s commitments before the challenge is drawn, vanishes at a uniform challenge with probability at most $\deg D / p_{\text{Pallas}}$ (*Halo 2 Guide*, §“Why checking one random point is a proof”). At the Orchard parameters, 2^{11} rows and constraint-degree bound 9, the constraint expression has degree below $9 \cdot 2^{11}$, and so does the committed quotient, of 8 chunks of degree below 2^{11} , times the vanishing polynomial of degree 2^{11} (*Halo 2 Guide*, §“The complete Halo 2 protocol”, Phase 4). Hence $\deg D < 9 \cdot 2^{11} = 18432$, and the term is below $18432 / p_{\text{Pallas}} < 2^{-239}$. The term bounds one check at one challenge and is not a security level of the proof: the complete protocol has further randomised checks, and under the Fiat–Shamir transform an adversary may retry challenges, one hash query per retry (*Halo 2 Guide*, §“Removing the verifier: the Fiat–Shamir transform”).

Remark 9.13 (Supporting analysis (ii): extraction for the inner-product commitment). The inner-product polynomial commitment, whose commitments lie in $\mathbb{G}_{\text{Vesta}}$, has an interactive opening protocol that is knowledge-sound under the discrete-logarithm assumption in that group, Assumption 9.10, with an expected-polynomial-time tree extractor (*Halo 2 Guide*, §“The inner-product polynomial commitment”, Theorem “Knowledge soundness of the opening protocol”). The Fiat–Shamir-compiled argument inherits the interactive extraction bound only up to a loss in the adversary’s random-oracle query count (§“From the interactive extractor to the non-interactive argument”), and the tighter analyses in the algebraic group model are proved for

the analysed protocol, not for the deployed transcript (§“The algebraic group model”). No loss factor is asserted.

Assumption 9.14 (Zero knowledge of the Action proof). Let the hash of the Fiat–Shamir transform be modelled as a programmable random oracle. There is an efficient simulator that, given only the primary inputs $\text{pub}_1, \dots, \text{pub}_n$ of a bundle and programming the random oracle, outputs an aggregate proof whose distribution is statistically indistinguishable from that of an honestly generated proof for any $\text{aux}_1, \dots, \text{aux}_n$ with $(\text{pub}_i, \text{aux}_i) \in \mathcal{R}_{\text{Action}}$ for every i : every distinguisher that makes polynomially many random-oracle queries, whatever its running time, has negligible advantage. This is zero knowledge of statistical grade in the non-interactive, random-oracle form of the *Crypto Guide*’s Definition “Zero knowledge” (§“The simulation paradigm and zero knowledge”).

Assumption 9.14 is an assumption, not a proved result. The *Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”, records honest-verifier zero knowledge of the interactive protocol and zero knowledge of its Fiat–Shamir compilation in the programmable random-oracle model, subject to its composition and Fiat–Shamir hypotheses, without reproving it. Assumptions 9.11 and 9.14 are the two assumptions on the Action proof that “Security” (§12) composes with the propositions of the preceding sections.

10 Note encryption

This section constructs the in-band delivery of a created note: its plaintext, its encryption to the recipient by one-shot Diffie–Hellman on the diversified base, the outgoing ciphertext keyed from ovk and the Action’s own public fields, and the trial decryption and acceptance checks by which a recipient obtains a note consistent with the published cmx . It states the two assumptions that note encryption adds, proves decryption correctness, confidentiality and key privacy of the encryption to the recipient, confidentiality of the outgoing ciphertext, and the consistency of accepted notes, and states what each key tier can decrypt.

10.1 The note plaintext

Every Action carries, beside the extracted commitment cmx of its created note, a *note ciphertext* $(\text{epk}^*, C_{\text{enc}}, C_{\text{out}})$. The component C_{enc} encrypts the plaintext of the created note so that the holder of the recipient’s incoming viewing key ivk recovers the note and its memo from the chain alone (protocol specification, §“In-band secret distribution (Sapling and Orchard)” and §“Action Descriptions”). The component C_{out} lets the sender, or any party that holds the outgoing viewing key ovk under which it was formed, recover the same plaintext (§10.3). The fields epk^* , C_{enc} and C_{out} are not inputs of the Action statement (Definition 9.2), which therefore does not constrain them; the checks that make a decrypted note consistent with cmx are made by the recipient (§10.4).

Definition 10.1 (Note plaintext). Let a created note have recipient address (d, pk_d) and value $v \in \{0, \dots, 2^{64} - 1\}$. Its *note plaintext* is the tuple $\text{np} := (\text{leadByte}, d, v, \text{rseed}, \text{memo})$, encoded

as the byte string

$$\text{leadByte} \parallel \text{LEBS2OSP}_{88}(d) \parallel \text{I2LEOSP}_{64}(v) \parallel \text{rseed} \parallel \text{memo},$$

where leadByte is one byte, the format version of “The note seed” (§4.3), equal to $0x03$ for an Ironwood-pool output; d is the 88-bit diversifier of the address (11 bytes); v occupies 8 bytes; rseed is the 32-byte note seed; and memo is a 512-byte string chosen by the sender, whose use is by agreement between sender and recipient (protocol specification, §“Note Plaintexts and Memo Fields” and §“Encodings of Note Plaintexts and Memo Fields”). The encoding has the fixed length $1 + 11 + 8 + 32 + 512 = 564$ bytes, so the length of a ciphertext carries no information about the plaintext.

The plaintext omits pk_d , ρ , ψ and rcm . The recipient computes $\text{pk}_d = [\text{ivk}] \text{g}_d$, takes $\rho = \text{nf}^{\text{old}}$ from the same Action (“Nullifier chaining”, §6.3), and derives ψ and rcm from rseed (§4.3).

For each output the sender draws rseed uniformly from the 32-byte strings, independently of every other output. The ephemeral secret key of this section is

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \text{I2LEOSP}_{256}(\text{nf}^{\text{old}}))),$$

derived in “The note seed” (§4.3) together with ψ and rcm . The sender redraws rseed when $\text{esk} = 0$ or $\text{cmx} = \perp$, so $\text{esk} \in \mathbb{F}_{\mathcal{P}_{\text{Vesta}}}^*$ (protocol specification, §“Sending Notes (Orchard)”; ZIP 2005, “Changes to the Protocol Specification”, §4.7.3).

10.2 Encryption to the recipient

Construction 10.2 (Encryption to the recipient). Let (d, pk_d) be the recipient’s address, $\text{g}_d = \text{DiversifyHash}(d)$ its diversified base (“Diversified addresses”, §3.3), and esk as in §10.1. A star encoding P^* enters a byte string as the 32 bytes $\text{LEBS2OSP}_{256}(P^*)$ (§2.1), with which it is identified below. The sender computes the *ephemeral public key*, the *shared secret* and the *encryption key*

$$\begin{aligned} \text{epk} &:= [\text{esk}] \text{g}_d, & s &:= [\text{esk}] \text{pk}_d, \\ K_{\text{enc}} &:= \text{KDF}(s, \text{epk}) := \text{BLAKE2b-256}(\text{Zcash_OrchardKDF}, s^* \parallel \text{epk}^*), \end{aligned}$$

with $\text{BLAKE2b-256}(P, x)$ unkeyed BLAKE2b with a 32-byte output and the 16-byte personalisation P , and the ciphertext $C_{\text{enc}} := \text{Enc}_{K_{\text{enc}}}(\text{np})$. The pair $\text{Enc}_K, \text{Dec}_K$ denotes ChaCha20-Poly1305 encryption and decryption under the 256-bit key K with the all-zero 96-bit nonce and empty associated data; decryption returns $\perp$ when the tag does not verify (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”). The Action publishes the 32-byte field epk^* and C_{enc} , of $564 + 16 = 580$ bytes, the plaintext and the 16-byte tag (protocol specification, §“Encryption (Sapling and Orchard)”, §“Orchard Key Agreement”, §“Orchard Key Derivation” and §“Symmetric Encryption”).

The key agreement is Diffie–Hellman on Pallas: public keys are non-identity points, private keys lie in $\mathbb{F}_{\mathcal{P}_{\text{Vesta}}}^*$, and both the derivation of a public key from a base and the agreement are scalar multiplication (protocol specification, §“Orchard Key Agreement”). A consensus rule requires epk^*

to be the canonical encoding of a non-identity Pallas point (protocol specification, §“Action Descriptions”). The fixed nonce is safe because every key encrypts once: K_{enc} depends on the fresh esk of its output, and the key of C_{out} is derived separately (§10.3). The scheme is the *Crypto Guide*’s Construction “DH-based hybrid public-key encryption” (§“Hybrid public-key encryption”) over the per-address base g_d in place of a fixed generator, as the same volume records for deployed note delivery (§“Shielded-note delivery as deployed hybrid encryption”).

Proposition 10.3 (Decryption correctness). *Let (d, pk_d) be an address of the incoming viewing key ivk , so $\text{pk}_d = [\text{ivk}] g_d$ (§3.3), and let $\text{epk} = [\text{esk}] g_d$ with $\text{esk} \in \mathbb{F}_{p_{\text{Vesta}}}^*$. Then*

$$[\text{ivk}] \text{epk} = [\text{esk}] \text{pk}_d = s.$$

Hence the holder of ivk computes $K_{\text{enc}} = \text{KDF}([\text{ivk}] \text{epk}, \text{epk})$ from the published epk^ and obtains $\text{Dec}_{K_{\text{enc}}}(C_{\text{enc}}) = \text{np}$, and a party that knows $(\text{pk}_d, \text{esk})$ obtains the same shared secret as $[\text{esk}] \text{pk}_d$.*

Proof. In the cyclic Pallas group $[a]([b]P) = [ab]P = [b]([a]P)$, so $[\text{ivk}]([\text{esk}] g_d) = [\text{esk}]([\text{ivk}] g_d) = [\text{esk}] \text{pk}_d$. The recipient decodes $\text{epk} = \text{abst}_{\text{p}}(\text{epk}^*)$, and because the star encoding is canonical (§2.1) it re-encodes to the published bytes. The function KDF is deterministic and both parties apply it to (s^*, epk^*) , so they derive the same K_{enc} ; correctness of ChaCha20-Poly1305 gives $\text{Dec}_{K_{\text{enc}}}(\text{Enc}_{K_{\text{enc}}}(\text{np})) = \text{np}$. This is the *Crypto Guide*’s Theorem “Correctness of hybrid encryption” (§“Hybrid public-key encryption”) over the base g_d . $\square$

Assumption 10.4 (The note-encryption hashes as random oracles). In security arguments the key-derivation function KDF (BLAKE2b-256 personalised Zcash_OrchardKDF), the function PRF^{ock} (BLAKE2b-256 personalised Zcash_Orchardock , constructed in §10.3), and PRFexpand on inputs with leading byte $0x04$, the derivation of esk from rseed , are modelled as independent random oracles (*Crypto Guide*, §“The random oracle model”, Definition “Random oracle”). The PRF assumption (Assumption 2.12) does not suffice for the last: it requires a key unknown to the adversary, whereas the plaintext that K_{enc} encrypts contains rseed , the key from which esk , and hence K_{enc} , is derived. On its other inputs PRFexpand is used only under Assumption 2.12. The assumption is a heuristic about BLAKE2b, not a theorem; every result that uses KDF , PRF^{ock} or the derivation of esk names it.

Assumption 10.5 (Confidentiality and integrity of ChaCha20-Poly1305). For a uniform 256-bit key that encrypts a single plaintext, ChaCha20-Poly1305 with the all-zero nonce and empty associated data is

- (a) *confidential*: the encryptions of any two plaintexts of equal length are computationally indistinguishable; and
- (b) *integral*: no efficient adversary without the key outputs a ciphertext, other than one it was given, that decrypts to a value other than $\perp$.

The *Crypto Guide*’s Theorem “Security of ChaCha20-Poly1305” (§“The ChaCha20-Poly1305 AEAD”) derives both, with explicit bounds, from the pseudorandomness of ChaCha20. The protocol specification requires one-time INT-CTXT and IND-CPA security of the scheme,

properties (b) and (a) above (*Crypto Guide*, Definitions “Ciphertext integrity, INT-CTXT” and “CPA security”, §“Authenticated encryption with associated data” and §“Chosen-plaintext security”; protocol specification, §“Symmetric Encryption”). The scheme is not key-committing: a party that chooses two keys can construct one ciphertext that decrypts validly under both (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Remark “Wrong-key rejection and key commitment”). No result of this volume relies on the tag against a sender who chooses the key.

Proposition 10.6 (Confidentiality and key privacy of note encryption). *Let a key be generated honestly with $\text{use_sk} = \text{false}$. An efficient adversary receives the address (d, pk_d) of the key at an index of its choice and data z that an efficient algorithm computes independently of the key and of rseed ; it chooses a value v , a memo and $\rho \in \mathbb{F}_{\text{Pallas}}$. An output to (d, pk_d) with these values and $\rho = \text{nf}^{\text{old}}$ is formed as in §10.1 and the construction above, with rseed uniform, and cmx is the extracted commitment of its note. Then the triple $(\text{epk}^*, C_{\text{enc}}, \text{cmx})$ is computationally indistinguishable from (E^*, C', cmx) , where E is a uniform non-identity Pallas point and C' is the encryption of a fixed 564-byte string under an independent uniform key. The hypotheses are Assumptions 3.16, 10.4, 10.5, 2.8 (for g_d), 3.13 (for the joint distribution of ivk and d) and 2.12 (for the dependence of cmx on rseed), together with Assumption 2.22 for Lemma “Rejection in key generation” (§3.2).*

Since E and C' are computed from neither the plaintext nor the address, the pair $(\text{epk}^*, C_{\text{enc}})$ reveals neither the plaintext, rseed included, nor the recipient’s address: given two addresses of independent keys, the data z may contain the second, and the ciphertext is indistinguishable from one independent of both. The value cmx is included so that the result applies within an Action. Of the hypotheses only Assumption 3.16, used through the computational Diffie–Hellman problem that it implies, concerns the one-shot key agreement between esk and pk_d ; Assumptions 10.4 and 10.5 concern the hash functions and the cipher. The adversary is passive: the volume does not claim indistinguishability of encryptions against an adversary that may also obtain decryptions of ciphertexts of its choice other than the challenge, before and after receiving it (IND-CCA2), which the protocol specification, §“Key Derivation”, requires of the scheme.

Proof. The proof is a sequence of games (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”); each reduction below runs the adversary and the rest of the game. Let Q be the event that the adversary queries the oracle of Assumption 10.4 for PRFexpand at an input keyed by rseed .

Preliminary step. As in the proof of Proposition 3.18, the key is replaced by a single draw without rejection and its triple $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$ by independent uniform elements (Lemma 2.15, Assumption 2.12; Lemma “Rejection in key generation”, §3.2, Assumptions 2.22 and 2.8). Then (ivk, dk) is replaced by $(\text{Commit}_{\text{ivk}}^{\text{ivk}}, (\text{ak}, \text{nk}), u_1)$ (Assumption 3.13; the reduction draws ask and nk itself). Now $d = \text{FF1-AES256}_{u_1}(\text{LE}_{88}(j))$, and ivk is, independently of d , 0 with probability $1/p_{\text{Vesta}}$ and otherwise uniform on the set X_{Pallas} of x -coordinates of non-identity points, of $(p_{\text{Vesta}} - 1)/2$ elements, except on the negligible event $\hat{M} = \perp$ (Lemma “Distribution of ivk ”, §3.2). The redraw of rseed is replaced by a single trial, at the cost of the redraw probability, negligible as in the proof of Proposition 4.8.

(1) By Assumption 10.4, esk is ToScalar of a random-oracle output at an input keyed by rseed . It is replaced by an independent uniform element of $\mathbb{F}_{p_{\text{Vesta}}}^*$. The oracle’s value at that input is used

nowhere else, so the games are identical until Q occurs; they differ by at most $\Pr[Q]$ in the new game plus the reduction distance of ToScalar, below 2^{-257} , and $1/p_{\text{Vesta}}$ for the value 0 (*Crypto Guide*, §“Security as a game”, Remark “Game-hopping”; §2.3).

(2) The key K_{enc} is replaced by an independent uniform key. The games differ only on the event C that the adversary queries the KDF oracle at $s^* \parallel \text{epk}^*$, and $s = [\text{esk}] \text{pk}_d$ solves the computational Diffie–Hellman instance $(g_d, \text{pk}_d, \text{epk})$. This is the game hop of the *Crypto Guide*’s Theorem “Passive security of hybrid encryption” (§“Hybrid public-key encryption”), with loss factor the number q_K of KDF queries, run over g_d . A distinguisher $\mathcal{D}$ for Assumption 3.16 receives $([a]P, [b]P, [c]P)$ with a uniform on X_{Pallas} and b, c uniform. It answers each new query of GroupHash under the domain `z.cash:0rchain-gd` with $[t]P$ for a fresh uniform t , recorded, a uniform point as in the model of Assumption 2.8, so that $g_d = [t]P$ with t known. It sets $\text{pk}_d := [t]([a]P) = [a]g_d$, which embeds a as ivk , and $\text{epk} := [t]([b]P) = [b]g_d$, which embeds b as esk ; it draws u_1 and rseed , computes d , cmx and C_{enc} under a uniform key itself, and answers the other oracles lazily. It then decodes the first 32 bytes of a uniformly chosen KDF query to a point S and outputs 1 exactly when $S = [t]([c]P)$. The simulation differs from the game only when $\text{ivk} = 0$ in the game, or $b = 0$ or $t = 0$ in the simulation, of total probability at most $3/p_{\text{Vesta}}$. On a Diffie–Hellman triple $\mathcal{D}$ therefore outputs 1 with probability at least $\Pr[C]/q_K - 3/p_{\text{Vesta}}$; on a random triple the point $[t]([c]P)$ is, for $t \neq 0$, uniform and independent of the view, and $\mathcal{D}$ outputs 1 with probability at most $4/p_{\text{Vesta}}$. An algorithm that computes Diffie–Hellman secrets on Pallas thus decides DDH, and $\Pr[C] \leq q_K (\epsilon_D + 7/p_{\text{Vesta}})$ for the advantage ϵ_D of $\mathcal{D}$. The exponent embedded as ivk is uniform on X_{Pallas} in Assumption 3.16. Against the computational Diffie–Hellman problem with a uniform exponent on $\mathbb{F}_{p_{\text{Vesta}}}$, which lies in X_{Pallas} with probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$ and on those instances gives an exact simulation, the same reduction loses a factor $2p_{\text{Vesta}}/(p_{\text{Vesta}} - 1)$, about 2 (proof of Proposition 3.18, part (d)).

(3) Now $\text{epk} = [\text{esk}] g_d$ with esk uniform on $\mathbb{F}_{p_{\text{Vesta}}}^*$ is a uniform non-identity point for every generator g_d , and the KDF context contains no key of the recipient (*Crypto Guide*, §“Key privacy”, Remark “Key privacy of note encryption and diversified addresses”). The ciphertext C_{enc} is an encryption under a uniform key used once, and Assumption 10.5(a) replaces np by the fixed string; the reduction obtains C_{enc} from its challenger and computes the rest. The last game outputs (E^*, C', cmx) .

(4) In the last game rseed enters the view only through cmx , that is through $\text{PRFexpand}_{\text{rseed}}$ under the leading bytes `0x09` and `0x0B`. A distinguisher with oracle access to $\text{PRFexpand}_{\text{rseed}}$ or to a random function computes cmx from its oracle, runs the adversary, and outputs 1 when some key k of a query at a leading byte `0x04` satisfies $\text{PRFexpand}_k([0x09] \parallel \text{l2LEOSP}_{256}(\rho))$ equal to its oracle’s answer at that input. It outputs 1 with probability at least $\Pr[Q]$ in the first case. In the second the view depends on that answer only through cmx , and the trapdoor rcm , ToScalar of the independent answer at the `0x0B` input, is within 2^{-257} of uniform; by Proposition 4.5(a), cmx is therefore independent of that answer up to $2^{-257} + \epsilon_{\perp}$, for $\epsilon_{\perp}$ the probability that the hash point of $\mathcal{M}(n)$ is $\perp$, negligible as in the proof of Proposition 4.8. The distinguisher thus outputs 1 with probability at most $q \cdot 2^{-512} + 2^{-257} + \epsilon_{\perp}$, for q such queries, and $\Pr[Q]$ in the last game is negligible under Assumption 2.12 (with Lemma 2.15). Every reduction of hops (2) and (3) draws rseed itself and detects Q , so each hop changes the probability of Q by at most its own cost. Hence $\Pr[Q]$ is negligible in the game of hop (1) as well, and the sum of the costs of all hops is negligible.

The DDH hybrid of the *Crypto Guide*’s Theorem “ElGamal is key-private under DDH” (§“Key privacy”) is not used: as stated it embeds a uniform exponent of $\mathbb{F}_{p_{\text{Vesta}}}$ as the recipient’s key, whereas ivk is not uniform on $\mathbb{F}_{p_{\text{Vesta}}}$; and the factor-2 argument above transfers a search problem such as computational Diffie–Hellman, not a decision problem, since on instances outside X_{Pallas} a distinguisher’s output is unconstrained. $\square$

10.3 The outgoing ciphertext

Construction 10.7 (Outgoing ciphertext). The sender chooses an outgoing viewing key ovk of its own, such as that of the address whose note the Action consumes (“Viewing keys”, §3.2), or $\text{ovk} = \perp$. If $\text{ovk} \neq \perp$, the *outgoing cipher key* is

$$\begin{aligned} \text{ock} &:= \text{PRF}_{\text{ovk}}^{\text{ock}}(\text{cv}^{\text{net}}, \text{cmx}, \text{epk}) \\ &:= \text{BLAKE2b-256}(\text{Zcash_Orchardock}, \text{ovk} \parallel (\text{cv}^{\text{net}})^* \parallel \text{I2LEOSP}_{256}(\text{cmx}) \parallel \text{epk}^*), \end{aligned}$$

where cv^{net} is the Action’s net value commitment (Definition 8.2) and cmx its extracted note commitment (Definition 4.4), and the *outgoing plaintext* is the $32 + 32 = 64$ -byte string $\text{op} := \text{pk}_d^* \parallel \text{I2LEOSP}_{256}(\text{esk})$. If $\text{ovk} = \perp$, the sender draws ock uniformly from the 256-bit strings and op uniformly from the 64-byte strings. In both cases $C_{\text{out}} := \text{Enc}_{\text{ock}}(\text{op})$, of $64 + 16 = 80$ bytes, and the Action publishes it; with $\text{ovk} = \perp$ no outgoing viewing key recovers it (protocol specification, §“Encryption (Sapling and Orchard)”, §“Sending Notes (Orchard)” and §“Pseudo Random Functions”).

Construction 10.8 (Recovery with ovk). For an Ironwood-pool output, the holder of ovk

- (i) computes ock from ovk and the Action’s public cv^{net} , cmx and epk^* , and decrypts C_{out} under ock to the outgoing plaintext op , rejecting $\perp$;
- (ii) parses op as a 32-byte string b_1 followed by a 32-byte string b_2 , and reads esk as the integer $\text{LEOSP}_{256}(b_2)$, rejecting $\text{esk} \geq p_{\text{Vesta}}$, and pk_d as the point $\text{abst}_p(b_1)$, rejecting $\perp$ and $\mathcal{O}$;
- (iii) computes $s := [\text{esk}] \text{pk}_d$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$ and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$, rejecting $\perp$;
- (iv) rejects unless $\text{leadByte} = 0x03$;
- (v) with $\rho := \text{nf}^{\text{old}}$ of the same Action, rejects unless

$$\text{esk} = \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \text{I2LEOSP}_{256}(\rho)));$$

- (vi) computes $\text{g}_d := \text{DiversifyHash}(d)$, then ψ , then rcm under $0x03$ from g_d , pk_d , v , ρ and ψ , by steps (iv) and (v) of the derivation of §4.3;
- (vii) rejects if the note commitment of $((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ (Definition 4.4) is $\perp$ or its x -coordinate differs from cmx , or if $([\text{esk}] \text{g}_d)^* \neq \text{epk}^*$;
- (viii) otherwise returns the note $((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$ and memo.

The order g_d and pk_d , then ψ , then rcm is forced because rcm depends on all three. The specification's decryption sections still use the legacy derivation of rcm ; ZIP 2005 specifies the reordered procedure (protocol specification, §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”); ZIP 2005, “Changes to the Protocol Specification”, §4.20.2 and §4.20.3). The specification's further check that b_1 re-encodes pk_d holds for every string that $abst_p$ decodes, the star encoding being canonical (§2.1). For pk_d the specification's procedure rejects only $\perp$; the rejection of $\mathcal{O}$ in step (ii) makes the returned pk_d a transmission key of Definition “Note” (§4.1).

Remark 10.9 (Transplant resistance). The key ock is a function of the Action's own cv^{net} , cmx and epk . A ciphertext C_{out} that a party without ovk copies into an Action whose (cv^{net}, cmx, epk) differ is decrypted by the holder of ovk under a key that, by Assumption 10.4, is uniform and independent of the copied ciphertext; by Assumption 10.5(b) that decryption returns $\perp$ except with negligible probability. A copied C_{out} is therefore rejected, not attributed to the other Action.

Proposition 10.10 (Confidentiality of the outgoing ciphertext). *Let the sending key be generated honestly with $use_qsk = \text{false}$. To an efficient party that holds neither its ovk nor its r_{ivk} , and may hold its ak , nk , ivk and dk , each ciphertext C_{out} that the holder of the key forms is computationally indistinguishable, jointly with every other public field of its Action, from the encryption of a fixed 64-byte string under an independent uniform key. The hypotheses are Assumptions 2.12, 3.13, 10.4, 10.5, 9.14, 2.22 and 2.8.*

Proof. As in the preliminary step of the proof of Proposition 10.6, the key is replaced by a single draw without rejection and its triple (ask, nk, r_{ivk}) by independent uniform elements (Lemma 2.15, Assumption 2.12; Lemma “Rejection in key generation”, §3.2, Assumptions 2.22 and 2.8). After this step, Assumption 3.13 makes ovk indistinguishable from a uniform 32-byte string independent of ivk and dk ; the reduction draws ask and nk itself, obtains (ivk, dk, ovk) from its challenger, computes every other field, and simulates the Action proof, whose witness contains r_{ivk} , under Assumption 9.14. Then ock is a random-oracle output (Assumption 10.4) at an input that the party queries with negligible probability, hence uniform, and distinct Actions give distinct inputs, their epk differing except with negligible probability, so each ock encrypts once; Assumption 10.5(a) then applies. With $ovk = \perp$, ock is uniform by construction. The ciphertext C_{out} thus reveals nothing about (pk_d, esk) . $\square$

A holder of ovk recovers, for every output sent under that ovk , the pair (pk_d, esk) , hence the shared secret $[esk] pk_d$ (Proposition 10.3), K_{enc} , the note plaintext and the recipient's address (d, pk_d) ; in this way the sender, or a party to whom the sender discloses ovk , reads outgoing notes (protocol specification, §“Decryption using an Outgoing Viewing Key (Sapling and Orchard)”). By this route it reads only outputs sent under that ovk . Detecting the notes sent to the key's own addresses by trial decryption requires ivk (§10.4), and ovk yields no ivk (Proposition 3.18, “Capability separation”, part (c), for keys generated with $use_qsk = \text{false}$).

10.4 Trial decryption and note acceptance

Construction 10.11 (Trial decryption). No public field of an Action names its recipient, so the holder of an incoming viewing key attempts every output: it computes $\text{epk} := \text{abst}_p(\text{epk}^*)$, rejecting $\perp$, which consensus excludes, then $s := [\text{ivk}] \text{epk}$, $K_{\text{enc}} := \text{KDF}(s, \text{epk})$ and $\text{np} := \text{Dec}_{K_{\text{enc}}}(C_{\text{enc}})$ (protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”). A result other than $\perp$ is a *candidate*, not an accepted note.

The same ivk serves the outputs of both pools, since keys and addresses are defined for the Orchard protocol, not for a pool (§3.4), and an address does not name a pool (ZIP 326, “Scanning for Ironwood-pool and Orchard-pool notes”); the pool of an output is that of the bundle that carries it. For an honestly produced ciphertext under a key independent of the trial key, decryption returns $\perp$ except with negligible probability (Assumption 10.5(b); *Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Remark “Wrong-key rejection and key commitment”); against a sender who chooses the keys the tag gives no such guarantee.

Construction 10.12 (Acceptance of an Ironwood-pool output). Given a candidate $\text{np} = (\text{leadByte}, d, v, \text{rseed}, \text{memo})$ of an Ironwood-pool output, the recipient accepts only if every check passes:

- (i) $\text{leadByte} = 0x03$, the recipient’s enforcement of the lead-byte rule of “The note seed” (§4.3);
- (ii) with $\rho := \text{nf}^{\text{old}}$ of the same Action and $\underline{\rho} := \text{l2LEOSP}_{256}(\rho)$, the note is re-derived in the order

$$\begin{aligned} g_d &:= \text{DiversifyHash}(d), & \text{pk}_d &:= [\text{ivk}] g_d, \\ \psi &:= \text{ToBase}(\text{PRFexpand}_{\text{rseed}}([0x09] \parallel \underline{\rho})), \\ \text{rcm} &:= \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(\text{pre_rcm})), \end{aligned}$$

with the 137-byte string pre_rcm of §4.3 formed from g_d , pk_d , v , ρ and ψ ;

- (iii) $([\text{esk}] g_d)^* = \text{epk}^*$ for

$$\text{esk} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}([0x04] \parallel \underline{\rho})),$$

a check that ZIP 2005 places directly after the computation of g_d ;

- (iv) the note commitment cm' of the re-derived note (Definition 4.4) is not $\perp$ and $\text{Extract}_p(\text{cm}') = \text{cmx}$.

The accepted note is $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$, with memo (ZIP 2005, “Changes to the Protocol Specification”, §4.20.2 and §4.20.3; protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)” and §“Note Plaintexts and Memo Fields”).

The order in (ii) is forced because rcm depends on g_d , pk_d and ψ ; the recovery with ovk of §10.3 follows the same order. The specification’s decryption sections still use the legacy derivation of

rcm; ZIP 2005 specifies both reordered procedures. The specification's check $\text{rcm} < p_{\text{Vesta}}$ holds for every output of ToScalar.

Remark 10.13 (Role of check (iii)). The Action statement (Definition 9.2) does not constrain epk. Check (iii) makes the recipient accept only an epk formed on the base g_d of the diversifier in the plaintext. Without it a sender could agree the key with one address of a recipient while naming the diversifier of another, and link the two addresses by observing whether the recipient accepts (ZIP 212, “Motivation”).

Remark 10.14 (Where the Ironwood derivation is checked). The Action statement takes ψ and rcm as witnesses and does not check their derivation from rseed (Definition 9.2). An accepted Ironwood-pool note has, by checks (i), (ii) and (iv), ψ and rcm derived under $0x03$ and the commitment cmx. For a coinbase output, consensus runs the recovery with ovk of §10.3 under the all-zero ovk (“Chain state and pool rules”, §11.4), which checks the lead byte and the whole $0x03$ derivation. For every other output, these recipient checks are the only check of the $0x03$ note format.

Orchard-pool outputs carry lead byte $0x02$ and use the legacy trapdoor derivation of the protocol specification, §“Sending Notes (Orchard)”; the recipient decrypts them with the same ivk under that byte and derivation (§“Note Plaintexts and Memo Fields”), on which this volume does not rely.

Proposition 10.15 (Consistency of accepted notes). *Let the acceptance procedure accept an Ironwood-pool output with note $n = ((d, \text{pk}_d), v, \rho, \psi, \text{rcm})$.*

- (a) *The note n is addressed to $(d, [\text{ivk}] g_d)$, an address of the recipient's key; it has $\rho = \text{nf}^{\text{old}}$ of the same Action; and its ψ and rcm are derived from its rseed under $0x03$.*
- (b) *The extracted commitment of n satisfies*

$$\text{Extract}_p(\text{NoteCommit}_{\text{rcm}}(M(n))) = \text{cmx}.$$

- (c) *For every efficient sender, which may choose epk^* , C_{enc} , C_{out} and the keys under which they are formed, the probability that the recipient accepts n while the sender outputs a note $n' \neq n$ with $\text{cmx}(n') = \text{cmx}$ is negligible, under the binding of note commitments (Proposition 4.5), hence under Assumptions 2.22 and 2.8. In particular, under Assumption 9.11, the committed values and the trapdoor of the created note that an extractor recovers from the Action proof, which satisfy condition A2 of the Action statement (Definition 9.2), are $(g_d, \text{pk}_d, v, \rho, \psi)$ and rcm modulo p_{Vesta} , those of n , except with negligible probability.*

Proof. Parts (a) and (b) hold by construction: acceptance requires checks (i), (ii) and (iv), and check (iv) recomputes the commitment of n deterministically and compares its x -coordinate with cmx, whatever the ciphertext and whichever key decrypted it. The address $(d, [\text{ivk}] g_d)$ is an address of the key because FF1-AES256_{dk} is a permutation of the 88-bit strings, so every d is the diversifier of one index (§3.3).

(c) Consider the algorithm that generates the recipient's key, runs the sender, runs the acceptance procedure on its output, and outputs n and the sender's n' . When the recipient

accepts, n opens $\text{cmx} \neq \perp$ by (b), so an $n' \neq n$ with $\text{cmx}(n') = \text{cmx}$ gives two distinct notes with equal extracted commitments other than $\perp$, which Proposition 4.5(b) excludes except with negligible probability. For the particular case, the extractor of Assumption 9.11 run on the sender is such a sender-side algorithm, with an opening of the committed values and a trapdoor in place of a note. Outside the negligible $\perp$ -weakened case of condition A2, which yields an input on which NoteCommit returns $\perp$ (Proposition 2.24(iii)), its opening (M', rcm') satisfies $\text{Extract}_p(\text{NoteCommit}_{\text{rcm}'}(M')) = \text{cmx}$. Step 2 of the proof of Proposition 4.5(b), which uses Proposition 2.24(ii) and the fact that $H_D \neq \mathcal{O}$, then gives $M' = M(n)$ and $\text{rcm}' \equiv \text{rcm} \pmod{p_{\text{Vesta}}}$ except with negligible probability, and $M(n)$ determines $(g_d, \text{pk}_d, v, \rho, \psi)$ because the encoding is injective. The proof uses neither the integrity of ChaCha20-Poly1305 nor key commitment, which the scheme lacks (Assumption 10.5): a ciphertext valid under two keys yields under each a candidate that must still pass check (iv). $\square$

On acceptance the recipient records n with its note position, the index of cmx among the leaves of the note commitment tree of the output's pool, fixed by the order in which the chain appends extracted commitments ("The Merkle hash and the tree", §5.1; protocol specification, §"Note Commitment Trees"). The position determines the authentication path, which the recipient computes from public data (Lemma 5.15) and which, with the opening n of the leaf cmx (Proposition 10.15), a later Action consuming n requires (conditions A1 and A3 of Definition 9.2).

The key tiers of Proposition 3.18 ("Capability separation") have the following powers over note ciphertexts, for keys generated with $\text{use_qsk} = \text{false}$.

Incoming tier. A holder of the incoming viewing key (dk, ivk) , of which trial decryption and acceptance use only ivk , detects and reads every note sent to any address of the key, value and memo included. It cannot compute the nullifiers of these notes, which require nk , and (dk, ivk) yields neither nk nor ask (Proposition 3.18(b)). It cannot link a published nullifier to one of them, for notes with pairwise distinct ρ (Proposition 6.10), so the nullifier sets alone do not show it which are consumed; and it cannot authorise a spend.

Full-viewing tier. A holder of the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$ derives (dk, ivk) and ovk ("Viewing keys", §3.2). It detects and reads incoming notes as above, and reads the outputs sent under its ovk (§10.3), not those sent under $\text{ovk} = \perp$ or another outgoing viewing key. With nk it computes the nullifier $\text{nf} = \text{DeriveNullifier}_{\text{nk}}(\rho, \psi, \text{cm})$ of each accepted note (Definition 6.3) and detects its consumption by the membership of nf in the nullifier set of the note's pool ("Nullifier sets", §6.2; protocol specification, §"Decryption using an Incoming Viewing Key (Sapling and Orchard)", notes). It cannot spend, since it yields no ask (Proposition 3.18(a)). For each unspent note the owner thus holds the note, its position, its authentication path and its nullifier, the consumed-note inputs of an Action (Definition 9.2); spending authority is ask .

11 Transactions and consensus rules

This section assembles the public data of an Action and of a bundle, states the transaction format that carries them, constructs the digest that both kinds of signature sign and proves in Proposition 11.10 which data the signatures fix, states the chain state and the rules of each pool, and gives the procedures of the sender and of the verifier, with a worked two-Action payment.

Table 6. The fields of an Action description, in the order of their encoding, with the type that parsing enforces and the byte length of the encoding.

Field	Type enforced on parsing	Encoding	Bytes	Constructed in
cv^{net}	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$, the identity admitted	$(cv^{\text{net}})^*$	32	Definition 8.2
nf	$\{0, \dots, p_{\text{Pallas}} - 1\}$	$\text{l2LEOSP}_{256}(\text{nf})$	32	Definition 6.3
rk	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$; the identity excluded by a separate rule	rk^*	32	§7.2
cmx	$\{0, \dots, p_{\text{Pallas}} - 1\}$	$\text{l2LEOSP}_{256}(\text{cmx})$	32	Definition 4.4
epk	$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}}) \setminus \{\mathcal{O}\}$	epk^*	32	§10.2
C_{enc}	580-byte string	itself	580	§10.2
C_{out}	80-byte string	itself	80	§10.3
Action description			820	

11.1 Action descriptions and bundles

The objects of this subsection are constructed in §4 to §10. The subsection fixes their encodings and their grouping; it redefines neither the Action nor the bundle (Definitions “Action” and “Bundle”, §4.4). As in §10.2, a star encoding P^* enters a byte string as the 32 bytes $\text{LEBS2OSP}_{256}(P^*)$ and is identified with them.

Definition 11.1 (Action description). The *Action description* of an Action is the byte string

$$(cv^{\text{net}})^* \parallel \text{l2LEOSP}_{256}(\text{nf}) \parallel rk^* \parallel \text{l2LEOSP}_{256}(\text{cmx}) \parallel epk^* \parallel C_{\text{enc}} \parallel C_{\text{out}}$$

of $5 \cdot 32 + 580 + 80 = 820$ bytes, whose fields, in this order, are those of Table 6: the net value commitment of Definition 8.2, the nullifier of the consumed note (Definition 6.3), the randomised validating key of “Randomised validating keys” (§7.2), the extracted commitment of the created note (Definition 4.4), and the ephemeral public key and the two ciphertexts of “Encryption to the recipient” (§10.2) and “The outgoing ciphertext” (§10.3) (protocol specification, §“Action Descriptions” and §“Action Description Encoding and Consensus”).

Every field of an Action description must be the canonical encoding of its type (protocol specification, §“Action Descriptions”; for cmx also §“Action Description Encoding and Consensus”). Hence nf and cmx are integers below p_{Pallas} , and neither is checked to be the x -coordinate of a Pallas point; epk is a non-identity point by its type; rk is a point, and the rule of “Randomised validating keys” (§7.2) excludes the identity; and cv^{net} may be the identity. The Action’s spend-authorisation signature, of 64 bytes (“The RedPallas signature scheme”, §7.1), and its proof are not part of the Action description. The bundle carries the signatures, one per Action, and one aggregate proof, whose per-Action instances are in the order of the Action descriptions (ZIP 229, “Consensus Rules”).

A bundle of one pool with $n \geq 1$ Actions publishes:

- (i) its n Action descriptions;
- (ii) one anchor rt (“Anchors”, §5.2), shared by all n Actions: no Action description carries

- an anchor, and the transaction carries one anchor field for each pool with Actions, anchorOrchard and anchorIronwood;
- (iii) one flags byte (“Bundle flags”, §9.1), whose bit 0 is enableSpends and bit 1 enableOutputs, and whose bit 2 is enableCrossAddress in flagsIronwood and reserved and 0 in flagsOrchard;
 - (iv) the balancing value v^{balance} of “The balancing value” (§8.2);
 - (v) one aggregate proof over its n Actions (“The Action circuit and the Halo 2 proof”, §9.3);
 - (vi) n spend-authorisation signatures, one per Action;
 - (vii) one binding signature (“The binding signature”, §8.3).

The anchor and the flags are common to all Actions of the bundle and are encoded once (protocol specification, §“Action Transfers and their Descriptions” and §“Action Descriptions”, note). A transaction carries at most one bundle per pool, hence one binding signature for each pool with Actions (ZIP 229, “Transaction Format” and “Consensus Rules”). Figure 4 shows each public field with the checks that consume it.

A bundle that consumes s real notes and creates o real notes has $n \geq \max(s, o)$ Actions, since every Action consumes exactly one note and creates exactly one; its $n - s$ consumed sides and $n - o$ created sides without a real note carry dummy notes of value 0 (Definition “Dummy notes”, §4.4; protocol specification, §“Dummy Notes (Orchard)”). When the consumed value exceeds the payment and the fee, the difference is *change*: an ordinary created note to an address of the sender, constructed as every other created note.

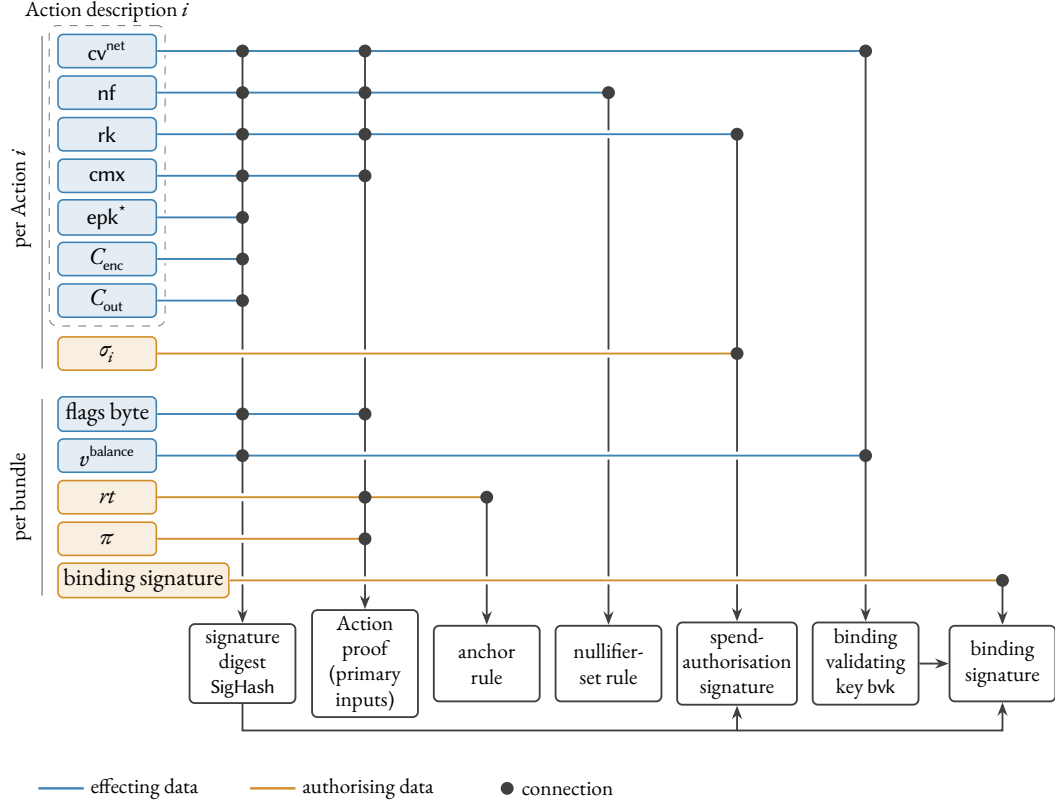


Figure 4. The public data of one bundle and the checks that consume them. Rows are the fields: per Action i , the net value commitment cv^{net} , the nullifier nf , the randomised validating key rk , the extracted commitment cmx , the ephemeral key epk^* and the ciphertexts C_{enc} and C_{out} of the Action description, and the spend-authorisation signature σ_i ; per bundle, the flags byte, the balancing value $v^{balance}$, the anchor rt , the aggregate proof π and the binding signature. Each column ends in the check or derived key that consumes it. A dot joins a field to a column; a row that crosses a column without a dot is not joined to it. Blue rows are effecting data and amber rows authorising data (“Transaction digests and signatures”, §11.3). Every effecting field, the ciphertexts included, joins the signature digest SigHash, which feeds every spend-authorisation signature and the binding signature; the anchor and the proof do not join it (Proposition 11.10). The fields nf , cmx , cv^{net} , rk , the flags and rt are primary inputs of the Action proof; rt also feeds the anchor rule, nf the nullifier-set rule, rk the spend-authorisation signature, and cv^{net} and $v^{balance}$ the binding validating key bvk.

11.2 The transaction format

The transaction formats in force are version 5, specified in ZIP 225, and version 6, specified in ZIP 229; a transaction of any other version, version 4 included, is invalid (protocol specification, §“Transaction Consensus Rules”, as amended by ZIP 2003, “Specification”; ZIP 259, Abstract). Of the two Orchard-protocol bundles a version-5 transaction can carry only the Orchard-pool bundle; a transaction with Ironwood-pool Actions is of version 6 (protocol specification, §“Action Descriptions”). The Orchard-pool rules of “Chain state and pool rules” (§11.4) apply to Orchard-pool Actions in both versions (ZIP 229, “Consensus Rules”). *Sprout*, Zcash’s first shielded protocol, spends its notes only through *JoinSplit descriptions*, its transfer descriptions, which only transaction versions 2 to 4 carry (protocol specification, §“Transaction Encoding and Consensus”); Sprout funds are therefore unspendable, and their value remains counted as issued, in the sense of the total issued supply defined in “Chain state and pool rules” (§11.4) (ZIP 2003, Abstract and “Interaction with the proposed Network Sustainability Mechanism”; ZIP 259, Abstract). This subsection describes version 6.

Construction 11.2 (Version-6 transaction). A version-6 transaction consists of the fields of the version-5 format up to and including the Orchard component (ZIP 225), followed by the Ironwood component. Its first five fields form the header:

- (i) header, whose bit 31, the *fOverwintered* flag, is set and whose bits 0 to 30 hold the version, 6;
- (ii) *nVersionGroupId*, which must be `0xD884B698`;
- (iii) *nConsensusBranchId*, which must equal the consensus branch identifier used by the signature digest of “Transaction digests and signatures” (§11.3), the identifier of the consensus branch under which the transaction is validated, where a *consensus branch* is a sequence of consecutive blocks under one consensus rule set whose first block is the genesis block or follows a block validated under older rules, and each network upgrade fixes for the branch that it begins a globally unique non-zero 32-bit *consensus branch identifier* (ZIP 200, Terminology and “Specification”);
- (iv) *lock_time*, the transparent lock time, which no rule of this volume uses;
- (v) *nExpiryHeight*, the *expiry height*.

In version 6, bits 0 and 1 of *flagsOrchard* are named *enableSpends* and *enableOutputs*, as in *flagsIronwood*, and bits 2 to 7 of *flagsOrchard* are reserved and must be 0, in version 5 and version 6 alike (“Bundle flags”, §9.1). The Ironwood component has the structure of the Orchard component, with its own flags, balancing value, anchor, proof and signatures; both encode each Action as the 820-byte Action description of §11.1 (ZIP 229, “Transaction Format” and “Consensus Rules”; protocol specification, §“Transaction Consensus Rules”).

Definition 11.3 (Coinbase transaction). A *coinbase transaction* is the first transaction of a block; it collects the block subsidy, the value newly issued in that block, and the fees of the

Table 7. The Ironwood component of a version-6 transaction with n Ironwood-pool Actions (ZIP 229, “Transaction Format”). The fields after the count are present if and only if $n > 0$.

Field	Bytes	Content
nActionsIronwood	compactSize	the count n
vActionsIronwood	$820n$	the n Action descriptions of §11.1
flagsIronwood	1	bit 0 enableSpends, bit 1 enableOutputs, bit 2 enableCrossAddress, bits 3 to 7 reserved and 0
valueBalanceIronwood	8	the balancing value, signed 64-bit little-endian
anchorIronwood	32	the anchor rt
sizeProofsIronwood	compactSize	the proof length, $2720 + 2272n$
proofsIronwood	$2720 + 2272n$	the aggregate proof
vSpendAuthSigIronwood	$64n$	one spend-authorisation signature per Action, in Action order
bindingSigIronwood	64	the binding signature

block’s other transactions. Every block has exactly one (protocol specification, §“Coinbase Transactions”).

The field nExpiryHeight is 0, which disables expiry, or a block height. The *expiry rule* is: for a transaction other than a coinbase transaction, nExpiryHeight is at most 499 999 999, and such a transaction with nExpiryHeight $\neq 0$ is invalid in every block of height greater than nExpiryHeight; the nExpiryHeight of a coinbase transaction equals the height of its block (protocol specification, §“Transaction Consensus Rules”; ZIP 203, “Specification”; ZIP 229, “Transaction Format”). The sender’s choice of the expiry height is stated in “Construction of a transaction” (§11.5).

Definition 11.4 (Ironwood component). Let n be the number of Ironwood-pool Actions of a version-6 transaction. The *Ironwood component* is the sequence of the fields of Table 7, in that order. The fields after nActionsIronwood are present if and only if $n > 0$, and an absent valueBalanceIronwood is taken to be 0. A count and a proof length are encoded as *compactSize*, the variable-length integer encoding of Bitcoin (ZIP 229, “Transaction Format”).

The consensus rules on the component are: $n < 2^{16}$; if $n > 0$, at least one of enableSpends and enableOutputs of flagsIronwood is 1; bits 3 to 7 of flagsIronwood are 0; valueBalanceIronwood lies in $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$, where $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ satoshi is a protocol constant (protocol specification, §“Constants”); and proofsIronwood has exactly $2720 + 2272n$ bytes (ZIP 229, “Consensus Rules”; protocol specification, §“Transaction Consensus Rules” and §“Action Description Encoding and Consensus”). The flags byte differs from flagsOrchard only in bit 2, which is enableCrossAddress in flagsIronwood and reserved and 0 in flagsOrchard (“Bundle flags”, §9.1). The proof length is the one that ZIP 229 fixes; the *Halo 2 Guide*, §“Halo 2’s specifics: parameters, generators from a hash, the deferred multi-scalar multiplication, and the proof bytes”, paragraph “The bytes”, describes what its constant and its per-Action term cover. For one Action the proof has $2720 + 2272 = 4992$ bytes.

11.3 Transaction digests and signatures

Write $h(P, x) := \text{BLAKE2b-256}(P, x)$, unkeyed BLAKE2b with a 32-byte output, the 16-byte personalisation P and the input x (§10.2); the personalisation is mixed into the initialisation vector (*Crypto Guide*, §“Domain separation and personalisation”). For a byte string C and $0 \leq a \leq b \leq |C|$, the string $C[a:b]$ consists of the bytes a to $b - 1$ of C , counted from 0. Every field enters a digest in its encoding of Table 6 or Table 7, and β denotes the 4-byte little-endian encoding of the consensus branch identifier.

Definition 11.5 (Transaction identifier). The *transaction identifier* of a version-6 transaction is

$$\text{txid} := h(\text{ZcashTxHash_} \parallel \beta, d_{\text{hdr}} \parallel d_{\text{tr}} \parallel d_{\text{sap}} \parallel d_{\text{orc}} \parallel d_{\text{lrw}}),$$

the Ironwood digest last, where:

- (i) the *header digest* d_{hdr} is h under ZTxIdHeadersHash of the 4-byte little-endian encodings of header, $n\text{VersionGroupId}$, $n\text{ConsensusBranchId}$, lock_time and $n\text{ExpiryHeight}$;
- (ii) the *transparent digest* d_{tr} and the *Sapling digest* d_{sap} are those of ZIP 244, “T.2: transparent_digest” and “T.3: sapling_digest”, the latter with the version-6 node of ZIP 229; they cover the transparent inputs and outputs and the Sapling component, the component of Zcash’s second shielded protocol;
- (iii) for $n \geq 1$ Ironwood-pool Actions, with the fields of Action i indexed by i , the *Ironwood digest* is

$$\begin{aligned} d_{\text{lrw}} &:= h(\text{ZTxIdIronwd_H_v6}, d_C \parallel d_M \parallel d_N \parallel \text{flagsIronwood} \parallel \text{valueBalanceIronwood}), \\ d_X &:= h(P_X, r_1^X \parallel \dots \parallel r_n^X) \quad (X \in \{C, M, N\}), \end{aligned}$$

with the *compact*, *memo* and *non-compact* records and personalisations

$$\begin{aligned} r_i^C &:= \text{nf}_i \parallel \text{cmx}_i \parallel \text{epk}_i^* \parallel C_{\text{enc},i}[0:52], & P_C &:= \text{ZTxIdIrActCH_v6}, \\ r_i^M &:= C_{\text{enc},i}[52:564], & P_M &:= \text{ZTxIdIrActMH_v6}, \\ r_i^N &:= (\text{cv}_i^{\text{net}})^* \parallel \text{rk}_i^* \parallel C_{\text{enc},i}[564:580] \parallel C_{\text{out},i}, & P_N &:= \text{ZTxIdIrActNH_v6}; \end{aligned}$$

for $n = 0$ it is $d_{\text{lrw}} := h(\text{ZTxIdIronwd_H_v6}, \varepsilon)$;

- (iv) the *Orchard digest* d_{orc} has the same structure over the Orchard-pool Actions, with flagsOrchard and $\text{valueBalanceOrchard}$, under the personalisation ZTxIdOrchardH_v6 , with the Action digests personalised

$$P_C = \text{ZTxIdOrcActCHash}, \quad P_M = \text{ZTxIdOrcActMHash}, \quad P_N = \text{ZTxIdOrcActNHash};$$

without Orchard-pool Actions it is $h(\text{ZTxIdOrchardH_v6}, \varepsilon)$.

(ZIP 244, “Digests”, “txid_digest”, “T.1: header_digest” and “T.4: orchard_digest”; ZIP 229, “Ironwood component digest”, “Anchor commitment (version 6)” and “Summary of the resulting digest structure”).

The ciphertext C_{enc} is the ChaCha20 encryption of the 564-byte note plaintext, byte for byte, followed by the 16-byte tag (*Crypto Guide*, §“The ChaCha20-Poly1305 AEAD”, Construction “ChaCha20-Poly1305”; “The note plaintext”, §10.1). The offsets therefore split it into the encryption of the 52-byte prefix of the plaintext, the lead byte, d , v and rseed ($1 + 11 + 8 + 32 = 52$), the encryption of the 512-byte memo, ending at $52 + 512 = 564$, and the tag. The empty Ironwood digest differs from the empty Orchard digest, since the two personalisations differ. Neither pool digest contains the anchor.

Definition 11.6 (Authorising-data digest; effecting and authorising data). The *authorising-data digest* of a version-6 transaction is

$$\text{auth} := h(\text{ZTxAuthHash}_- \parallel \beta, a_{\text{tr}} \parallel a_{\text{Sap}} \parallel a_{\text{Orc}} \parallel a_{\text{lrw}}),$$

where a_{tr} is the transparent scripts digest and a_{Sap} the Sapling authorising-data digest (ZIP 244, “Authorizing Data Commitment”; ZIP 229), and, for $n \geq 1$ Ironwood-pool Actions,

$$a_{\text{lrw}} := h(\text{ZTxAuthIronwdH}_v6, \text{proofsIronwood} \parallel \text{vSpendAuthSigIronwood} \parallel \text{bindingSigIronwood} \parallel \text{anchorIronwood}),$$

and $a_{\text{lrw}} := h(\text{ZTxAuthIronwdH}_v6, \varepsilon)$ for $n = 0$; the Orchard digest a_{Orc} has the same structure under ZTxAuthOrchaH_v6 . Each Orchard-protocol anchor is committed if and only if its component has Actions. The *effecting data* of a transaction are the data under the transaction identifier digest; its *authorising data* are the proofs, the signatures, the anchors and the transparent scripts, the data under the authorising-data digest (ZIP 229, “Ironwood component digest” and “Anchor commitment (version 6)”; ZIP 244, “Authorizing Data Commitment”).

ZIP 244, “Authorizing Data Commitment”, describes the pair of the transaction identifier and the authorising-data digest as a commitment to all data of the transaction. In version 6 the anchors are authorising data so that a transaction can be signed before its anchor is chosen and its proofs are computed; replacing a bundle’s anchor and proof together does not change the transaction identifier (ZIP 229, Motivation and “Anchor commitment (version 6)”). Below the two roots, a node whose directly hashed input differs between versions 5 and 6 has a version-6 personalisation of its own; the roots keep theirs and hash one more child in version 6. The nodes that omit or include an anchor in version 6 are personalised ZTxIdSSpendNH_v6 , ZTxAuthSapliH_v6 , ZTxIdOrchardH_v6 and ZTxAuthOrchaH_v6 (ZIP 229, “Ironwood component digest” and “Anchor commitment (version 6)”).

Definition 11.7 (Signature digest). The *signature digest* of a transaction is its digest not associated with a transparent input, under the hash type SIGHASH_ALL, which covers all transparent inputs and outputs:

$$\text{SigHash} := h(\text{ZcashTxHash}_- \parallel \beta, d_{\text{hdr}} \parallel s_{\text{tr}} \parallel d_{\text{Sap}} \parallel d_{\text{Orc}} \parallel d_{\text{lrw}}),$$

whose header and shielded children are those of txid and whose child s_{tr} is the transparent signature digest of ZIP 244, “S.2: transparent_sig_digest”. For a coinbase transaction and for a transaction without transparent inputs, $s_{\text{tr}} = d_{\text{tr}}$, so $\text{SigHash} = \text{txid}$; otherwise s_{tr} commits in addition

to the values and scripts of the coins that the transparent inputs spend. The digest SigHash is the message of every spend-authorisation signature and of each pool's binding signature, the message left as a parameter in "The RedPallas signature scheme" (§7.1) (ZIP 244, "Signature Digest"; ZIP 229, "Transaction Identifiers, Auth Digests, and Signature Digests"; protocol specification, §"Action Descriptions" and §"Balance and Binding Signature (Orchard)").

Proposition 8.10, which holds for every message, applies to the binding signature on SigHash.

Remark 11.8 (Version 5). In version 5 the root of the transaction identifier has four children, the header, transparent, Sapling and Orchard digests, under the same root personalisation. The Orchard digest, under `ZTxIdOrchardHash`, hashes

$$d_C \parallel d_M \parallel d_N \parallel \text{flagsOrchard} \parallel \text{valueBalanceOrchard} \parallel \text{anchorOrchard},$$

with the Action digests of version 6, and the Orchard authorising-data digest, under `ZTxAuthOrchardHash`, covers the proofs and the signatures only. In version 5 the Orchard anchor is therefore effecting data and fixed by the signatures; the proofs are authorising data in both versions. A version-5 transaction carries Orchard-pool Actions only (ZIP 244, "txid_digest", "T.4: orchard_digest" and "A.3: orchard_auth_digest").

Figure 5 draws both trees. The binding of the signatures needs one further assumption, stated at its first use.

Assumption 11.9 (Collision resistance of BLAKE2b). No efficient algorithm outputs, except with negligible probability, two distinct pairs (P, x) and (P', x') , each of a 16-byte personalisation and an input string, with $h(P, x) = h(P', x')$. The notion is the *Crypto Guide's* Definition "Collision resistance" (§"Security notions: preimage, second-preimage, and collision resistance"), read for the unkeyed deployment as that volume's Remark "Convention: keyed definitions, unkeyed deployments" (§"Hash functions and the random oracle model", §"Syntax") prescribes; the personalisation is mixed into the initialisation vector (§"Domain separation and personalisation"), so the pair (P, x) is the hashed input. It is used by Proposition 11.10 and by the consensus-branch binding below.

Proposition 11.10 (Bundle binding). *Let T be a version-6 transaction, and for a bundle of T , of either pool, let $\text{rk}_1, \dots, \text{rk}_n$ be the randomised validating keys of its Actions, $\text{rk}_i = [\text{rsk}_i] G^{\text{SpendAuth}}$ with $\text{rsk}_i = \text{ask}_i + \alpha_i$ as in "Randomised validating keys" (§7.2), where ask_i is the spend authorising key of the spending key that authorises Action i . Assume:*

- (H) *each ask_i is uniform on the non-zero scalars a for which $[a] G^{\text{SpendAuth}}$ has even y -coordinate (§3.1), and every value of the experiment other than a signature by the holder of ask_i is computed from $\text{ak}_i^P = [\text{ask}_i] G^{\text{SpendAuth}}$ and from values drawn independently of ask_i ;*
- (S) *the holder of ask_i signs under rsk_i only $\text{SigHash}(T)$, and signs any other message only under $\text{ask}_i + \alpha$ for a fresh randomiser α drawn by `GenRandom`.*

An adversary receives T and all other public data, and obtains on request, for messages of its choice, signatures of the key holders under fresh randomisers as in (S); it receives no ask_i and no rsk_i .

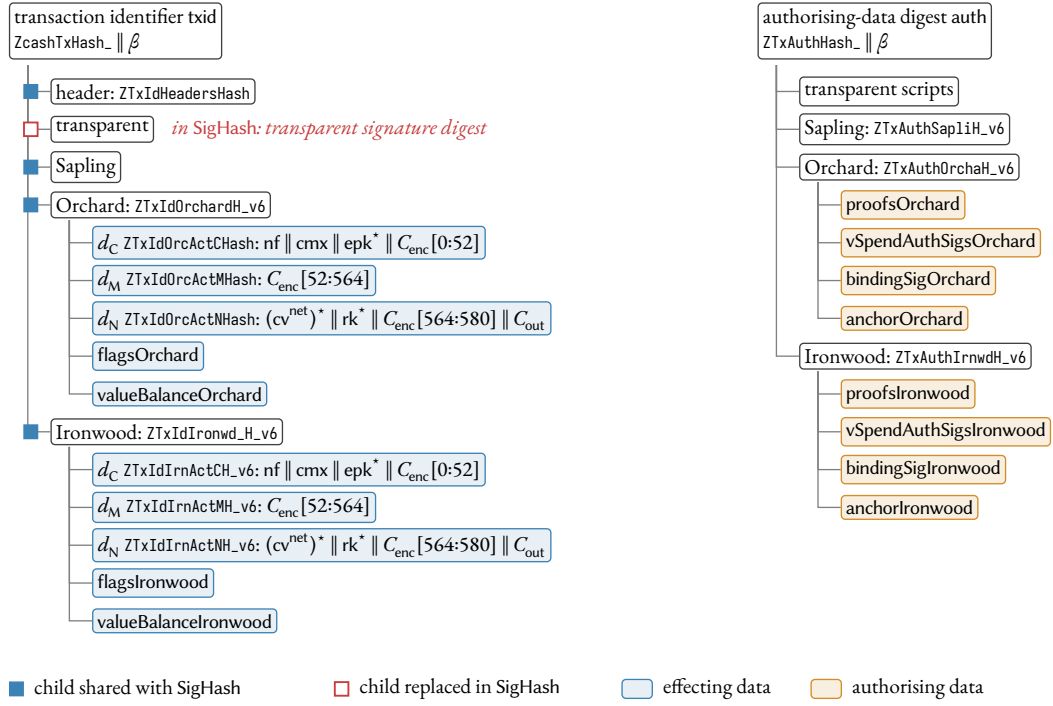


Figure 5. The version-6 transaction identifier (left) and authorising-data digest (right) as trees of personalised BLAKE2b-256 nodes, with the Orchard and Ironwood components expanded (ZIP 244; ZIP 229). Each node hashes the concatenation of its children, in order, under the personalisation shown; the string β encodes the consensus branch identifier. The Action digests d_C , d_M and d_N hash the listed fields of every Action of their component, Action after Action; the byte flagsIronwood is the Ironwood component's own flags byte. The header, transparent and Sapling digests are not expanded. The signature digest SigHash has the root personalisation and the children of txid (filled squares), except the transparent child (open square), which it replaces by the transparent signature digest. The anchors and the proofs lie under the authorising-data digest only. By Proposition 11.10, the signatures fix every field under txid and no field under the authorising-data digest.

Under Assumptions 11.9, 2.22 and 7.3, the probability that it outputs an accepted transaction T' that contains an Action description with validating key rk_i for some i and whose effecting data differ from those of T is negligible.

The signatures thus fix the effecting data: every Action's nf , cmx , cv^{net} , rk , epk , C_{enc} and C_{out} , the number and order of the Actions, each bundle's flags and balancing value, the header, with the consensus branch identifier and the expiry height, and the transparent data. They fix neither the anchor nor the proof of a bundle: for a pair (rt', π') such that π' verifies on the primary inputs formed with rt' , replacing (rt, π) by (rt', π') changes neither the transaction identifier nor any signed message, so every signature remains valid, and π' binds rt' as a primary input (Definition 9.2).

Proof. Let E be the event that the adversary outputs such a T' . On E , let i be the least index for which T' contains an Action description with key rk_i , and let σ' be the spend-authorisation signature that T' carries for it. Since T' is accepted, σ' is valid under rk_i on $m' := \text{SigHash}(T')$ (“Randomised validating keys”, §7.2). Let $m := \text{SigHash}(T)$, and split E into E_+ , where $m' = m$, and E_- , where $m' \neq m$.

Case E_- : a collision. The digest SigHash is computed by the tree of its definition from the effecting data and, when the transaction has transparent inputs, from the values and scripts of the coins that they spend. Each node hashes a string from which its children and its fields are recovered uniquely: its input consists of fixed-length fields and digests, of concatenations of fixed-length per-Action records whose number the length determines, or, for transparent outputs, of records whose variable-length script carries its length; the two nodes that share a personalisation, the transparent digest and the transparent signature digest, hash strings of different lengths (ZIP 244, “TxId Digest” and “Signature Digest”). Compare the trees of T and T' from the root, whose outputs are equal. At a node with equal outputs, either the two (personalisation, input) pairs differ, a collision of h , or they agree, and then the node's children have equal outputs and its fields are equal. The effecting data of T and T' differ, so some field, or some number of records, differs, and descending along the nodes that cover it ends in a collision. An algorithm that runs the experiment, which it can since it samples every key, and performs this descent outputs a collision with probability $\Pr[E_-]$, which is negligible by Assumption 11.9.

Case E_+ : a forgery. An algorithm $\mathcal{B}$ plays the adversary of Proposition 7.12. It receives $Y = [x] G^{\text{SpendAuth}}$ for x uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, with access to H and to the signing oracle, and draws i^* uniformly from $\{1, \dots, n\}$. If $Y = \mathcal{O}$ or Y has odd y -coordinate, it stops; otherwise, an event of probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$, the point Y has the distribution that (H) gives to $ak_{i^*}^P$, and $\mathcal{B}$ sets $ak_{i^*}^P := Y$. It computes every other value of the experiment as (H) permits, in particular the proof of T , whose witness (Definition 9.2) contains ak^P and α but not ask ; it draws every randomiser by GenRandom itself; it obtains each signature by the holder of ask_{i^*} , on m and on the adversary's requests, as the oracle's answer on the pair (α, M) ; and it produces the other signatures with keys it holds. The adversary's view is then that of the experiment. On E_+ with $i = i^*$, $\mathcal{B}$ outputs (rk_{i^*}, m', σ') with the randomiser α_{i^*} , of its own choice, as part (ii) of Proposition 7.12 admits. The oracle answered under the key rk_{i^*} only on m , except when a randomiser drawn for a request equals α_{i^*} , which has negligible probability by the lemma on the distance of GenRandom from uniform (§7.2); outside that event the output is a forgery. The index i^* is independent of

the adversary’s view, so $\mathcal{B}$ forges with probability at least

$$\frac{p_{\text{Vesta}} - 1}{2p_{\text{Vesta}}} \cdot \frac{\Pr[E_{\neq}]}{n}$$

less a negligible term, where $n < 2^{16}$. By Proposition 7.12(ii), under Assumptions 2.22 and 7.3, $\Pr[E_{\neq}]$ is negligible.

Anchor and proof. By the definitions above, the anchor and the proof of each bundle are inputs of the authorising-data digest and of no node of txid or SigHash. Replacing them changes neither, and every signature, whose message is SigHash, remains valid. The verifier checks the bundle’s proof against the primary inputs formed with the bundle’s anchor field (protocol specification, §“Action Descriptions”), now rt' , so the modified bundle is accepted only if π' verifies for rt' , and the anchor rule then applies to rt' . $\square$

Remark 11.11 (Hypothesis (H) for honest keys). For keys generated with `use_qsk = false` from uniform spending keys, hypothesis (H) holds up to a negligible change in the probability of every efficiently decidable event, in particular of the event of Proposition 11.10, which an algorithm holding the keys decides by verifying T' and comparing its effecting data with those of T . By Lemma 2.15 (Assumption 2.12) the triple $(ask, nk, r_{\text{ivk}})$ is indistinguishable from independent uniform elements; the sign normalisation of “The spending key and the spend-side secrets” (§3.1) maps a uniform non-zero ask to the uniform distribution on the set of (H); and rejection, by Lemma “Rejection in key generation” (§3.2), costs a negligible term under Assumptions 2.12, 2.22 and 2.8. A dummy spend uses such a key (Definition “Dummy notes”, §4.4).

Remark 11.12 (Consensus-branch binding). The consensus branch identifier enters SigHash twice, in the root personalisation and in the header digest, and a consensus rule requires `nConsensusBranchId` to equal the identifier of the consensus branch under which the transaction is validated (protocol specification, §“Transaction Consensus Rules”; ZIP 244, “txid_digest” and “T.1: header_digest”). Under any other branch a signed transaction T either fails that rule or carries another `nConsensusBranchId`, hence other effecting data, and Proposition 11.10 excludes valid signatures on such a transaction that keeps an Action of T , except with negligible probability. A signed transaction is therefore valid under one consensus branch only and cannot be carried across the activation of a network upgrade (ZIP 259, “Backward compatibility”). That the anchors and the proofs are authorising data does not affect this binding, since the header is effecting data.

11.4 Chain state and pool rules

The Orchard pool and the Ironwood pool each have their own chain state: a note commitment tree (“The Merkle hash and the tree”, §5.1) with its anchors (“Anchors”, §5.2), a nullifier set (“Nullifier sets”, §6.2) and a chain value pool balance, defined below. An Action is checked only against the state of its own pool (ZIP 229, Terminology and Rationale, “Separate state”; protocol specification, §“Nullifier Sets” and §“Note Commitment Trees”).

Definition 11.13 (Chain value pool balances). The *Ironwood chain value pool balance* of a block chain is the negation of the sum of the `valueBalanceIronwood` fields of all its transactions, and the *Orchard chain value pool balance* the negation of the sum of their `valueBalanceOrchard` fields, an absent field counting as 0. The *total issued supply* is the sum of the balances of all chain value pools of the protocol specification, §“Chain Value Pool Balances”, the transparent pool and the other shielded pools included, computed without overflow (ZIP 209, Terminology and “Specification”).

The consensus rules on the balances are: a block that would make the chain value pool balance of any pool negative is invalid, and a block that would make the total issued supply exceed `MAX_MONEY` (§11.2) is invalid (ZIP 209, “Specification”; protocol specification, §“Chain Value Pool Balances”).

Remark 11.14 (Outflow bound). Fix a pool P and write each of its balancing values as $v = v^+ - v^-$ with $v^+ := \max(v, 0)$ and $v^- := \max(-v, 0)$. The chain value pool balance of P is $\sum v^- - \sum v^+$, summed over the transactions of the chain, so the rule that it is non-negative at every block states that at every block the net value that has left P through positive balancing values, $\sum v^+$, is at most the net value that entered it through negative ones, $\sum v^-$. Value created inside P by a failure of soundness can therefore leave P only up to the value recorded as having entered it (ZIP 209, “Specification”).

The following Orchard-pool rules apply to every transaction, whatever its version (ZIP 258, “Consensus rules from NU6.3 activation”; ZIP 229, “Consensus Rules”).

- (i) Cross-address transfers are disabled for every Orchard-pool Action: bits 2 to 7 of `flagsOrchard` are 0, so every Orchard-pool Action has `enableCrossAddress` = 0 and primary input `disableCrossAddress` = 1 (“Bundle flags”, §9.1), and the verifying key that consensus selects for every Action enforces condition A10 of Definition 9.2. The meaning of A10 and its pool-level effect are those stated at A10 in “The Action statement” (§9.2).
- (ii) For every transaction, `valueBalanceOrchard` ≥ 0 : net value may leave the Orchard pool and never enter it, so the Orchard chain value pool balance never increases.

Of the two Orchard-protocol pools, therefore, only the Ironwood pool accepts net inflow: a *shielding transaction*, one whose transparent inputs fund a negative balancing value, can place that negative balancing value, among the Orchard-protocol pools, only in its Ironwood component (ZIP 258, same section; protocol specification, §“Shielded Pools and Notes”).

The coinbase transaction of “The transaction format” (§11.2) is subject to the following rules on the shielded path (protocol specification, §“Transaction Consensus Rules”; ZIP 229, “Consensus Rules”; ZIP 258, “Consensus rules from NU6.3 activation”).

- (iii) A coinbase transaction has no Orchard-pool Actions.
- (iv) The bit `enableSpends` is 0 in `flagsOrchard` of a coinbase transaction of version 5 or 6 and in `flagsIronwood` of a coinbase transaction of version 6. By condition A8 of Definition 9.2, under Assumption 9.11, every Action of a coinbase transaction has, except with negligible

probability, an extracted witness with $v^{\text{old}} = 0$: a coinbase transaction consumes no note of non-zero value. The rule concerns coinbase transactions only; every other transaction may spend Ironwood-pool notes.

- (v) Every Ironwood-pool output of a coinbase transaction must decrypt to a note plaintext by the recovery with ovk of “The outgoing ciphertext” (§10.3) under the all-zero outgoing viewing key, the string of 32 zero bytes, so its plaintext is public and its lead byte is $0x03$, the only lead byte that recovery accepts (ZIP 213, “Specification”; ZIP 258, “Changes to the Protocol Specification”).

Rule (v) is the only check by consensus of the lead byte of an Orchard-protocol output. Every Ironwood-pool output uses the note format of lead byte $0x03$ (“The note seed”, §4.3), and the Action statement does not check the derivation of rcm (Definition 9.2, Remark “What the statement does not check”); for every other output the lead byte is encrypted and the recipient enforces it, decryption returning $\perp$ when the byte is not allowed for the pool (“Trial decryption and note acceptance”, §10.4; protocol specification, §“Decryption using an Incoming Viewing Key (Sapling and Orchard)”).

The rules of ZIP 218, “Shielded pool action limits”, bound per block the number of Orchard-pool Actions and a shielded budget that counts them together with the descriptions of the other shielded protocols; they do not mention Ironwood-pool Actions, which a version-6 transaction counts in the separate field nActionsIronwood (ZIP 229, “Transaction Format”). These are block-level rules, outside the Action statement.

11.5 Construction of a transaction

The following procedure extends the single spend of the *Crypto Guide*’s Example “An Orchard shielded spend” (§“A shielded spend, end to end”) to a bundle with signatures and balance. Its steps follow the order of the preceding sections.

Construction 11.15 (Ironwood-pool bundle). The input is the sender’s keys; the Ironwood-pool notes to consume, with their leaf positions; each recipient’s address, which its holder derives from an incoming viewing key (“Diversified addresses”, §3.3) and hands to the sender; the amounts; and the fee. The sender performs the following steps.

- (1) *Pairing*. For s consumed and o created notes it forms $n \geq \max(s, o)$ Actions (§11.1). A dummy consumed note comes from a fresh uniformly random spending key, with value 0, a uniform rseed , $\rho = \text{Extract}_p$ of a uniform Pallas point and an arbitrary authentication path; a dummy created note has value 0 and a random address (Definition “Dummy notes”, §4.4).
- (2) *Anchor*. It takes as rt the root of the Ironwood tree’s final treestate at a recent block, at the depth that “Anchors” (§5.2) recommends, and computes the authentication path of each real consumed note from public data (Lemma 5.15).
- (3) *Created notes*. For each Action it computes nf^{old} of the consumed note by Definition 6.3, sets $\rho^{\text{new}} := \text{nf}^{\text{old}}$ (“Nullifier chaining”, §6.3), draws a fresh uniform 32-byte rseed , derives esk , ψ^{new} and rcm^{new} under lead byte $0x03$ as in “The note seed” (§4.3), redrawing

rseed when $\text{esk} = 0$ or $\text{cmx} = \perp$, and computes cm^{new} by Definition 4.4 and $\text{cmx} := \text{Extract}_p(\text{cm}^{\text{new}})$.

- (4) *Value commitments.* For each Action it draws rcv uniformly from $\mathbb{F}_{p_{\text{Vesta}}}$ and sets $\text{cv}^{\text{net}} := [v^{\text{old}} - v^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}] R^{\text{Orchard}}$ (Definition 8.2).
- (5) *Randomised keys.* For each Action it draws α by GenRandom and sets $\text{rsk} := \text{ask} + \alpha$ and $\text{rk} := \text{ak}^P + [\alpha] G^{\text{SpendAuth}}$ (“Randomised validating keys”, §7.2), with ask and ak^P of the consumed note’s spending key, the dummy key for a dummy spend.
- (6) *Flags and balance.* It sets $\text{enableSpend} = \text{enableOutput} = 1$, and $\text{enableCrossAddress} = 1$ whenever some Action’s created note pays an expanded receiver other than that of its consumed note (condition A10 of Definition 9.2); it sets $v^{\text{balance}} := \sum (v^{\text{old}} - v^{\text{new}})$ over the Actions, and chooses the created values so that the fee is the value left in the transparent transaction value pool (“The balancing value”, §8.2).
- (7) *Proof.* It computes one Halo 2 proof over the n instances

$$(\text{rt}, \text{cv}^{\text{net}}, \text{nf}^{\text{old}}, \text{rk}, \text{cmx}, \text{enableSpend}, \text{enableOutput}, 1 - \text{enableCrossAddress})$$

with the witnesses of Definition 9.2 (“The Action circuit and the Halo 2 proof”, §9.3). By Assumption 9.11 a prover of an accepted proof knows, for each Action, a witness satisfying A1 to A10; under Assumption 9.14 the proof reveals nothing beyond the instances.

- (8) *Encryption.* For each Action it computes $\text{epk} := [\text{esk}] g_d$, C_{enc} and C_{out} as in “Encryption to the recipient” (§10.2) and “The outgoing ciphertext” (§10.3).
- (9) *Header.* It sets the version to 6, $\text{nConsensusBranchId}$ to the identifier of the consensus branch in force, and nExpiryHeight (§11.2).
- (10) *Digest.* It computes SigHash (“Transaction digests and signatures”, §11.3).
- (11) *Spend authorisation.* For each Action it computes the spend-authorisation signature under rsk on SigHash.
- (12) *Binding signature.* It sets $\text{bsk} := \sum \text{rcv} \bmod p_{\text{Vesta}}$ and $\text{bvk} := \sum \text{cv}^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$, checks $\text{bvk} = [\text{bsk}] R^{\text{Orchard}}$, which the protocol specification recommends to the signer, and computes the binding signature under bsk on SigHash (“The binding signature”, §8.3).

(Protocol specification, §“Sending Notes (Orchard)”, §“Dummy Notes (Orchard)”, §“Spend Authorization Signature (Sapling and Orchard)” and §“Balance and Binding Signature (Orchard)”.)

Steps (10) to (12) depend on neither rt nor the proof, which are authorising data, so signing can precede proving (ZIP 229, Motivation).

The sender sets $n\text{ExpiryHeight}$. ZIP 218, “Default expiry delta”, recommends to node implementations that create transactions a default expiry height 120 blocks above the current height (ZIP 203, “Specification”), which at the 25-second target spacing stated in “Anchors” (§5.2) is $120 \times 25 = 3000$ seconds, 50 minutes. The consensus rule is the expiry rule of “The transaction format” (§11.2).

Example 11.16 (A two-Action payment). The example is numeric only in amounts, fees, value differences, the balancing value, sizes and counts, each the output of a script; every curve point, field element, hash, ciphertext and signature is named and its derivation shown, not evaluated. The subscripts S, R and D mark the sender’s key, the recipient’s key and the dummy spending key.

Amounts. The sender holds one Ironwood-pool note n_1^{old} of 7 ZEC, 700 000 000 zatoshi, and pays 5 ZEC, 500 000 000 zatoshi, to a recipient’s address addr_R . One real consumed note and two created notes need $\max(1, 2) = 2$ Actions, hence two logical actions, and the conventional fee of “The balancing value” (§8.2) is $5000 \cdot \max(2, 2) = 10\,000$ zatoshi. The change is $700\,000\,000 - 500\,000\,000 - 10\,000 = 199\,990\,000$ zatoshi. The following table pairs the notes into Actions.

	Action 1	Action 2
consumed note	n_1^{old} , the sender’s	n_2^{old} , a dummy
spending key	the sender’s: $\text{ask}_S, \text{ak}_S^P, \text{nk}_S$	the dummy key: $\text{ask}_D, \text{ak}_D^P, \text{nk}_D$
v_i^{old}	700 000 000	0
created note	n_1^{new} to addr_R	n_2^{new} , the change, to addr_S
v_i^{new}	500 000 000	199 990 000
$v_i^{\text{old}} - v_i^{\text{new}}$	200 000 000	−199 990 000

The net values sum to $\text{valueBalancelronwood} = 200\,000\,000 + (-199\,990\,000) = 10\,000$ zatoshi, the fee. The transaction has no transparent inputs or outputs, so the fee is the whole value remaining in its transparent transaction value pool.

Public fields. The recipient derives $\text{addr}_R = (d_R, \text{pk}_{d,R})$ from its incoming viewing key (“Diversified addresses”, §3.3) and gives it to the sender; addr_S is an address of the sender’s own key. Write $(g_{d,1}, \text{pk}_{d,1})$ and $(g_{d,2}, \text{pk}_{d,2})$ for the expanded receivers of addr_R and addr_S , and $\text{nk}_1 := \text{nk}_S$, $\text{nk}_2 := \text{nk}_D$. For $i = 1, 2$ the sender computes, from the consumed note’s fields $(\rho_i^{\text{old}}, \psi_i^{\text{old}}, \text{cm}_i^{\text{old}})$,

$$\text{nf}_i^{\text{old}} := \text{DeriveNullifier}_{\text{nk}_i}(\rho_i^{\text{old}}, \psi_i^{\text{old}}, \text{cm}_i^{\text{old}}) \quad (\text{Definition 6.3}),$$

$$\rho_i^{\text{new}} := \text{nf}_i^{\text{old}} \quad (\S 6.3),$$

draws a uniform rseed_i , derives $\text{esk}_i, \psi_i^{\text{new}}$ and $\text{rcm}_i^{\text{new}}$ from rseed_i under lead byte $0x03$ (“The note seed”, §4.3), and sets

$$\begin{aligned} \text{cm}_i^{\text{new}} &:= \text{NoteCommit}_{\text{rcm}_i^{\text{new}}}(\text{g}_{d,i}^* \parallel \text{pk}_{d,i}^* \parallel \text{LE}_{64}(v_i^{\text{new}}) \parallel \text{LE}_{255}(\rho_i^{\text{new}}) \parallel \text{LE}_{255}(\psi_i^{\text{new}})), \\ \text{cmx}_i &:= \text{Extract}_P(\text{cm}_i^{\text{new}}) \end{aligned}$$

(Definition 4.4). With rcv_i uniform on $\mathbb{F}_{P_{\text{Vesta}}}$ and α_i drawn by GenRandom ,

$$\begin{aligned} \text{cv}_1^{\text{net}} &:= [200\,000\,000] V^{\text{Orchard}} + [\text{rcv}_1] R^{\text{Orchard}}, & \text{rk}_1 &:= \text{ak}_S^P + [\alpha_1] G^{\text{SpendAuth}}, \\ \text{cv}_2^{\text{net}} &:= [-199\,990\,000] V^{\text{Orchard}} + [\text{rcv}_2] R^{\text{Orchard}}, & \text{rk}_2 &:= \text{ak}_D^P + [\alpha_2] G^{\text{SpendAuth}}, \end{aligned}$$

the negative difference read in $\mathbb{F}_{p_{\text{Vesta}}}$ (Definition 8.2; “Randomised validating keys”, §7.2). The anchor rt is the anchor of a recent Ironwood-pool block, with the authentication path of $\text{Extract}_p(\text{cm}_1^{\text{old}})$; the path of the dummy is arbitrary, since A3 exempts $v_2^{\text{old}} = 0$. The flags are $\text{enableSpend} = \text{enableOutput} = \text{enableCrossAddress} = 1$, because both created notes pay expanded receivers other than those of their Actions’ consumed notes (A10). One proof π covers the instances $(rt, \text{cv}_i^{\text{net}}, \text{nf}_i^{\text{old}}, \text{rk}_i, \text{cmx}_i, 1, 1, 0)$ for $i = 1, 2$. Finally $\text{epk}_i := [\text{esk}_i]_{\text{g}_{d,i}, C_{\text{enc},i}}$ encrypts the plaintext of n_i^{new} to its receiver, and $C_{\text{out},i}$ is formed under the sender’s ovk (“Encryption to the recipient”, §10.2; “The outgoing ciphertext”, §10.3).

Signatures, balance and size. The transaction has no transparent inputs, so SigHash equals its transaction identifier (“Transaction digests and signatures”, §11.3). The spend-authorisation signatures are σ_1 under $\text{rsk}_1 = \text{ask}_S + \alpha_1$ and σ_2 under $\text{rsk}_2 = \text{ask}_D + \alpha_2$, both on SigHash . The binding signing key is $\text{bsk} := \text{rcv}_1 + \text{rcv}_2 \bmod p_{\text{Vesta}}$, and

$$\begin{aligned} \text{bvk} &= \text{cv}_1^{\text{net}} + \text{cv}_2^{\text{net}} - [10\,000] V^{\text{Orchard}} \\ &= [200\,000\,000 - 199\,990\,000 - 10\,000] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}} = [\text{bsk}] R^{\text{Orchard}}, \end{aligned}$$

$\text{bindingSigIronwood}$ is the binding signature under bsk on SigHash . By Table 7 the Ironwood component has

$$1 + 2 \cdot 820 + 1 + 8 + 32 + 3 + (2720 + 2272 \cdot 2) + 2 \cdot 64 + 64 = 9141$$

bytes: the count, the Action descriptions, the flags, the balancing value, the anchor, the proof length, the proof of 7264 bytes, the spend-authorisation signatures and the binding signature.

The two views. The verifier receives only the public data: for each i the Action description $(\text{cv}_i^{\text{net}}, \text{nf}_i^{\text{old}}, \text{rk}_i, \text{cmx}_i, \text{epk}_i, C_{\text{enc},i}, C_{\text{out},i})$ and σ_i , and rt , the flags byte with bits 0, 1 and 2 set, $\text{valueBalanceIronwood} = 10\,000$, π and $\text{bindingSigIronwood}$. It runs the procedure of “Verification of a transaction” (§11.6). What these data reveal is the subject of Theorem 12.12 in “Privacy” (§12.4). The recipient trial-decrypts every output with its incoming viewing key. On output 1 it recovers the plaintext $(\text{txid}, d_R, 500\,000\,000, \text{rseed}_1, \text{memo})$, re-derives the note with $\rho = \text{nf}_1^{\text{old}}$ of the same Action, checks epk_1 and cmx_1 , and records the leaf position of cmx_1 (“Trial decryption and note acceptance”, §10.4); the sender recovers the change note of output 2 in the same way. Either note is later consumed by the procedure of this subsection.

11.6 Verification of a transaction

Each step of the following procedure is tagged with the requirements whose soundness part it serves and with the conditions of Definition 9.2 that it completes, in the pairing of the requirement map, Table 5.

Construction 11.17 (Verification). Given the chain state of each pool, a transaction T in a block is accepted, as far as its Orchard-protocol components are concerned, if and only if the following checks pass.

- (1) *Parsing.* The version of T is 5 or 6, with the version group identifier of its version, and $\text{nConsensusBranchId}$ equals the identifier of the consensus branch in force. Every field

of every Action description is the canonical encoding of its type (Table 6): cv^{net} and rk are points, epk is a non-identity point, and nf and cmx are integers below p_{Pallas} ; $rk \neq \mathcal{O}$; every signature encoding is canonical as RedPallas validation requires (§7.1); and each component satisfies the rules of “The transaction format” (§11.2) on its count, its reserved flag bits, its flags, the range of its balancing value and the length of its proof. The transaction T has a transparent input, a Spend description in its Sapling component or an Orchard-protocol bundle with $\text{enableSpend} = 1$, and it has a transparent output, an Output description in its Sapling component or an Orchard-protocol bundle with $\text{enableOutput} = 1$, where Spend and Output descriptions are the Sapling component’s descriptions of a consumed and of a created note.

- (2) *Proof.* For each pool with Actions, the aggregate proof verifies under the post-NU6.3 verifying key of “The Action circuit and the Halo 2 proof” (§9.3) on the per-Action primary inputs

$$(rt, cv^{\text{net}}, nf, rk, cmx, \text{enableSpend}, \text{enableOutput}, 1 - \text{enableCrossAddress}),$$

with rt and the flags read from the bundle. [R1 to R5, through A1 to A10.]

- (3) *Anchor rule.* Each bundle’s rt is the root of its pool’s final treestate at an earlier block of the chain (“Anchors”, §5.2). [R2, with A3.]
- (4) *Nullifier rule.* No nullifier repeats within T or across the transactions of the chain; the nullifiers of each pool are checked against that pool’s nullifier set (“Nullifier sets”, §6.2). [R3, with A5.]
- (5) *Spend authorisation.* Each Action’s spend-authorisation signature is valid under its rk on SigHash (“Randomised validating keys”, §7.2). [R5, with A6 and A7.]
- (6) *Balance.* For each pool with Actions, the verifier computes $\text{bvk} := \sum cv^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$ over the pool’s Actions and checks the binding signature under bvk on SigHash (“The binding signature”, §8.3; *Crypto Guide*, §“Binding signatures as a proof of knowledge of a discrete logarithm”); and the fee, the value remaining in the transparent transaction value pool, is non-negative (“The balancing value”, §8.2), a coinbase transaction satisfying instead the coinbase rule of the protocol specification, §“Transaction Consensus Rules”, which balances its outputs against the block subsidy and the fees of the block’s other transactions. [R4, with A4.]
- (7) *Pool rules.* The rules of “Chain state and pool rules” (§11.4), the expiry rule of “The transaction format” (§11.2) and the capacity rule of each pool’s note commitment tree (“The Merkle hash and the tree”, §5.1) hold.

(Protocol specification, §“Transaction Consensus Rules”, §“Action Descriptions”, §“Action Description Encoding and Consensus”, §“RedDSA, RedJubjub, and RedPallas”, §“Action Transfers and their Descriptions”, §“Nullifier Sets”, §“Note Commitment Trees”, §“Balance and Binding Signature (Orchard)” and §“Transactions and Treestates”; ZIP 229, “Consensus

Rules”; ZIP 2003, “Specification”; ZIP 259, Abstract.) These are the consensus checks on the Orchard-protocol components of T ; the remaining rules of §“Transaction Consensus Rules” concern its other components.

Remark 11.18 (Division of the checks). Under Assumption 9.11 the proof yields, for each Action, a witness that satisfies Definition 9.2 for its primary input. Whether rt is a root that the chain recorded and whether nf is new are facts about the chain’s history, which no proof over one Action’s data attests; the verifier checks them against its state, in steps (3) and (4). Beyond the parsing of step (1), the fee rule of step (6) and the rules of step (7), validity is thus the conjunction of one proof per bundle, two kinds of signature and these two checks against chain state; apart from the coinbase rule (v) of §11.4, no step decrypts or inspects a note.

Construction 11.19 (State update). On accepting a block, for each pool: every nullifier of the pool’s Actions enters the pool’s nullifier set, so that a later Action that publishes the same nullifier fails step (4); every cmx is appended to the pool’s note commitment tree, in the order of the transactions and, within a transaction, of the Actions, so that the note’s holder can later prove membership against an anchor whose tree contains it; and the chain value pool balance of the pool decreases by each transaction’s balancing value (protocol specification, §“Nullifier Sets”, §“Transactions and Treestates” and §“Chain Value Pool Balances”).

The verifier holds no viewing key and decrypts no Orchard-protocol ciphertext, except for the decryption of Ironwood-pool coinbase outputs under the all-zero outgoing viewing key that “Chain state and pool rules” (§11.4) requires, whose plaintexts are public by that rule (protocol specification, §“Transaction Consensus Rules”; ZIP 213, “Specification”). Its input is the transaction’s public data and the chain state; what those data reveal about the private inputs is the subject of Theorem 12.12 in “Privacy” (§12.4).

12 Security

This section composes the propositions of the preceding sections with the two assumptions on the Action proof, Assumptions 9.11 and 9.14. Theorem 12.4 states conservation per pool; its proof uses condition A4, the binding signature and Assumption 9.11. Lemma 12.5 and Theorem 12.6 exclude a second consumption of a note, Corollary 12.7 the Faerie Gold attack and Theorem 12.9 a consumption without the spend authorising key; Theorem 12.12 states what the public data of a bundle hide. Theorem 12.14 collects, under one list of assumptions, membership soundness, Theorems 12.6, 12.4 and 12.9 with Proposition 11.10, and Theorem 12.12.

Efficient adversaries and negligible functions are those of the *Crypto Guide*, §“Adversaries and the security parameter”, and computational indistinguishability is that of its Definition “Perfect, statistical, and computational indistinguishability” (§“Distribution ensembles and indistinguishability”). An adversary interacts with the honest parties, namely key holders, senders and signers, and with the random oracles of Assumptions 2.8, 7.3, 9.11, 9.14 and 10.4, which are independent of one another; every reduction simulates the honest parties and the random oracles. Every result of the section holds except with negligible probability, and every result that uses a property

of key generation is claimed for keys generated with `use_ksk = false` (“The spending key and the spend-side secrets”, §3.1).

Definition 12.1 (Pools, accepted Actions and witnesses).

- (a) A *pool* is the Orchard pool or the Ironwood pool. A statement made for each pool uses that pool’s note commitment tree, anchors, nullifier set and bundle, its balancing value, `valueBalanceOrchard` or `valueBalanceIronwood`, and its binding signature, `bindingSigOrchard` or `bindingSigIronwood`.
- (b) An adversary outputs a *block chain*: a sequence of blocks each of whose transactions passes the verification of “Verification of a transaction” (§11.6) against the chain state that the state update of that subsection produces from the preceding transactions. An Action, or a bundle, of a transaction of that chain is *accepted*.
- (c) The *witness* of an accepted Action is the auxiliary input that the extractor of Assumption 9.11 outputs for it, applied to the Action’s bundle as stated below. The Action *consumes* the note $(g_d^{\text{old}}, pk_d^{\text{old}}, v^{\text{old}}, \rho^{\text{old}}, \psi^{\text{old}}, rcm^{\text{old}})$ of its witness and *creates* the note $(g_d^{\text{new}}, pk_d^{\text{new}}, v^{\text{new}}, \rho^{\text{new}}, \psi^{\text{new}}, rcm^{\text{new}})$, with $\rho^{\text{new}} = nf^{\text{old}}$ (Definition 9.2); the values v^{old} and v^{new} are read from the witness.
- (d) In this section a note is given by its expanded receiver (g_d, pk_d) (Definition “Flag values and expanded receiver”, §9.1) in place of its address, and its trapdoor rcm acts through its residue modulo p_{Vesta} ; two notes are equal when these six components are. Its note commitment is $\text{NoteCommit}_{rcm}(M)$ and its extracted commitment is $\text{Extract}_p^\perp(\text{NoteCommit}_{rcm}(M))$, with M the message of Definition 4.4 formed from $(g_d, pk_d, v, \rho, \psi)$.
- (e) A *committed note* of a pool is a note whose extracted commitment is a leaf of that pool’s note commitment tree.

Assumption 9.11 is stated for one aggregate proof. For a block chain it is applied to each bundle, to the algorithm that runs the adversary, with every honest party and random oracle of the experiment, and outputs that bundle’s primary inputs and proof; the extractors of distinct bundles are run on the same execution. An oracle that a reduction does not simulate itself, such as the random oracle H of Assumption 7.3 or a signing oracle, receives each new query of a rewind run, and a query repeated on the same prefix of a run is answered as before. By the union bound over the polynomially many bundles, every witness satisfies Definition 9.2, except with negligible probability. By Remark “ $\perp$ -weakened conditions” (§9.2), conditions A1, A2, A3 and A7 then hold in their unweakened forms, without the $\perp$ case, except with negligible probability, under Assumptions 2.22 and 2.8. The results below include these events in their negligible terms.

Remark 12.2 (Binding for notes given by expanded receivers). Step 1 of the proof of Proposition 4.5(b) shows that the map $(g_d, pk_d, v, \rho, \psi) \mapsto M$ is injective, and Step 2 reads a note only through M and its trapdoor. For notes in the sense of (d) the two steps therefore show, under Assumptions 2.22 and 2.8: no efficient algorithm outputs, except with negligible probability, two

distinct notes with equal extracted commitments other than $\perp$. In particular a note that opens an extracted commitment is the only opening that an efficient algorithm finds. The proof of Proposition 6.8 likewise reads a note only through M , rcm , ρ and ψ , and applies to such notes.

12.1 Value conservation

Lemma 12.3 (Modular balance is integer balance). *Let $n < 2^{16}$; for $1 \leq i \leq n$ let v_i^{old} and v_i^{new} be integers in $\{0, \dots, 2^{64} - 1\}$; and let v^{balance} be an integer in $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$. If*

$$\sum_{i=1}^n (v_i^{\text{old}} - v_i^{\text{new}}) \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}},$$

then the two sides are equal as integers. The hypotheses hold for every accepted bundle of either pool: the bound on n and the range of the balancing value are consensus rules (“The transaction format”, §11.2; protocol specification, §“Transaction Consensus Rules”, for both pools), and v_i^{old} and v_i^{new} are 64-bit values of the auxiliary input of Definition 9.2.

Proof. Let $v^{\text{sum}} := \sum_i (v_i^{\text{old}} - v_i^{\text{new}}) - v^{\text{balance}}$. Each difference has absolute value at most $2^{64} - 1$, and $\text{MAX_MONEY} = 2.1 \cdot 10^{15}$ (protocol specification, §“Constants”), so

$$|v^{\text{sum}}| \leq (2^{16} - 1)(2^{64} - 1) + \text{MAX_MONEY} = 1\,208\,907\,374\,970\,555\,465\,089\,025 < 2^{80}.$$

The hypothesis makes v^{sum} a multiple of p_{Vesta} , and $p_{\text{Vesta}} > 2^{254}$ (§2.1); the only such multiple of absolute value below 2^{80} is 0. $\square$

The security argument of the protocol specification, §“Balance and Binding Signature (Orchard)”, bounds the balancing value by its signed 64-bit encoding instead, which places v^{sum} in

$$\{-1\,208\,916\,596\,242\,592\,319\,864\,832, \dots, 1\,208\,916\,596\,242\,592\,319\,864\,833\};$$

that bound is also below 2^{80} , so the conclusion holds without the range rule.

Theorem 12.4 (Balance). *Under Assumptions 9.11, 2.22, 2.8 and 7.3, for each pool, the Ironwood pool included, every accepted bundle of that pool output by an efficient adversary, with n Actions and balancing value v^{balance} , satisfies*

$$\sum_{i=1}^n (v_i^{\text{old}} - v_i^{\text{new}}) = v^{\text{balance}}$$

as integers, the values read from the Actions’ witnesses, except with negligible probability. The binding signature checked is that of the same pool, under the binding validating key bvk recomputed from that bundle’s net value commitments and balancing value; both pools use the bases V^{Orchard} and R^{Orchard} .

Proof. Fix a pool and an efficient adversary $\mathcal{A}$. The adversary outputs polynomially many bundles, and the union bound reduces the claim to the j -th bundle of the pool, for each j . Let $\mathcal{B}_j$ be the

algorithm that runs $\mathcal{A}$ with the extractors of Assumption 9.11 and outputs that bundle's net value commitments $cv_1^{\text{net}}, \dots, cv_n^{\text{net}}$, its balancing value, the pairs

$$(v_i^{\text{old}} - v_i^{\text{new}}, \text{rcv}_i \bmod p_{\text{Vesta}}) \quad (1 \leq i \leq n)$$

read from the witnesses, the digest SigHash of its transaction and the pool's binding signature. Algorithm $\mathcal{B}_j$ is efficient and forwards the queries of $\mathcal{A}$ to H .

(1) *Openings*. By condition A4 of Definition 9.2, $cv_i^{\text{net}} = [v_i^{\text{old}} - v_i^{\text{new}}] V^{\text{Orchard}} + [\text{rcv}_i] R^{\text{Orchard}}$, so each output pair is an opening of cv_i^{net} in the sense of “The binding signature” (§8.3), except with the negligible probability that a witness fails the definition.

(2) *Signature*. The bundle is accepted, so its binding signature is valid on SigHash under

$$\text{bvk} = \sum_{i=1}^n cv_i^{\text{net}} - [v^{\text{balance}}] V^{\text{Orchard}}$$

in the binding-signature instance of RedPallas, with base R^{Orchard} (step (6) of the verification of §11.6). This key belongs to the pool's own bundle: each pool's bvk sums only that pool's net value commitments and subtracts only its balancing value (§8.3).

(3) *Congruence*. Algorithm $\mathcal{B}_j$ is therefore an efficient party of the consequence of Proposition 8.10: it outputs the net value commitments and the balancing value of a bundle, openings of every commitment, and a binding signature under the resulting bvk on a message of its choice. The consequence is proved in two steps. The extraction of Proposition 7.12, in the random-oracle model for H (Assumption 7.3), yields b with $\text{bvk} = [b] R^{\text{Orchard}}$; if the openings did not balance modulo p_{Vesta} , part (ii) of Proposition 8.10 would turn the residual V^{Orchard} component into the discrete logarithm of V^{Orchard} to the base R^{Orchard} , which Assumptions 2.22 and 2.8 exclude. Hence $\sum_i (v_i^{\text{old}} - v_i^{\text{new}}) \equiv v^{\text{balance}} \pmod{p_{\text{Vesta}}}$, except with negligible probability.

(4) *Integers*. The bundle is accepted, so it has $n < 2^{16}$ Actions, its balancing value lies in the range $\{-\text{MAX_MONEY}, \dots, \text{MAX_MONEY}\}$, and the witnessed values lie in $\{0, \dots, 2^{64} - 1\}$. Lemma 12.3 turns the congruence into an equality of integers.

The argument uses no property of the pool beyond its own bundle and binding signature, and applies to the Ironwood pool as to the Orchard pool. $\square$

12.2 Double-spend resistance

Lemma 12.5 (One nullifier per note). *Under Assumptions 9.11, 2.22 and 2.8, except with negligible probability: whenever two accepted Actions, of either pool, have witnesses whose consumed notes have the same extracted commitment $\text{Extract}_P(\text{cm}^{\text{old}})$, the two witnesses consume the same note with the same nullifier deriving key nk , and the two Actions publish the same nullifier nf^{old} . The lemma uses neither v^{old} nor condition A3.*

Proof. Let w and w' be the two witnesses, primed components belonging to w' . Outside the negligible events of the paragraph after Definition “Pools, accepted Actions and witnesses”, both satisfy Definition 9.2 with A1 and A7 unweakened. Every step below is carried out by the efficient algorithm that runs the adversary with the extractors and searches the accepted Actions for such a pair.

(1) *The note.* By A1, $\text{cm}^{\text{old}} = \text{NoteCommit}_{\text{rcm}^{\text{old}}}(\mathcal{M}^{\text{old}}) \neq \perp$, and likewise for w' . The two consumed notes thus have equal extracted commitments other than $\perp$, and by Remark “Binding for notes given by expanded receivers” they are equal, except with negligible probability. Equal notes have equal messages and trapdoors modulo p_{Vesta} , hence $\text{cm}^{\text{old}} = \text{cm}'^{\text{old}}$.

(2) *The key.* By A7, $\text{pk}_d^{\text{old}} = [\text{ivk}] \text{g}_d^{\text{old}}$ with

$$\text{ivk} := \text{Commit}_{\text{r}_{\text{ivk}}}^{\text{ivk}}(\text{Extract}_{\text{p}}(\text{ak}^{\text{p}}), \text{nk}) \neq \perp,$$

and, the notes being equal, $\text{pk}_d^{\text{old}} = [\text{ivk}'] \text{g}_d^{\text{old}}$ likewise. Both values of $\text{Commit}^{\text{ivk}}$ are integers below $p_{\text{Pallas}} < p_{\text{Vesta}}$, read as scalars (§2.1), and g_d^{old} is a non-identity point of a group of prime order p_{Vesta} ; hence $\text{ivk} = \text{ivk}'$. By the binding of ivk to (ak, nk) (Proposition “Binding of ivk to (ak, nk) ”, §3.2), under Assumptions 2.22 and 2.8,

$$(\text{Extract}_{\text{p}}(\text{ak}^{\text{p}}), \text{nk}, \text{r}_{\text{ivk}} \bmod p_{\text{Vesta}}) = (\text{Extract}_{\text{p}}(\text{ak}'^{\text{p}}), \text{nk}', \text{r}_{\text{ivk}}' \bmod p_{\text{Vesta}}),$$

except with negligible probability; in particular $\text{nk} = \text{nk}'$.

(3) *The nullifier.* By A5, $\text{nf}^{\text{old}} = \text{DeriveNullifier}_{\text{nk}}(\rho^{\text{old}}, \psi^{\text{old}}, \text{cm}^{\text{old}})$ (Definition 6.3), a function of its four arguments, which are equal for w and w' by (1) and (2). The two Actions therefore publish the same nullifier. No step uses v^{old} or A3. $\square$

Theorem 12.6 (No double-spending). *Under Assumptions 9.11, 2.22 and 2.8, for each pool, except with negligible probability, no committed note of non-zero value is consumed by two accepted Actions of that pool. The guarantee is per pool: the two pools’ nullifiers are checked against separate sets.*

Proof. Let two distinct accepted Actions of one pool consume the same note. Their consumed notes have the same extracted commitment, so by Lemma 12.5, except with negligible probability, the two Actions publish the same nullifier. Both belong to transactions of the adversary’s block chain, and the nullifier rule of “Nullifier sets” (§6.2), step (4) of the verification of §11.6, rejects a nullifier of the pool that repeats within a transaction or across the transactions of the chain. The event therefore lies in the negligible event of the lemma. The argument uses neither the value nor the membership of the note and holds for every note; the case of a committed note of non-zero value is the one that moves value. The nullifier sets of the two pools are separate (§6.2), so the argument concerns two Actions of one pool only. $\square$

Corollary 12.7 (Faerie Gold resistance). *Under Assumptions 9.11, 2.22 and 2.8, except with negligible probability:*

- (a) *the notes created by distinct accepted Actions of one pool have pairwise distinct elements ρ , and are therefore pairwise distinct;*
- (b) *two distinct notes have distinct nullifiers, whatever their nullifier deriving keys: no efficient algorithm outputs two distinct notes, with note commitments other than $\perp$, and two keys nk , nk' , equal or not, under which the two notes have equal nullifiers.*

Hence the Faerie Gold attack, as defined in “Nullifier chaining” (§6.3), fails within a pool: (a) excludes two Actions creating the same note, which would carry one nullifier, and (b) excludes distinct notes sharing a nullifier, so that consuming one note accepted by a recipient never blocks another. The uniqueness of ρ is pool-local: the pools’ nullifier sets are separate, so one value of ρ may occur once in each pool.

Proof. (a) The note that the witness of an accepted Action creates has $\rho^{\text{new}} = \text{nf}^{\text{old}}$ of that Action: the Action statement fixes $\rho^{\text{new}} := \text{nf}^{\text{old}}$ and binds the created note to the published cmx by A2 (Definition 9.2), so the chaining rule of §6.3 is enforced through Assumption 9.11. Distinct accepted Actions of one pool publish distinct nullifiers, by the nullifier rule (§6.2). Their created notes therefore have distinct ρ , and notes with a distinct component are distinct. The created note is the only opening of cmx that an efficient algorithm finds (Remark “Binding for notes given by expanded receivers”); a recipient that accepts a note from the Action obtains that note (Proposition 10.15(c)) and checks $\rho = \text{nf}^{\text{old}}$ itself (Proposition 10.15(a)).

(b) Two distinct notes have distinct note commitments, except with negligible probability, by the same remark; Proposition 6.8, which admits every pair of nullifier deriving keys, then excludes equal nullifiers.

Faerie Gold. Let a recipient accept a note n' from an accepted Action of a pool, and let nf' be its nullifier under the recipient’s nk. By (a) no other accepted Action of the pool creates n' , so the notes that the recipient accepts from distinct Actions are distinct. An accepted Action of the pool whose witness consumes a note $n \neq n'$ publishes, by A5, the nullifier of n under the witnessed key, which differs from nf' by (b). The value nf' therefore enters the pool’s nullifier set only through an Action that consumes n' itself, and the consumption of another note does not make n' unspendable.

Pool locality. The argument of (a) uses the nullifier set of one pool. An Orchard-pool Action and an Ironwood-pool Action may publish the same nullifier and create notes with the same ρ ; a consumption in one pool inserts its nullifier into that pool’s set only, and so blocks no note of the other pool (Remark “Pool locality”, §6.2). $\square$

12.3 Authorisation of spends

Definition 12.8 (Spend-authority experiment). A key holder generates a spending key sk with $\text{use_qsk} = \text{false}$ and derives $(\text{ask}, \text{nk}, \text{r}_{\text{vk}})$, ak^{P} and $\text{ak} = \text{Extract}_{\text{p}}(\text{ak}^{\text{P}})$ as in “The spending key and the spend-side secrets” (§3.1), and ivk as in “Viewing keys” (§3.2). The adversary receives the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{vk}})$ but not ask , sees every public field, may create notes to any address of the key, and may obtain spend-authorisation signatures: on a query (α, m) , with $\alpha \in \mathbb{F}_{p_{\text{Vesta}}}$ and m a byte string, it receives a RedPallas signature on m under $\text{rsk} = \text{ask} + \alpha$ in the spend-authorisation instance, the signing model of Proposition 7.12. A *note of the key* is a note whose expanded receiver satisfies $\text{pk}_d = [\text{ivk}] g_d$ for the key’s ivk . The adversary *wins* if it outputs a block chain with an accepted Action whose witness consumes a note of the key, of any value, a dummy of value zero included, and whose spend-authorisation signature is valid under its rk on a signature digest m , where no query (α, m) had $\text{rk} = \text{ak}^{\text{P}} + [\alpha] G^{\text{SpendAuth}}$.

Theorem 12.9 (Spend authority). *For keys generated with $\text{use_qsk} = \text{false}$, and under Assumptions 9.11, 2.22, 2.8 (through the binding of $\text{Commit}^{\text{ivk}}$), 7.3 and 2.12, no efficient adversary wins the spend-authority experiment, except with negligible probability.*

Proof. Let an efficient adversary $\mathcal{A}$ win with probability ϵ , and write $G := G^{\text{SpendAuth}}$.

(1) *Idealised key.* By Lemma “Rejection in key generation” (§3.2), the key is replaced by a single draw from a uniform sk without rejection, at a cost negligible under Assumptions 2.12, 2.22 and 2.8. By Lemma 2.15 (Assumption 2.12), its triple $(\text{ask}, \text{nk}, \text{r}_{\text{ivk}})$, before the sign normalisation, is then replaced by independent uniform elements of $\mathbb{F}_{p_{\text{Vesta}}}$, $\mathbb{F}_{p_{\text{Pallas}}}$ and $\mathbb{F}_{p_{\text{Vesta}}}$, at a further negligible cost. Both hops apply because the event that $\mathcal{A}$ wins is efficiently decidable from the triple: an algorithm derives the key’s components, runs $\mathcal{A}$, answers its signing queries with ask , runs the extractors on its output and checks the winning condition. After this step nk and r_{ivk} are independent of ask , and, when $\text{ask} \neq 0$, the sign normalisation makes ak^{P} a uniform non-identity point of even y -coordinate (Remark “Hypothesis (H) for honest keys”, §11.3). This independence lets the reduction below simulate the full viewing key from ak^{P} alone.

(2) *Reduction.* An algorithm $\mathcal{B}$ plays the adversary of Proposition 7.12. It receives $Y = [x] G$ for x uniform on $\mathbb{F}_{p_{\text{Vesta}}}$, with access to H and to the signing oracle. If $Y = \mathcal{O}$ or Y has odd y -coordinate, it stops. Otherwise, an event of probability $(p_{\text{Vesta}} - 1)/(2p_{\text{Vesta}})$, since exactly half of the $p_{\text{Vesta}} - 1$ non-identity points have even y -coordinate (§2.1, on the star encoding), the point Y is distributed as ak^{P} in step (1) conditioned on $\text{ask} \neq 0$. Algorithm $\mathcal{B}$ sets $\text{ak}^{\text{P}} := Y$ and $\text{ak} := \text{Extract}_{\text{P}}(Y)$, draws nk and r_{ivk} uniformly, computes ivk , and gives $\mathcal{A}$ the full viewing key $(\text{ak}, \text{nk}, \text{r}_{\text{ivk}})$. It answers each query (α, m) of $\mathcal{A}$ with the oracle’s signature under $x + \alpha$, forwards the queries of $\mathcal{A}$ to H , and simulates every other honest party and random oracle. When $\mathcal{A}$ outputs a block chain, $\mathcal{B}$ runs the extractors of Assumption 9.11, as stated after Definition “Pools, accepted Actions and witnesses”, and searches for an Action that meets the winning condition, all of whose parts it can check; failing one, it stops. For the Action found, with witness w , it determines $\theta \in \{1, -1\}$ with $\text{ak}_w^{\text{P}} = [\theta] Y$, stopping if there is none, and outputs the Action’s triple (rk, m, σ) together with the witnessed randomiser α_w and θ .

(3) *The witnessed key.* Except with negligible probability, $\text{ak}_w^{\text{P}} \in \{Y, -Y\}$. By A7, in its unweakened form, $\text{pk}_d^{\text{old}} = [\text{ivk}_w] \text{g}_d^{\text{old}}$ with $\text{ivk}_w := \text{Commit}_{\text{r}_{\text{ivk}_w}}^{\text{ivk}}(\text{Extract}_{\text{P}}(\text{ak}_w^{\text{P}}), \text{nk}_w) \neq \perp$, and the consumed note is a note of the key, so $\text{pk}_d^{\text{old}} = [\text{ivk}] \text{g}_d^{\text{old}}$. As in step (2) of the proof of Lemma 12.5, $\text{ivk}_w = \text{ivk}$, and the binding of ivk to (ak, nk) gives $\text{Extract}_{\text{P}}(\text{ak}_w^{\text{P}}) = \text{ak} = \text{Extract}_{\text{P}}(Y)$, under Assumptions 2.22 and 2.8. By Lemma 2.4, $\text{ak}_w^{\text{P}} \in \{Y, -Y\}$. The statement does not constrain the sign of the witnessed point (Remark “Sign of ak^{P} ”, §9.2), and the proof treats both signs. By A6,

$$\text{rk} = \text{ak}_w^{\text{P}} + [\alpha_w] G = [\theta] Y + [\alpha_w] G.$$

(4) *Forgery.* An answer of the signing oracle to a query (α, m') has validating key $Y + [\alpha] G$ and message m' . The winning condition excludes a query (α, m) with $\text{rk} = Y + [\alpha] G$, so the output triple is not the triple of an oracle answer: it is a forgery in the sense of Proposition 7.12. Algorithm $\mathcal{B}$ is efficient, and forges with probability at least

$$\frac{p_{\text{Vesta}} - 1}{2p_{\text{Vesta}}} (\epsilon - \nu)$$

for a negligible ν , the costs of step (1) and of the events excluded in step (3).

(5) *Discrete logarithm*. By Proposition 7.12(i), an algorithm $\mathcal{D}$ runs $\mathcal{B}$ twice on shared coins and outputs, with at least the probability that part states for the forging probability of $\mathcal{B}$, a scalar rsk^* with $\text{rk} = [\text{rsk}^*] G$ for the rk of the first run. It outputs $x' := \theta (\text{rsk}^* - \alpha_w)$, with θ and α_w of the first run. By step (3), $[\theta] Y = [\text{rsk}^* - \alpha_w] G$, so $x' = x$. For $\theta = 1$ this is part (ii) of the proposition with $\alpha^* = \alpha_w$; for $\theta = -1$ the same relation gives $-x = \text{rsk}^* - \alpha_w$, so a witnessed point $-\text{ak}^P$ is no easier than ak^P : either sign yields ask , the discrete logarithm of the honest ak^P . By Assumption 2.22 the probability that $\mathcal{D}$ outputs x is negligible, hence so is the forging probability of $\mathcal{B}$, and with it ϵ ; the loss factor $2p_{\text{Vesta}} / (p_{\text{Vesta}} - 1)$ is about 2. $\square$

Remark 12.10 (Authority without linkability). An Action publishes rk and a signature under $\text{rsk} = \text{ask} + \alpha$, never ak^P or ak . For α drawn by GenRandom afresh per Action, Proposition 7.10 (Assumption 7.3) makes the keys rk of n Actions independent of their spend validating keys up to statistical distance $n (\delta_{512} + (q_b + n) \cdot 2^{-640})$, with $\delta_{512} < 2^{-257}$, and makes (rsk, rk) distributed as a fresh key pair; the signature is a function of rsk , the digest and fresh randomness. The keys rk of distinct spends of one key are therefore unlinkable, while Theorem 12.9 shows that an accepted Action with a valid signature under its rk still demonstrates knowledge of ask : from an adversary that produces one for an honestly generated key without its ask , the reduction of its proof computes ask .

12.4 Privacy

Definition 12.11 (Public leakage and privacy preconditions). The *public leakage* of a bundle is its pool, its number of Actions, its anchor, its flags and its balancing value; the rest of the transaction, namely its transparent components, the other pool's bundle and its header with the expiry height, is public as well. The *private inputs* of a bundle are, per Action, its witness (Definition 9.2), the plaintext of its created note ("The note plaintext", §10.1), the outgoing viewing key under which its C_{out} is formed, or $\perp$, and whether each side is real or a dummy. The *privacy preconditions* are:

- (P1) every key involved, namely the key of each consumed note, of each recipient, and the key whose ovk the sender uses, is generated honestly with $\text{use_qsk} = \text{false}$;
- (P2) the bundle is an Ironwood-pool bundle generated by the procedure of "Construction of a transaction" (§11.5): the seed rseed of each created note fresh and uniform, the trap-door rcv uniform, the randomiser α drawn by GenRandom, each real consumed note a note of the pool that the procedure of "Trial decryption and note acceptance" (§10.4) accepted from an output of an accepted Action, and dummies as in the protocol specification, §"Dummy Notes (Orchard)": a dummy consumed note has a random spending key, value 0, ρ the x -coordinate of a uniform point, a uniform rseed and an unchecked path, and a dummy created note is an ordinary note of value 0 to an address of a freshly generated key;
- (P3) the authentication path is computed locally from public data (Lemma 5.15);

(P4) the adversary holds no viewing key of a key involved (no nk of a spender, no ivk of a recipient, no ovk of the sender), and no output is a coinbase output.

Within these preconditions the adversary may choose which notes are consumed and the recipients' addresses, and may know the consumed notes' contents.

Under (P2) the real consumed notes of one key have pairwise distinct elements ρ . An accepted note has $\rho = \text{nf}^{\text{old}}$ of the Action whose output carries it (Proposition 10.15(a)); one output yields at most one note under one ivk; and distinct Actions of the pool publish distinct nullifiers (§6.2). The experiment of the next theorem imposes this property directly.

Theorem 12.12 (Privacy of an Action). *Assume preconditions (P1) to (P4), and let two sequences of private inputs of one Ironwood-pool bundle have equal public leakage, both satisfy Definition 9.2 with the common anchor and flags, and each Action consume either a leaf of the tree at that anchor not consumed before or a dummy, and create either a note to an honestly generated address or a dummy. Under Assumptions 9.14, 2.12, 3.13, 6.2, 3.16, 2.8, 10.4, 10.5 and 7.3, with Assumption 2.22 for Lemma “Rejection in key generation” (§3.2) and the redraw of the note seed (§4.3), the public data of the two bundles (per Action cv^{net} , nf, rk, cmx, epk*, C_{enc} , C_{out} and the spend-authorisation signature; the proof and the binding signature) are computationally indistinguishable. In particular, beyond its public leakage an Action reveals nothing about its private inputs, and an Action whose spend or output side is a dummy is indistinguishable from one whose sides are real.*

Precisely, in the following experiment every efficient adversary has negligible advantage. Keys are generated as in (P1), and the adversary receives addresses of them at indices of its choice. It outputs the leaves of the Ironwood-pool tree through an anchoring block, the rest of a transaction, flags, and two specifications of n Actions with equal balancing values. A specification gives, per Action, a consumed side, either “dummy” or a note at one of these addresses whose extracted commitment is a leaf; a created side, either “dummy” or one of these addresses with a value and a memo; and the sender's ovk, that of one of the keys or $\perp$. The consumed leaves of a specification are pairwise distinct, the consumed notes of one key have pairwise distinct ρ , and the construction of (P2) yields witnesses of Definition 9.2 with the anchor and flags. A challenger builds the bundle of specification β , for a uniform bit β , by that construction and gives the adversary its public data; the adversary outputs a bit β' , and its advantage is $|\Pr[\beta' = 1 \mid \beta = 0] - \Pr[\beta' = 1 \mid \beta = 1]|$.

Proof. For $\beta \in \{0, 1\}$ let G_0^β be the experiment with specification β . Each of the games G_1^β to G_7^β below changes the probability that the adversary outputs 1 by a negligible amount (Crypto Guide, §“The hybrid argument”, Lemma “Hybrid lemma”; each game is reached in polynomially many steps, one key or one Action at a time), and G_7^0 and G_7^1 are identically distributed. Every reduction runs the experiment with the adversary and samples every value that its hop does not concern. The keys involved are those of (P1), the spending keys of the dummy consumed notes and the keys of the dummy created notes.

Game G_1 (*proof*). The aggregate proof is replaced by the output of the simulator of Assumption 9.14 on the primary inputs, which are public; the witnesses satisfy Definition 9.2 by hypothesis. The cost is the statistical distance of that assumption. From G_1 on, a witness enters the game

only through the public fields computed from its components.

Game G_2 (*keys*). Each key involved is replaced in turn, as in the preliminary step of the proof of Proposition 10.6 and hybrids H_1 to H_3 of the proof of Proposition 6.10: by a single draw without rejection (Lemma “Rejection in key generation”, §3.2; Assumptions 2.12, 2.22 and 2.8); its triple $(\text{ask}, \text{nk}, r_{\text{ivk}})$ by independent uniform elements (Lemma 2.15); and its triple $(\text{ivk}, \text{dk}, \text{ovk})$ by $(\text{Extract}_p(W), u_1, u_2)$, for a uniform point W and uniform 32-byte strings u_1 and u_2 (Assumption 3.13, whose reduction draws ask and nk itself, and the negligible event that $\text{ask} = 0$ or that the hash point of $\text{Commit}^{\text{ivk}}$ is $\perp$). The hops apply because after G_1 the trapdoor r_{ivk} enters the game only through $(\text{ivk}, \text{dk}, \text{ovk})$. From G_2 on, the components ask , nk , ivk , dk and ovk of each key are independent, and dk and ovk are uniform.

Game G_3 (*outgoing ciphertexts*). Each C_{out} is replaced by the encryption of a fixed 64-byte string under an independent uniform key, by Proposition “Confidentiality of the outgoing ciphertext” (§10.3). For $\text{ovk} \neq \perp$, the key ock is a value of the random oracle PRF^{ock} (Assumption 10.4) at an input that begins with the uniform secret ovk , which q queries of the adversary hit with probability at most $q \cdot 2^{-256}$. Distinct Actions give distinct inputs, their epk differing except with negligible probability, so each ock is uniform and encrypts once, and Assumption 10.5(a) replaces the plaintext. For $\text{ovk} = \perp$, the key is uniform by construction and Assumption 10.5(a) applies directly.

Game G_4 (*randomised keys*). Each randomiser α_i is replaced by a uniform scalar, at a cost of at most $\delta_{512} + q_b \cdot 2^{-640}$ each, $\delta_{512} < 2^{-257}$, for q_b queries to H (Lemma “Distance of GenRandom from uniform”, §7.2; Assumption 7.3). Then $y_i := \text{rsk}_i = \text{ask} + \alpha_i$ is uniform and independent of ask and of every other value, and $\text{rk}_i = [y_i] G^{\text{SpendAuth}}$ (Proposition 7.10(i)); the signature σ_i is computed under y_i . From G_4 on, no ask enters the game.

Game G_5 (*nullifiers*). For each key that consumes a note in the bundle, real or dummy, the key nk enters G_4 only through the nullifiers of those notes: G_1 removed it from the proof and G_2 from the viewing keys. As in hybrids H_4 and H_5 of the proof of Proposition 6.10, $\text{PRF}_{\text{nk}}^{\text{nfOrchard}}$ is replaced by a uniformly random function (Assumption 6.2), and each multiplier of K^{Orchard} by a uniform scalar $u_i \in \mathbb{F}_{p_{\text{Vesta}}}$, at a statistical cost below 2^{-167} each; the elements ρ of the consumed notes of one key are pairwise distinct by the definition of the experiment, and a dummy consumed note has a key of its own. Each nullifier is then $\text{Extract}_p([u_i] K^{\text{Orchard}} + \text{cm}_i)$, the x -coordinate of a uniform point whatever cm_i is, since $K^{\text{Orchard}} \neq \mathcal{O}$ except with probability $1/p_{\text{Vesta}}$ (Assumption 2.8).

Game G_6 (*ephemeral keys and note ciphertexts*). For each created note in turn, real or dummy, epk is replaced by a uniform non-identity point and C_{enc} by the encryption of a fixed 564-byte string under an independent uniform key, by steps (1) to (4) of the proof of Proposition 10.6 (Assumptions 10.4, 3.16, 10.5, 2.8 and 2.12), which keep cmx . The other public data, the data z of that proposition, depend after G_2 to G_5 on the recipient’s key only through the addresses of that key given to the adversary and the outputs to them. The reduction of step (2) of that proof embeds the exponent a as the ivk of the recipient’s key in every such address, computing its transmission key as $[t]([a] P)$ for the diversified base $[t] P$ that it programs, and computes every other output to that key from its own esk ; the proof is otherwise as written.

Game G_7 (*note commitments*). For each created note in turn, cmx is replaced by the x -coordinate of a uniform point. After G_6 the seed rseed of the note enters the game only through ψ and rcm , that is through cm : step (1) of G_6 replaced esk , and the ciphertexts no longer depend on the plaintext. The proof of Proposition 4.8 then places cm within $\epsilon_{\text{PRF}} + \delta + \epsilon_{\perp}$ of a uniform point, with

$\delta < 2^{-257}$ and $\epsilon_{\text{PRF}}, \epsilon_{\perp}$ negligible under Assumptions 2.12, 2.22 and 2.8.

Value commitments and the binding signature. In G_7^β every field other than the net value commitments and the binding signature is distributed independently of β : the nullifiers and the extracted commitments are x -coordinates of uniform points, each epk is a uniform non-identity point, the ciphertexts encrypt fixed strings under fresh keys, each rk_i and σ_i come from a fresh key y_i , the proof is simulated from the primary inputs, and the rest of the transaction is common. The net values v_i^β of specification β enter only

$$\text{cv}_i^{\text{net}} = [v_i^\beta] V^{\text{Orchard}} + [\text{rcv}_i] R^{\text{Orchard}}, \quad \text{bsk} = \sum_{i=1}^n \text{rcv}_i,$$

and, through these, the digest, the primary inputs of the simulator and bvk . Let λ be the discrete logarithm of V^{Orchard} to the base R^{Orchard} , which exists because R^{Orchard} generates the group (§8.1). The map

$$(\text{rcv}_i)_i \mapsto (\text{rcv}_i + (v_i^0 - v_i^1) \lambda)_i$$

is a bijection of $\mathbb{F}_{p_{\text{Vesta}}}^n$. It carries the net value commitments of specification 0 to those of specification 1, and it preserves $\sum_i \text{rcv}_i$, because $\sum_i v_i^0 = \sum_i v_i^1$, the common balancing value (step (6) of the construction of §11.5). The rcv_i being uniform and independent, the tuple $(\text{cv}_1^{\text{net}}, \dots, \text{cv}_n^{\text{net}}, \text{bsk})$ has the same distribution for both β , and G_7^0 and G_7^1 are identically distributed. This step is perfect and needs no reduction; the game is not required to compute λ .

The advantage of the adversary is therefore at most the sum of the costs of the hops for $\beta = 0$ and $\beta = 1$, which is negligible. In G_7 a dummy side differs from a real side only through the value in cv^{net} , so two sequences that differ in whether a side is a dummy, with equal public leakage, are covered by the same argument. $\square$

Remark 12.13 (Non-claims). Theorem 12.12 hides nothing of the public leakage: each bundle’s balancing value, so the net amount that each transaction moves across a pool’s boundary is public; each pool’s chain value pool balance, the negation of the sum of its balancing values (“Chain state and pool rules”, §11.4), so a pool hides which notes are held by whom, never the aggregate value it holds; the number of Actions and the presence of each component; the flags; the transparent components; the fee; the anchor, which confines the consumed note to the leaves of the anchored tree (“Authentication paths”, §5.3); and the transaction’s timing and expiry height. Per Action the chain gains one nullifier and one extracted note commitment, which by the theorem reveal neither sender, recipient nor amount. The theorem claims nothing for coinbase outputs, whose outgoing ciphertexts decrypt under the all-zero ovk (§11.4); nothing against a holder of a viewing key of a key involved, whose powers are stated in “Nullifier uniqueness and unlinkability” (§6.4), “The outgoing ciphertext” (§10.3) and “Trial decryption and note acceptance” (§10.4); nothing for an Orchard-pool bundle, whose created notes use lead byte $0x02$ and a derivation on which the volume does not rely (§10.4); and nothing about linkage through information outside the transaction, such as an address given to several parties, voluntary disclosures, or wallet and network metadata.

12.5 The end-to-end theorem

Theorem 12.14 (End-to-end security). *Assume, listed once: Assumption 2.22; Assumptions 9.11 and 9.14, with Assumption 9.10 in the supporting analysis of Assumption 9.11; Assumption 3.16; Assumptions 2.12 and 6.2; Assumption 3.13; Assumptions 10.4 and 10.5; Assumption 11.9; and Assumptions 2.8 and 7.3, for GroupHash and H as random oracles. Clauses (iv) and (v) are key-derived and are claimed for keys generated with `use_qsk = false`; clauses (i) to (iii) use no property of key generation. For each pool, no efficient adversary achieves any of the following, except with negligible probability:*

- (i) *an accepted Action for which the extractor of Assumption 9.11 outputs no witness satisfying Definition 9.2; or a second opening, other than its witnessed created note, of the cmx of an accepted Action; or an accepted Action whose witnessed v^{old} is non-zero and whose consumed note is not a committed note of the pool, a leaf of the tree at the Action's anchor (membership soundness) [R1 and R2 soundness];*
- (ii) *two accepted Actions of the pool consuming one committed note of non-zero value [R3 soundness];*
- (iii) *an accepted bundle whose balancing value differs, as an integer, from the sum of its Actions' $v^{\text{old}} - v^{\text{new}}$ [R4 soundness];*
- (iv) *winning the spend-authority experiment of "Authorisation of spends", that is, an accepted Action consuming a note of an honestly generated key without a signature of the key holder, the adversary holding the key's full viewing key but not its ask; or an accepted transaction that changes the effecting data of a transaction T while retaining one of its Actions, where the Actions of T are authorised by holders of honestly generated keys who sign as hypothesis (S) of Proposition 11.10 states [R5];*
- (v) *for Ironwood-pool bundles, distinguishing the public data of two bundles with equal public leakage under preconditions (P1) to (P4) of "Privacy", in particular a real Action from a dummy one [R1 hiding; R2 hiding, without identifying the note; R3 hiding, without a public list of consumed notes; R4 hiding, without disclosed amounts].*

The bracketed tags refer to the requirement map of "The Action statement" (§9.2, Table 5), which is not repeated.

Proof. Each clause is a result of this section or of an earlier one.

(i) The first part is Assumption 9.11, applied to each bundle as stated after Definition "Pools, accepted Actions and witnesses", with Remark " $\perp$ -weakened conditions" (§9.2) for the unweakened forms of A1, A2, A3 and A7. By A2 the created note opens cmx, and by Remark "Binding for notes given by expanded receivers" it is the only opening that an efficient algorithm finds. For $v^{\text{old}} \neq 0$, condition A3, outside its $\perp$ case, makes the fold of A3 on $(\text{Extract}_p(\text{cm}^{\text{old}}), \text{pos}, \text{path})$, with the witnessed encodings c, reach the bundle's anchor rt , and the anchor rule, step (3) of the verification of §11.6, makes rt the anchor of an earlier block in the pool. By A1, cm^{old} is a point, so Proposition 5.9(iii), which applies to that fold (§9.2), gives, under Assumptions 2.22 and 2.8,

that $\text{Extract}_p(\text{cm}^{\text{old}})$ is the leaf appended at position pos , in the anchoring block or earlier: the consumed note is a committed note of the pool.

(ii) Theorem 12.6.

(iii) Theorem 12.4, which adds Assumption 7.3.

(iv) The first part is Theorem 12.9. The second is Proposition 11.10, under Assumptions 11.9, 2.22 and 7.3; its hypothesis (H) holds for honestly generated keys up to a negligible change in the probability of every efficiently decidable event, the event of that proposition included, under Assumptions 2.12, 2.22 and 2.8 (Remark “Hypothesis (H) for honest keys”, §11.3).

(v) Theorem 12.12.

Clauses (i) to (iii) use Assumptions 9.11, 2.22, 2.8 and 7.3 only, and no property of key generation; clauses (iv) and (v) use honest key generation with $\text{use_qsk} = \text{false}$. Each assumption of the list is used by some clause, and Assumption 9.10 enters only through the supporting analysis of Assumption 9.11 (“The Action circuit and the Halo 2 proof”, §9.3). $\square$

Remark 12.15 (Nature of the guarantees). Beyond Assumption 9.11, every soundness reduction ends in a discrete-logarithm problem on Pallas: either a non-trivial relation among values of GroupHash, modelled as a random oracle (Assumption 2.8), namely Sinsemilla collisions and $\perp$ outputs (Proposition 2.24), with membership soundness (Proposition 5.9), the binding of NoteCommit and of $\text{Commit}^{\text{ivk}}$, nullifier uniqueness (Proposition 6.8) and the independence of V^{Orchard} and R^{Orchard} (Proposition 8.10); or the discrete logarithm of a given point, in the spend-authority reduction through the RedPallas extraction of Proposition 7.12. Every extraction from a RedPallas signature, the binding signature included, holds in the random-oracle model for H (Assumption 7.3). The digest step of bundle binding ends instead in a BLAKE2b collision (Assumption 11.9). Soundness is therefore computational and conditional on Assumption 9.11, which “The Action circuit and the Halo 2 proof” (§9.3) classifies as designed but unspecified, with discrete logarithms on Vesta entering only its supporting analysis; no concrete bound is asserted, since that assumption carries no loss factor. In clause (v), the contribution of the proof is statistical (Assumption 9.14; *Halo 2 Guide*, §“Zero knowledge: hiding the witness in Halo 2”); that of the value commitments with the binding signature is perfect (the final step of the proof of Theorem 12.12); that of rk and the spend-authorisation signatures is statistical in the random-oracle model for H (game G_4 there); and the unlinkability of every other published field is computational. Assumption 6.2 is cryptanalytic, with no reduction to a standard problem.